Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

PickleVerse – Multiplayer Pickleball Skill-Training and Match Game with AI Coaching

Project code: PKV Group: GFA26SE111

Supervisor: Mr. Lại Đức Hùng

Team: Huỳnh Trung Kiên (Leader), Nguyễn Văn Thành Đạt,

Nguyễn Gia Huy, Trần Nguyễn Hiếu Thuận

– Ho Chi Minh City, November 2026 –

Table of Contents

I. Record of Changes 4
II. Software Design Document 4
1. System Design 4
1.1 System Architecture 4
1.2 Package Diagram 7
1.3 Technology Stack 11
1.4 Architecture Decisions 11
2. Database Design 12
2.1 Data Dictionary of the Core Tables 22
2.2 Design Notes 26
3. Detailed Design 26
3.1 Account and Authentication 26
3.2 Gameplay Core and Ball Physics 32
3.3 Training and Offline Synchronisation 34
3.4 Bot AI 39
3.5 Online Matchmaking and Netcode 42
3.6 AI Coaching 49
3.7 Progress, Economy, Social and Replays 54
3.8 Administration and Moderation 56
3.9 API Design 60
3.10 Security Design 63
4. Workflows and State Changes 63
4.1 Activity Diagrams 63
4.2 State Machine Diagrams 69

I. Record of Changes

Date	A* M, D	In charge	Change Description
27/10/2026	A	Nguyễn Gia Huy	v0.1 – Architecture, workflows W01–W04 and state machines summarised for the Review 2 design brief (BM01).
06/11/2026	A	Nguyễn Văn Thành Đạt	v0.2 – First SDD on the official template: architecture, packages, database groups, 7 class/sequence pairs, activities and state machines.
11/11/2026	M	Nguyễn Gia Huy	v1.0 – Rewritten after the internal review: deployment diagram and architecture decisions; full package diagrams; logical ERDs covering all 51 tables with keys and a data dictionary; UML class diagrams with class specifications; 15 sequence diagrams with alternative paths; new sections for gameplay physics, netcode, bot AI, API and security design; activity diagrams with swimlanes; 14 corrected state machines.

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

PickleVerse uses a client–server architecture with five deployable sub-systems (Figure 1). The Unity game client contains the presentation, the local gameplay simulation used for training and bot matches, and an offline store. All persistent data and business rules that must be trusted are on the server side: the ASP.NET Core Web API owns accounts, progression, economy, matchmaking and moderation; the Unity dedicated server owns the state of online matches; the AI coaching service analyses shot telemetry; the React administration application is a pure client of the Web API. Background workers run inside the Web

API process. PostgreSQL is the system of record, Redis holds short-lived coordination data, and object storage keeps replays.

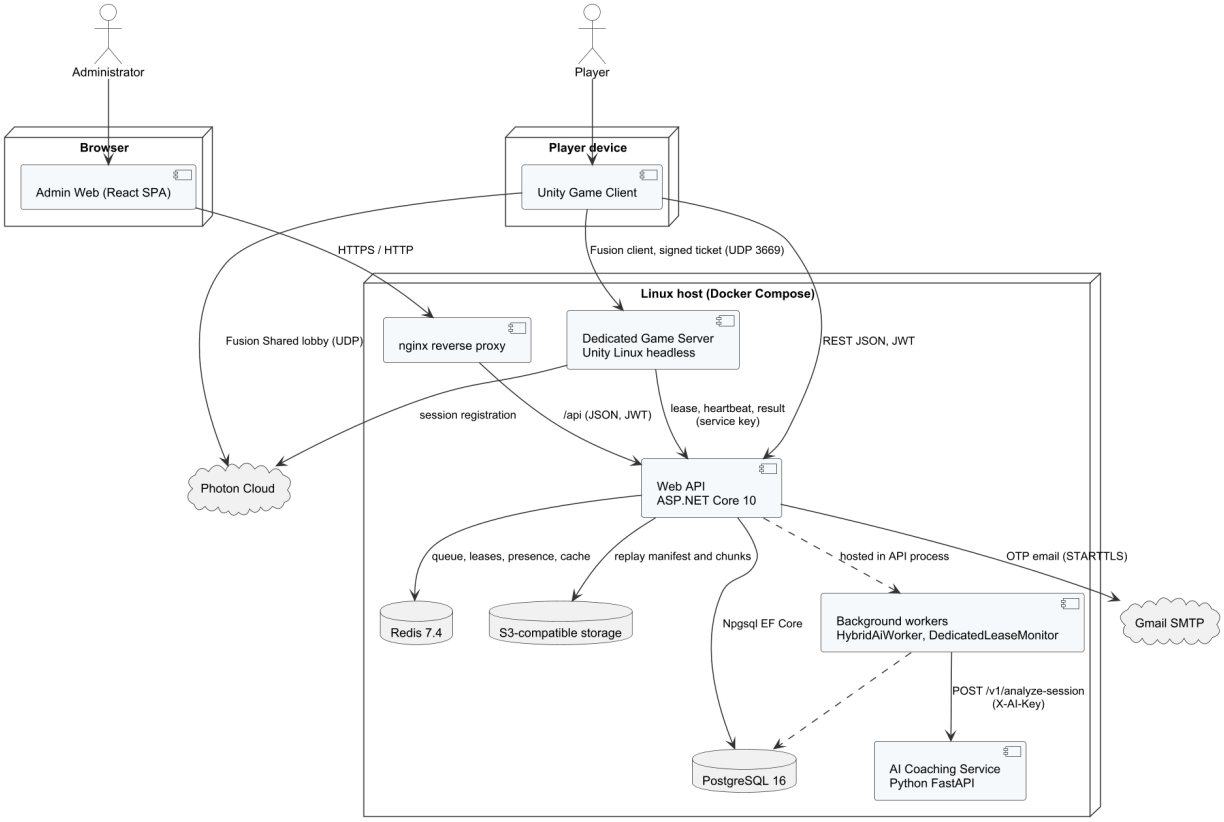


Figure 1. System architecture (component view)

Table 1. Architecture components

Component	Technology	Responsibility
Unity game client	Unity 6000.6.2f1, C#, uGUI, Input System	Scenes and UI, local physics and rules for training and bot matches, Photon Fusion client, offline training store, REST client (ApiClient) to the Web API
Dedicated game server	Unity Linux headless build, Photon Fusion 2.1.3 (server mode)	Authoritative simulation of online matches; validates signed tickets; sends heartbeats, telemetry, replay and result to the Web API
Web API	ASP.NET Core (.NET 10), layered API → Services → Repositories → Entities	REST endpoints, JWT authentication, business rules, transactions through IUnitOfWork and Npgsql EF Core
Background workers	Hosted services in the Web API	HybridAiWorker (analysis jobs), DedicatedLeaseMonitor (expired leases), ModerationExpiryWorker (expired suspensions), AnalyticsProjectionWorker (daily aggregates), PasswordRecoveryMailQueue (e-mail sending)
AI coaching service	Python 3.12, FastAPI, scikit-learn	Rule engine for technique errors, summary and drill recommendations, gated tier model; internal API protected by an API key
Admin web application	React 18, TypeScript 5, Vite 5, React Query, Axios	Administration screens served by nginx; calls the Web API with the administrator's JWT
PostgreSQL 16	Relational database	51 tables: accounts, profiles, content, matches, economy, moderation, telemetry and coaching
Redis 7.4	In-memory store	Matchmaking queue and search state, dedicated-server lease, presence, leaderboard cache (15 s)

Component	Technology	Responsibility
Object storage	S3-compatible container	Replay manifests and compressed chunks
Photon Cloud	External service	Fusion session lookup and relay for clients and the dedicated server
Gmail SMTP	External service	Delivery of verification and password-reset e-mails

Online matches support two authority modes recorded in GameMatches.AuthorityMode. In **dedicated** mode, enabled by the configuration flag DedicatedServer:Enabled (true on staging), the dedicated server runs the simulation and is the only authority for points, faults and the result. In **shared** mode, kept for local development without a server, the room host runs the simulation through Photon Fusion shared mode. The staging deployment has one dedicated-server container, so one authoritative match runs at a time; the API reports the slot as busy and the client waits (Report 3, MSG36). Scaling to more matches requires more server containers and is outside release 1.0.

Deployment

Figure 2 shows the staging deployment on the host tad-min.io.vn (Ubuntu 24.04, Docker Engine). GitHub Actions validates each push to main, publishes images to Docker Hub and deploys them over SSH with deploy.sh, which pulls images by digest and restarts the Docker Compose stack. Only the ports listed in Table 2 are reachable from the Internet.

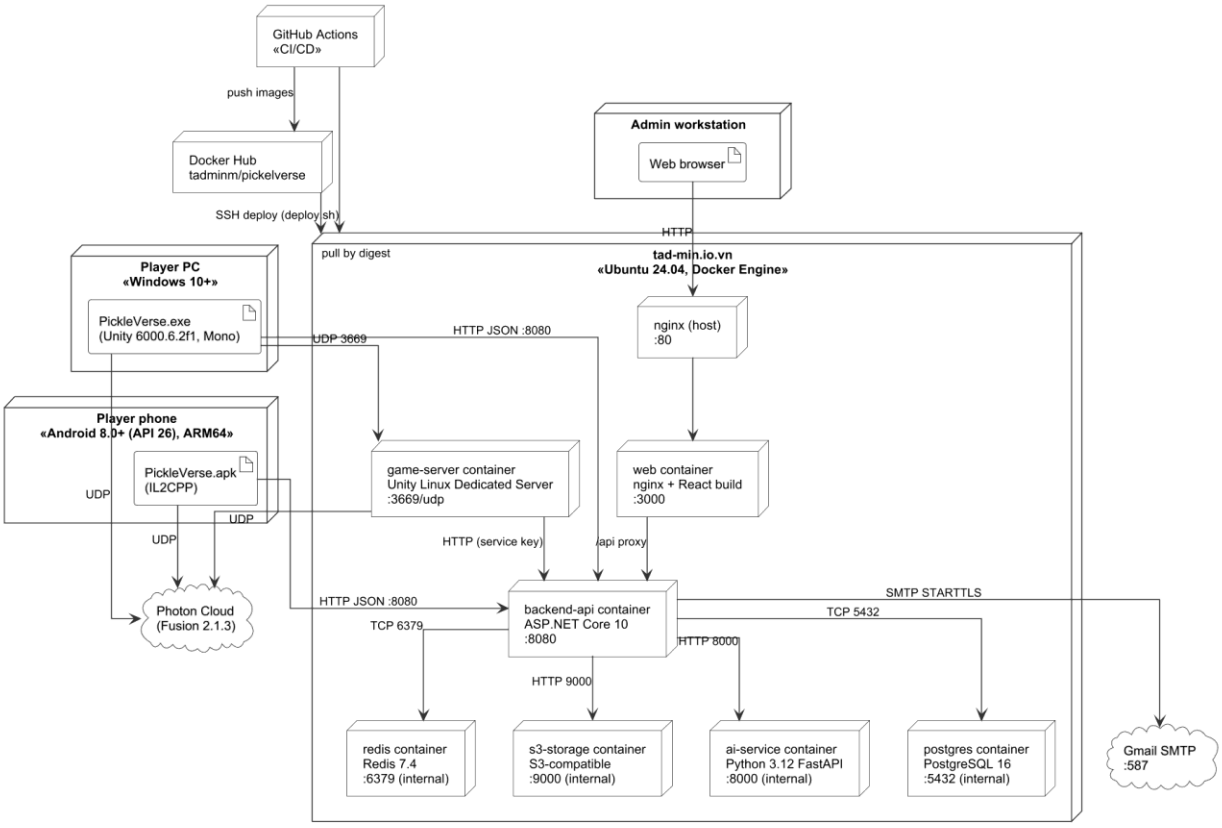


Figure 2. Deployment diagram of the staging environment

Table 2. Deployment nodes

Node / container	Image or artifact	Port	Notes
Host nginx	Ubuntu package	80/tcp	Reverse proxy to the web container; TLS termination is planned for production (port 443)
web	tadminm/pickelverse:web-<release>	3000/tcp (internal)	nginx serving the React build; proxies /api to backend-api
backend-api	tadminm/pickelverse:api-<release>	8080/tcp	Web API and background workers
game-server	tadminm/pickelverse:server-<release>	3669/udp	Unity dedicated server; limited to 2 CPUs and 2 GB RAM
ai-service	tadminm/pickelverse:ai-<release>	8000/tcp (internal)	FastAPI coaching service, reachable only from backend-api
postgres	PostgreSQL 16 (pinned digest)	5432/tcp (internal)	Volume for data; schema and seed from src/Database.sql
redis	Redis 7.4 (pinned digest)	6379/tcp (internal)	Coordination data; no persistence required
s3-storage	S3-compatible server (pinned digest)	9000/tcp (internal)	Bucket pickleverse for replays
Player PC / phone	PickleVerse.exe (Windows, Mono) / PickleVerse.apk (Android 8.0+, IL2CPP, ARM64)	–	Connects to the API, Photon Cloud and the game server

1.2 Package Diagram

The three code bases are organised in layers. Dependencies point only downwards: a package may use the packages below it but never the reverse. Figures 3 to 5 show the packages of the Web API, the Unity client and the admin web with the AI service.

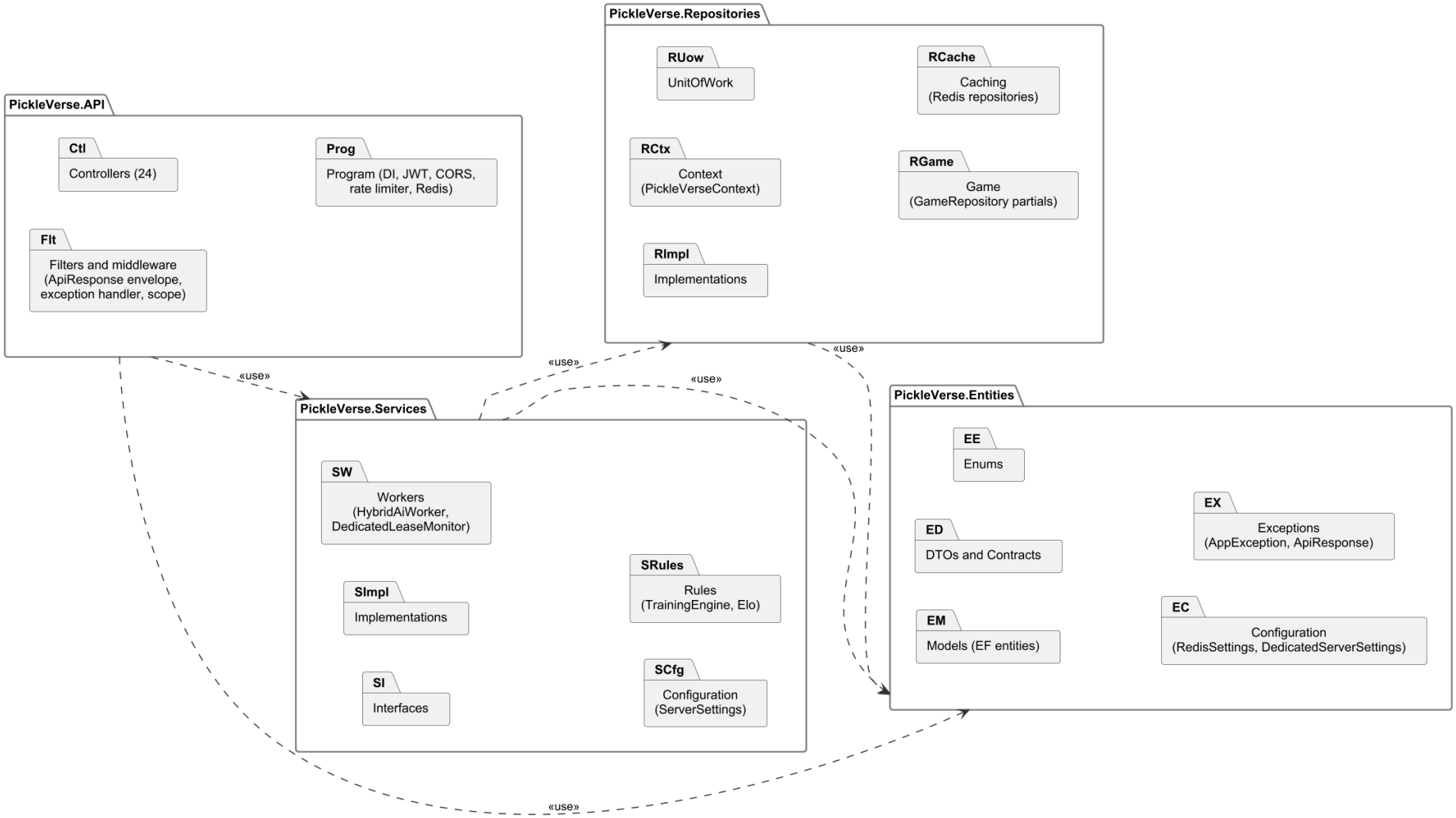


Figure 3. Package diagram of the Web API solution

Table 3. Web API package descriptions

No	Package	Description
01	PickleVerse.API	Entry point: 24 controllers, Program.cs (dependency injection, JWT, CORS, rate limiter, Redis), filters that wrap responses in ApiResponse<T>, the global exception handler that maps service exceptions to HTTP status codes
02	PickleVerse.Services	Business logic: service interfaces (Interfaces) and implementations (Implementations), background workers, typed settings (ServerSettings) and pure rule classes (TrainingEngine, ProgressionEngine, EloPolicy)
03	PickleVerse.Repositories	Data access: PickleVerseContext (Npgsql EF Core mapping of the 51 tables), IUnitOfWork, repositories, GameRepository partial classes for match and progress persistence, Redis repositories for queues, leases and caches
04	PickleVerse.Entities	Shared model: EF entity classes, DTOs and contracts, enums, configuration models (RedisSettings, DedicatedServerSettings) and exception/response types (AppException, ApiResponse)

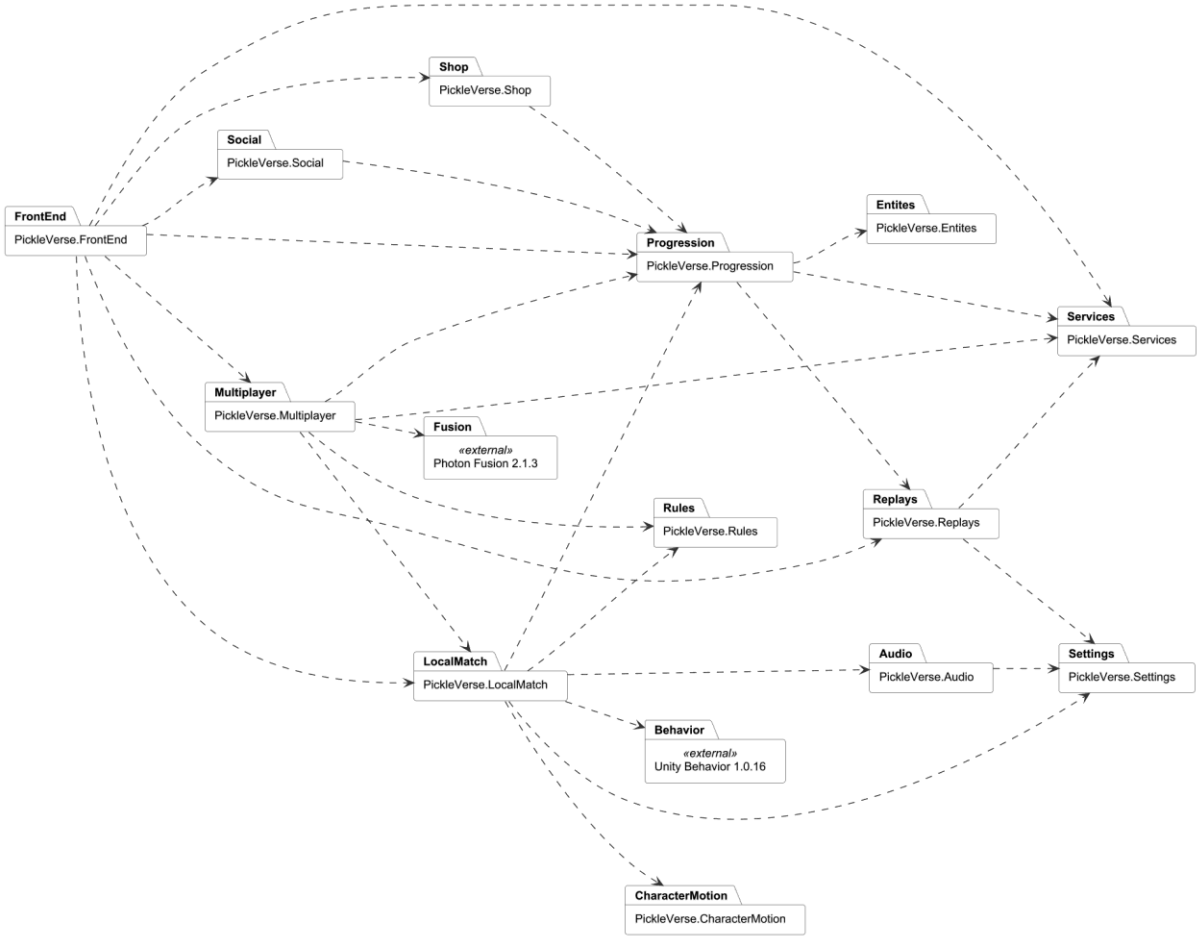


Figure 4. Package diagram of the Unity client (assembly definitions)

Table 4. Unity client package descriptions

No	Package	Description
01	PickleVerse.FrontEnd	Menus and screens: login, loading, menu, mode selection, settings entry, overlays, history and missions popups
02	PickleVerse.Multiplayer	Online flow: MultiplayerApp, PhotonSession, room state, NetworkMatch and NetworkPlayer, dedicated-server client and reconnection

No	Package	Description
03	PickleVerse.LocalMatch	Local gameplay: BotMatch, MatchActor, ball flight, paddle contact, shot input, LocalBot and BhTreeBot, training and tutorial directors
04	PickleVerse.Rules	Engine-independent rules shared by local and online play (scoring, faults, court geometry)
05	PickleVerse.Progression	ProgressionRuntime, training engine on the client, offline training store, wallet and profile cache
06	PickleVerse.Services	ApiClient, AccountService and backend settings
07	PickleVerse.Entites	Data contracts shared by client assemblies (room models, DTOs); the name keeps the original spelling of the assembly
08	PickleVerse.Shop	Shop and collection scenes
09	PickleVerse.Social	Friends, blocking and leaderboard UI
10	PickleVerse.Replays	Replay recording, upload and the replay viewer
11	PickleVerse.Settings	Settings scene: audio, graphics, key binding and touch layout
12	PickleVerse.Audio	Music player and sound effects policy
13	PickleVerse.CharacterMotion	Character animation controller and movement blending

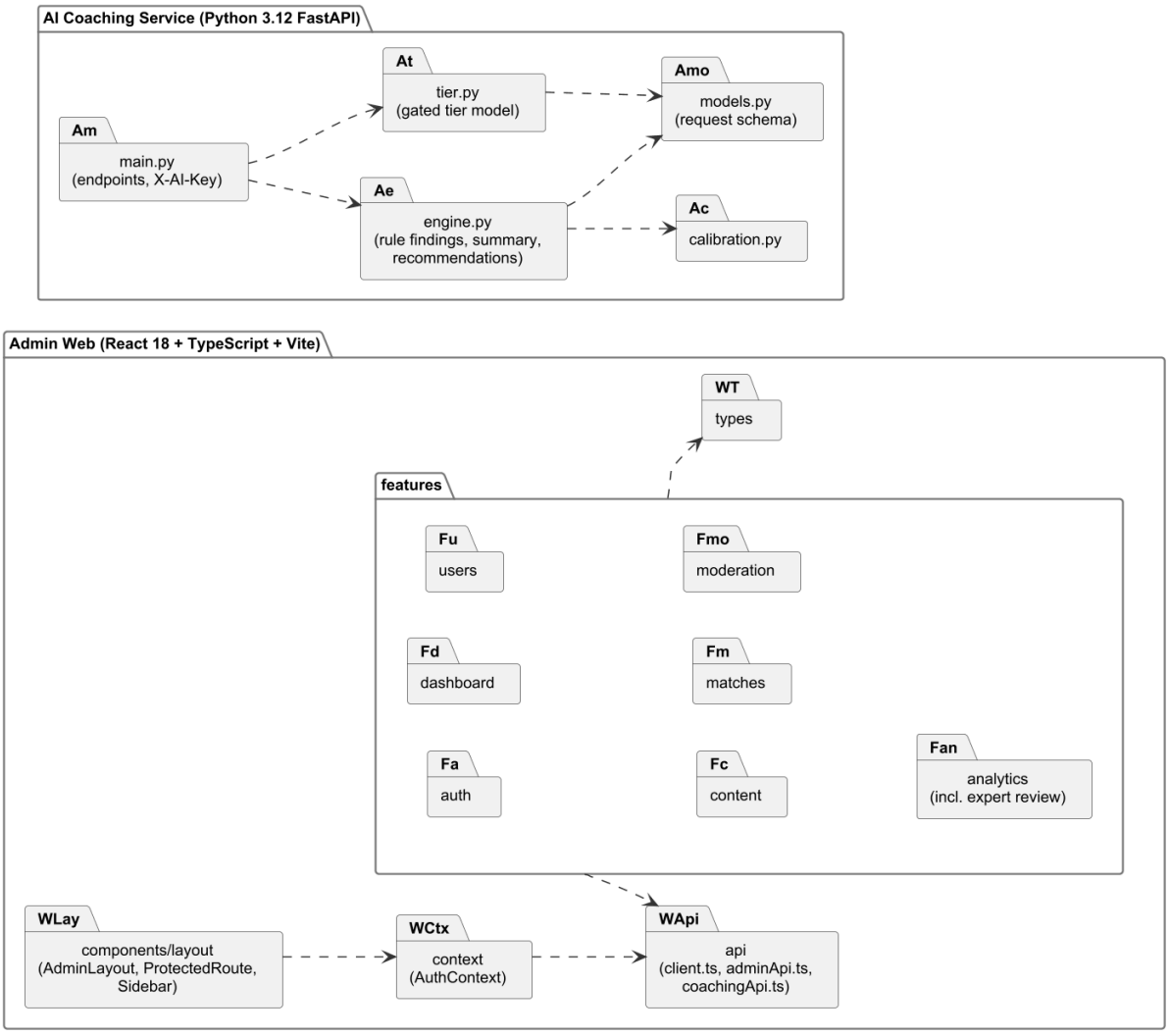


Figure 5. Package diagram of the admin web application and the AI coaching service

Table 5. Admin web and AI service package descriptions

No	Package	Description
01	features (admin web)	One folder per area: auth, dashboard, users, content, matches, moderation, analytics (including expert review); each holds its pages
02	api	Axios client with the JWT interceptor (client.ts) and typed API modules (adminApi.ts, coachingApi.ts)
03	context	AuthContext: stores the session and exposes sign-in and sign-out
04	components/layout	AdminLayout, Sidebar and ProtectedRoute, which redirects users without the Admin role to the login page
05	main.py (AI service)	FastAPI application, request validation and the X-AI-Key check
06	engine.py	Rule-based findings for 15 error codes, summary and at most three recommendations
07	tier.py	Gated tier model: 8 features, minimum confidence 0.65, minimum 20 opportunities
08	calibration.py / models.py	Threshold calibration against expert labels; request and response schemas

1.3 Technology Stack

Table 6. Technology stack

Layer	Technology and version	Purpose
Game client	Unity 6000.6.2f1, C#, uGUI, Input System, Unity Behavior 1.0.16, Photon Fusion 2.1.3	Gameplay, UI, bot behaviour graph, networking
Game server	Unity 6000.6.2f1 Linux dedicated server build	Authoritative online matches
Web API	ASP.NET Core on .NET 10, Npgsql EF Core 10, StackExchange.Redis 3.4, BCrypt.Net-Next 4.2, MailKit 4.17, AWSSDK.S3 4.0, JwtBearer 10	REST API, persistence, caching, password hashing, e-mail
AI service	Python 3.12, FastAPI, Pydantic, scikit-learn	Coaching analysis and tier model
Admin web	React 18, TypeScript 5, Vite 5, TanStack React Query 5, Axios 1.6, Recharts 2, Tailwind CSS 3	Administration UI
Data	PostgreSQL 16, Redis 7.4, S3-compatible storage	Persistence, coordination, replays
DevOps	Docker, Docker Compose, GitHub Actions, Docker Hub, nginx, Ubuntu 24.04	Build, delivery and hosting

1.4 Architecture Decisions

Table 7 records the main decisions that differ from the registered proposal, the alternatives considered and their consequences. Report 1 (LI-08) refers to this table.

Table 7. Architecture decisions

#	Decision	Alternatives	Reason	Consequence
1	ASP.NET Core Web API (.NET 10)	Spring Boot; FastAPI	The team writes C# for Unity; sharing the language reduces context switching and lets DTOs be checked against the Unity client in IntegrationChecks	The AI service stays in Python; two runtimes on the server
2	Photon Fusion 2 with a dedicated server	Netcode for GameObjects with Relay (prototype 07–21/10/2026); Mirror	The NGO prototype showed unstable relayed connections; Fusion provides lag compensation, input authority and a server mode	Dependency on Photon Cloud and its CCU plan (Report 2, risk 4)

#	Decision	Alternatives	Reason	Consequence
3	Custom fixed-step ball simulation (120 Hz)	Unity PhysX rigidbody	Deterministic, engine-independent flight and bounce that the server and replays can reproduce exactly	Collision with paddle and net is implemented by the team (sphere sweep against a contact box)
4	uGUI for the game interface	UI Toolkit	Existing art assets and world-space panels were built with uGUI; better support for runtime animation	Two UI systems are not mixed; layout code is in C#
5	Server-authoritative results and rewards	Client-reported results	Prevents cheating on Elo, currency and missions	Bot matches are reported by the client; only Elo-neutral practice uses this path except Ranked Bot, whose result is validated against the server-created match
6	Single schema file (src/Database.sql) without migrations during development	EF Core migrations	The schema changed often; one reviewed file is the single source for schema and seed data	Database resets are allowed only on the development instance (AGENTS.md)
7	Gated AI tier model with rule fallback	Always show the model output	Too little labelled real data; a wrong tier would mislead beginners	The tier is hidden until the acceptance criteria of NFR-06 are met

2. Database Design

The logical database design is derived directly from src/Database.sql (51 tables, 366 columns, 71 foreign keys). To keep the diagrams readable the schema is divided into seven subject areas (Figures 6 to 12). Each diagram uses crow's-foot notation: * marks NOT NULL columns, «PK» and «FK» mark keys, and tables drawn without columns belong to another area and are shown only for their relationships. Table names follow the PascalCase convention of the EF model; the AI tables (ai*) and ratingledger use lower case because they were created for the Python service and the rating ledger and are mapped explicitly in PickleVerseContext.

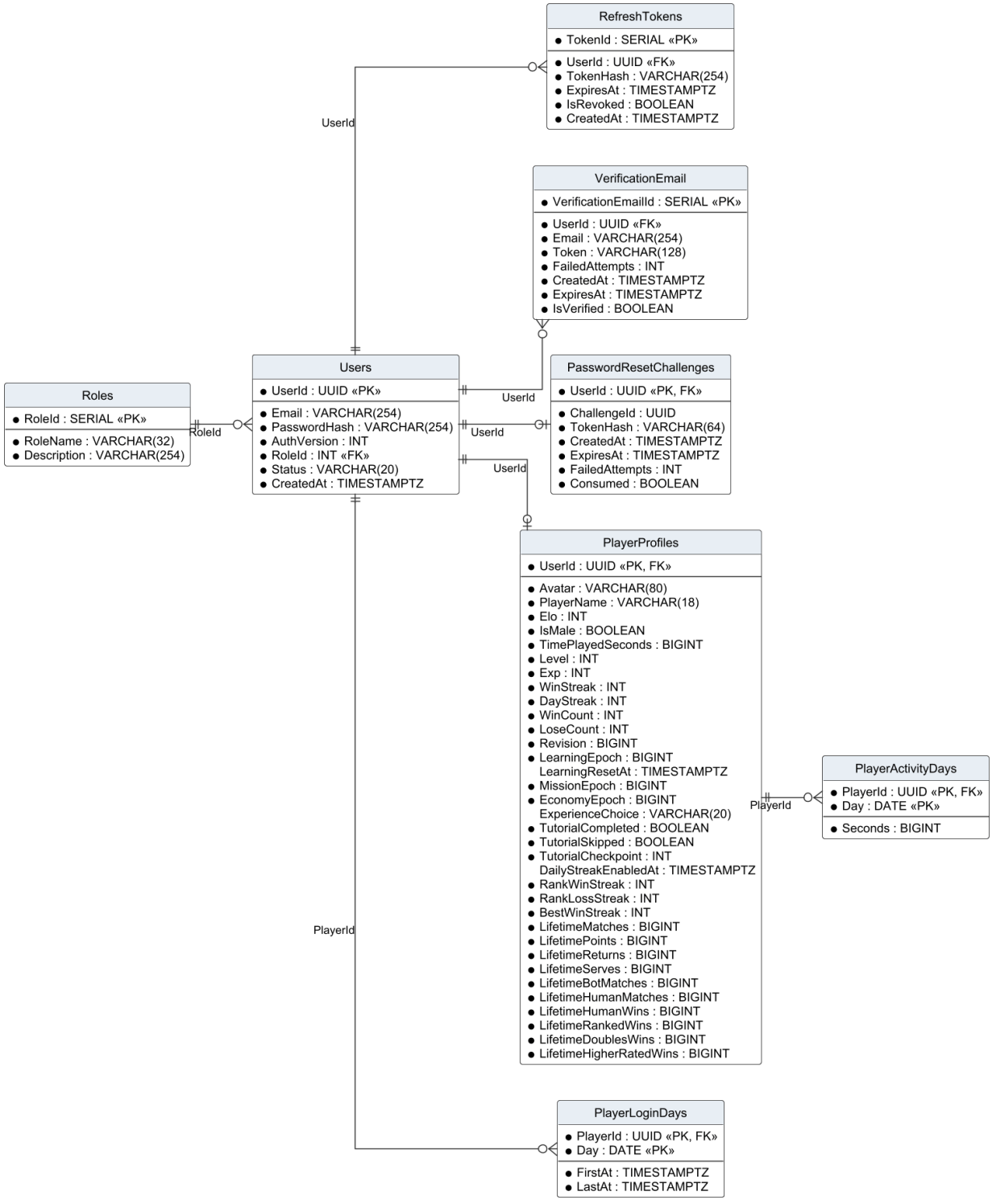


Figure 6. Logical ERD – accounts and profiles

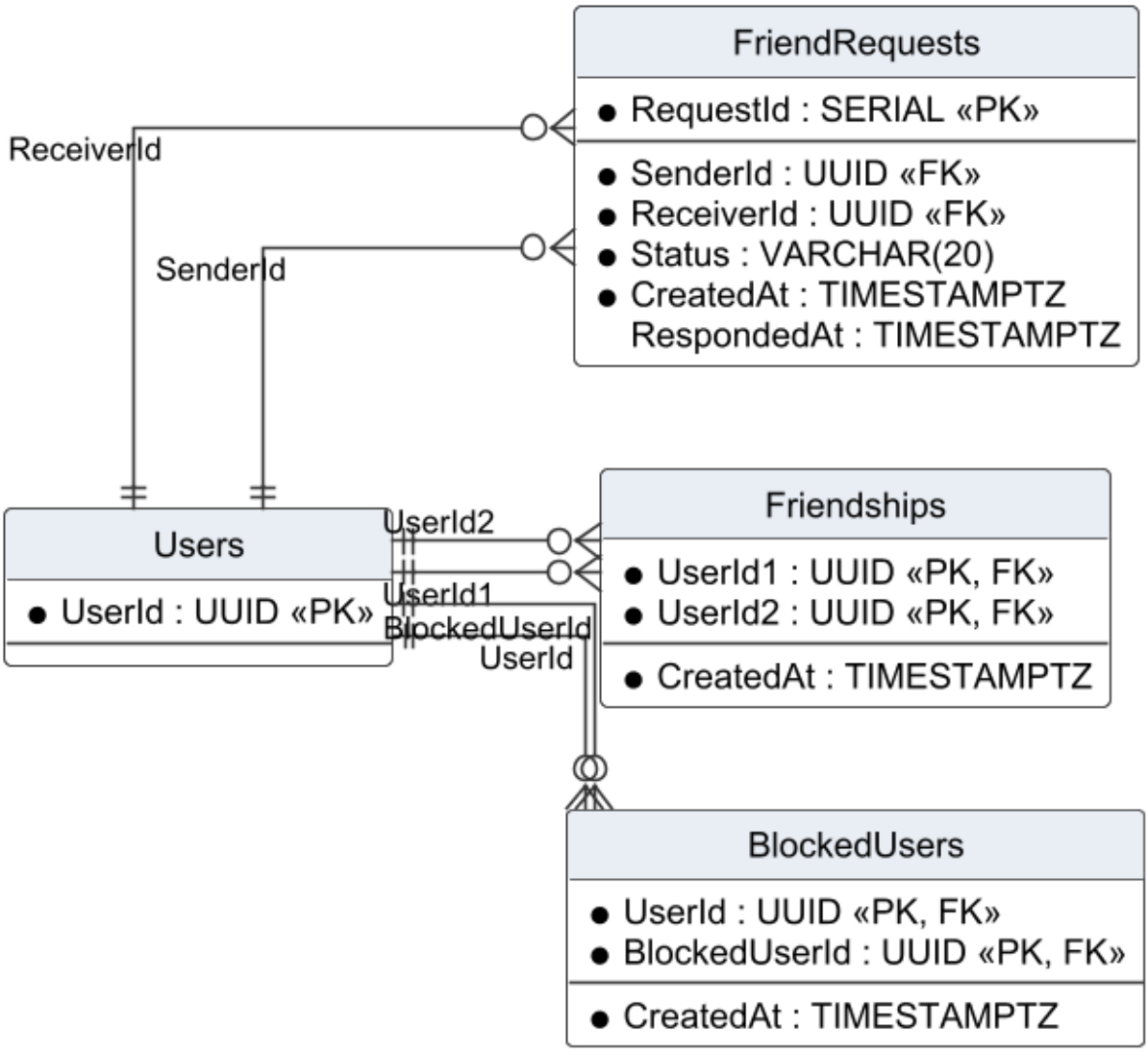


Figure 7. Logical ERD – friends and blocking

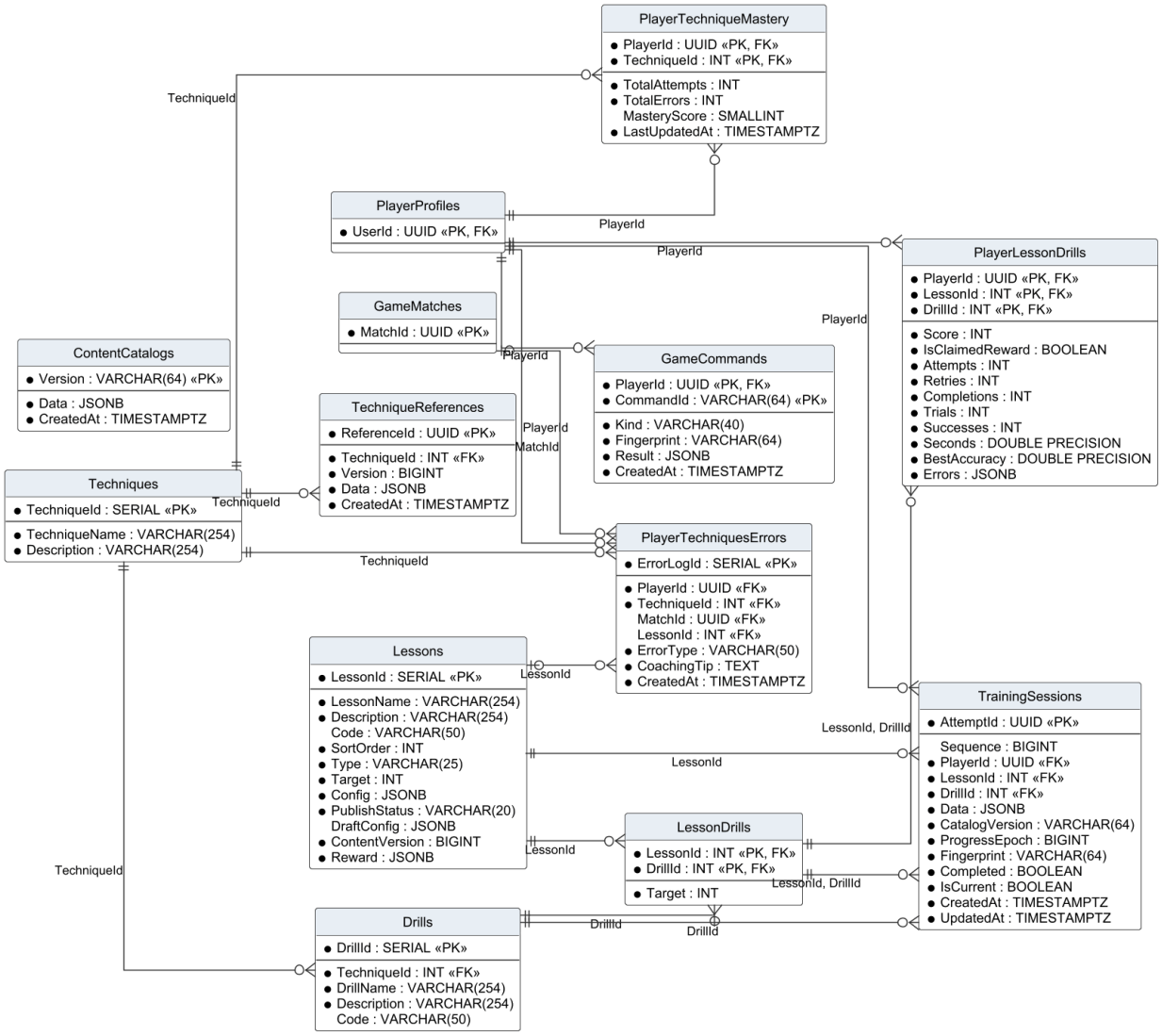


Figure 8. Logical ERD – techniques, lessons and training

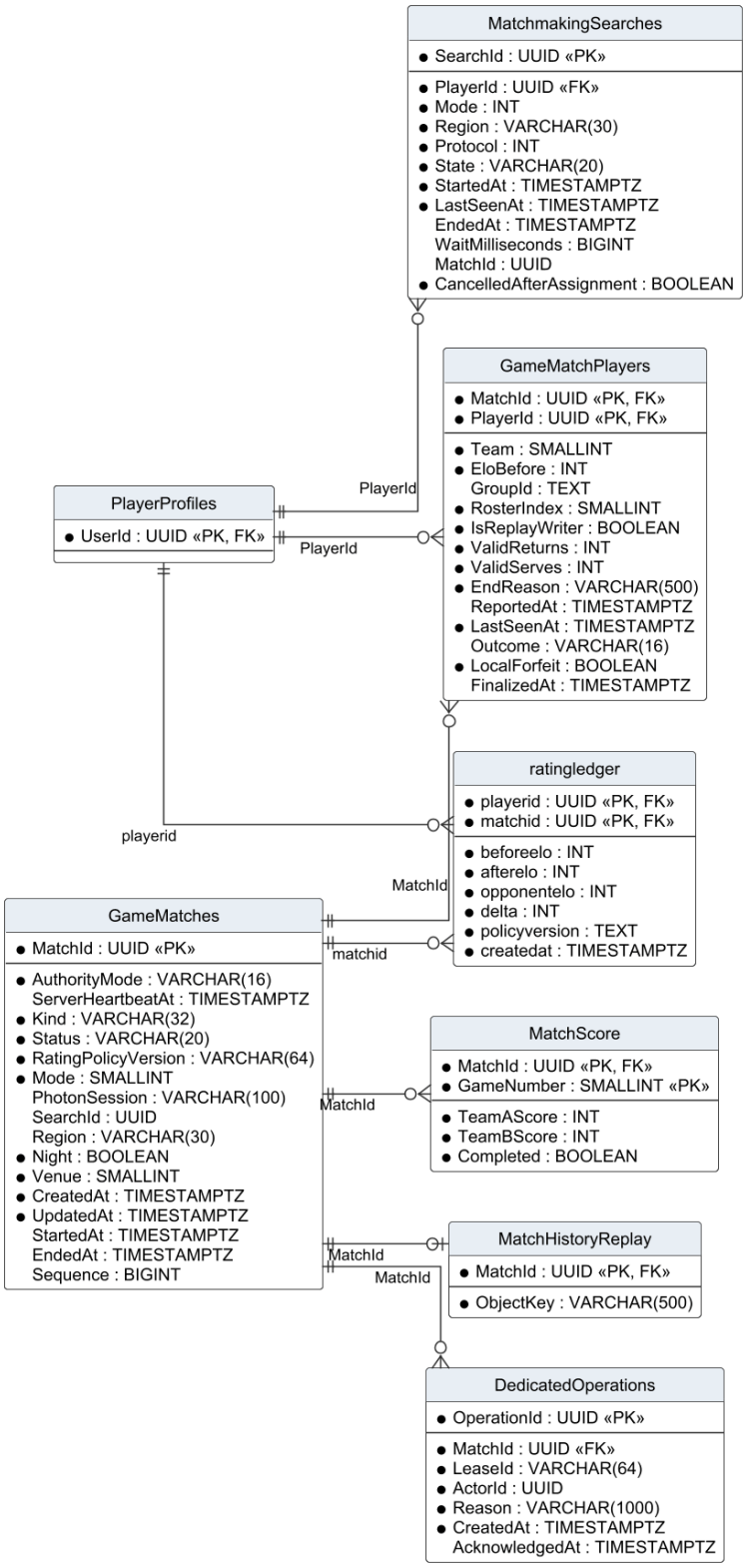


Figure 9. Logical ERD – matches, ratings, replays and matchmaking

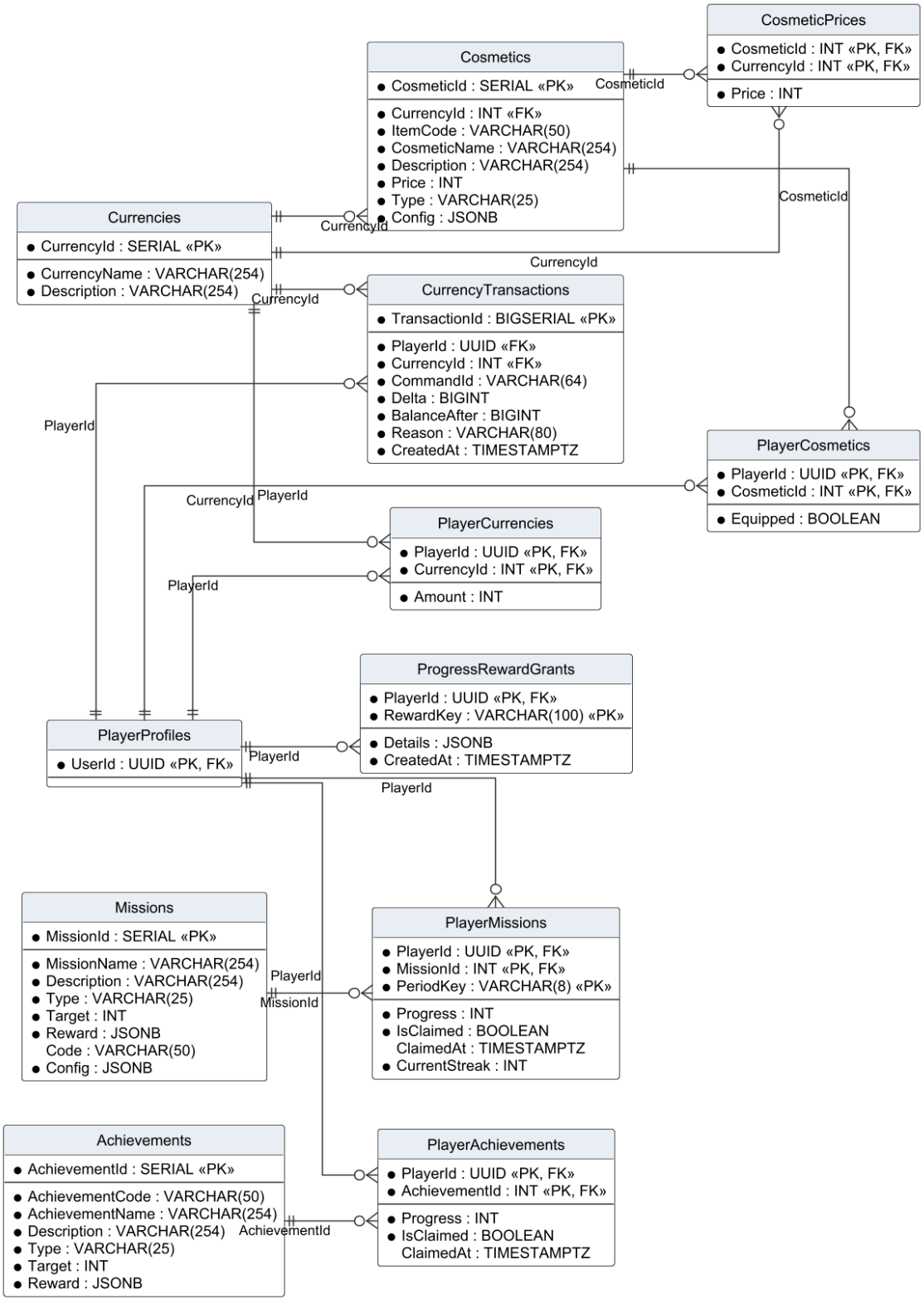


Figure 10. Logical ERD – currencies, cosmetics, missions and achievements

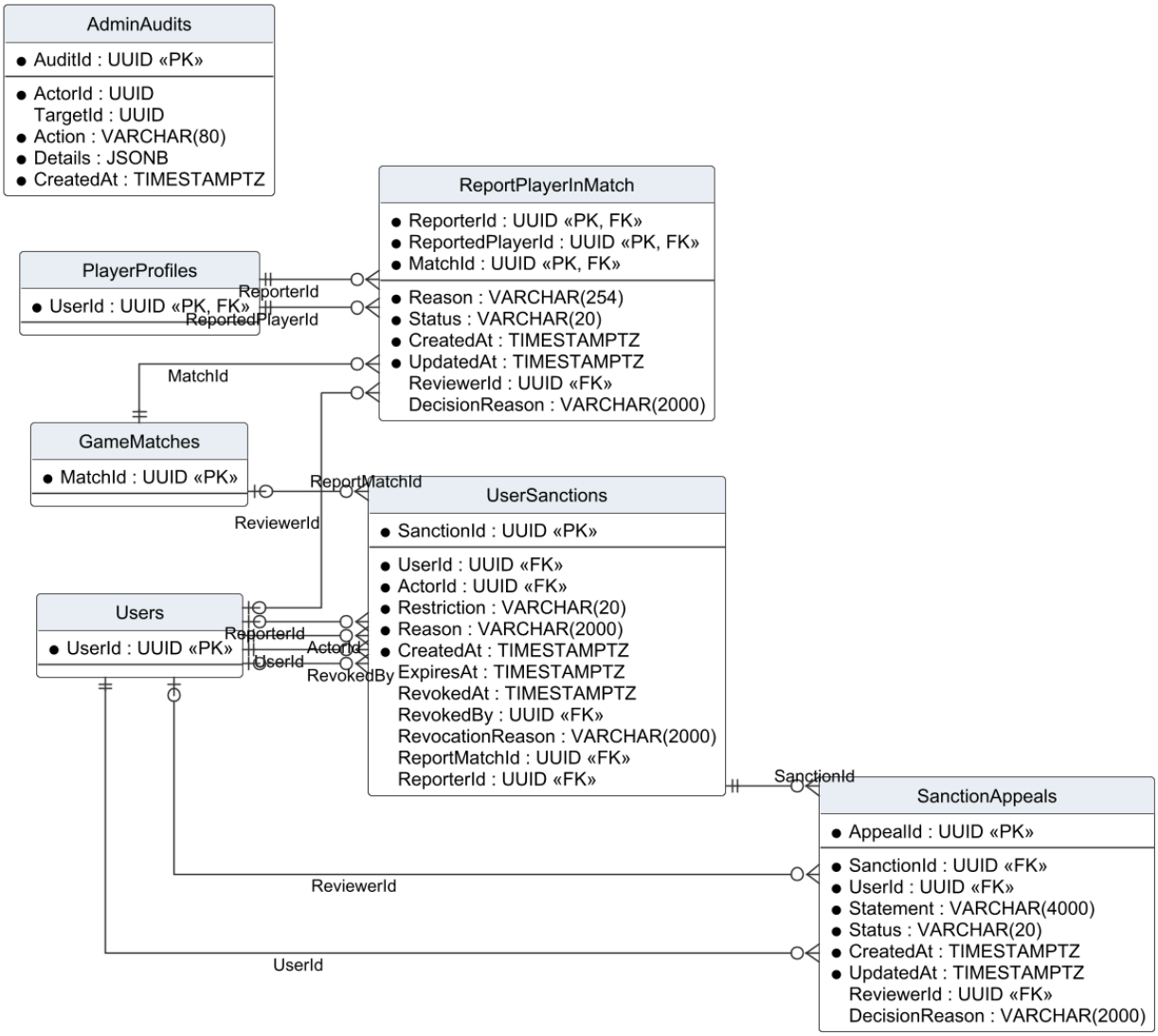


Figure 11. Logical ERD – reports, sanctions, appeals and audit

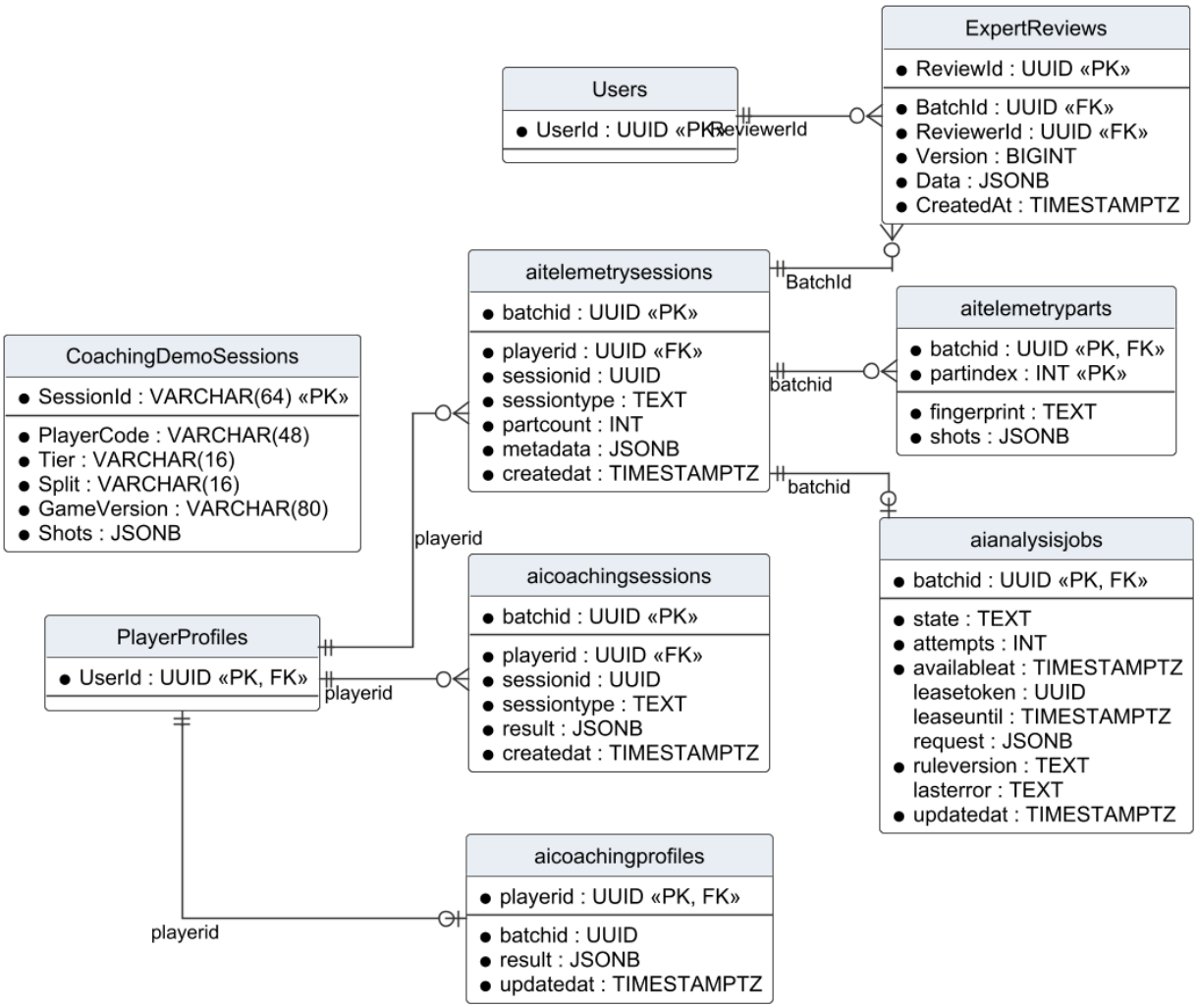


Figure 12. Logical ERD – telemetry, coaching and expert review

Table Descriptions

Table 8. Table descriptions (51 tables)

No	Table	Description
01	Roles	Role catalogue (Player, Admin). - Primary keys: RoleId - Foreign keys: none
02	Users	Account: e-mail, BCrypt password hash, role, status (ACTIVE, SUSPENDED, BANNED), e-mail verification flag and auth version used to revoke tokens. - Primary keys: UserId - Foreign keys: RoleId → Roles
03	RefreshTokens	Refresh tokens stored as SHA-256 hashes with expiry and revocation time. - Primary keys: TokenId - Foreign keys: UserId → Users
04	VerificationEmail	E-mail verification codes (HMAC of the code), expiry, attempts and resend time. - Primary keys: VerificationEmailId - Foreign keys: UserId → Users
05	PasswordResetChallenges	The current password-reset challenge of an account (one per user). - Primary keys: UserId - Foreign keys: UserId → Users

No	Table	Description
06	PlayerProfiles	Game profile of an account: display name, character, onboarding choice, level, XP, Elo, wins, losses, streaks, statistics and equipped items. - Primary keys: UserId - Foreign keys: UserId → Users
07	FriendRequests	Friend requests with status PENDING, ACCEPTED or DECLINED. - Primary keys: RequestId - Foreign keys: SenderId → Users; ReceiverId → Users
08	Friendships	Accepted friendships stored once per pair (UserId1 < UserId2). - Primary keys: UserId1, UserId2 - Foreign keys: UserId1 → Users; UserId2 → Users
09	BlockedUsers	Blocks from one user to another. - Primary keys: UserId, BlockedUserId - Foreign keys: UserId → Users; BlockedUserId → Users
10	GameMatches	Matches of every kind with mode, kind, authority mode, status (PREPARING, PLAYING, FINISHED, CANCELLED), rating policy and timestamps. - Primary keys: MatchId - Foreign keys: none
11	PlayerLoginDays	Days (Vietnam time) on which a player signed in; used for retention. - Primary keys: PlayerId, Day - Foreign keys: PlayerId → Users
12	PlayerActivityDays	Days on which a player was active, with active seconds. - Primary keys: PlayerId, Day - Foreign keys: PlayerId → PlayerProfiles
13	MatchmakingSearches	Durable log of ranked searches with status SEARCHING, MATCHED, BOT, CANCELLED or TIMEOUT. - Primary keys: SearchId - Foreign keys: PlayerId → PlayerProfiles
14	ReportPlayerInMatch	Player reports, one per reporter, reported player and match, with status and decision. - Primary keys: ReporterId, ReportedPlayerId, MatchId - Foreign keys: ReviewerId → Users; ReporterId → PlayerProfiles; ReportedPlayerId → PlayerProfiles; MatchId → GameMatches
15	UserSanctions	Suspensions and bans with actor, reason, expiry and revocation data. - Primary keys: SanctionId - Foreign keys: UserId → Users; ActorId → Users; RevokedBy → Users; ReportMatchId → GameMatches; ReporterId → Users
16	SanctionAppeals	Appeals, at most one per sanction, with statement, status and decision. - Primary keys: AppealId - Foreign keys: SanctionId → UserSanctions; UserId → Users; ReviewerId → Users
17	MatchHistoryReplay	Object-storage key of the replay manifest of a match. - Primary keys: MatchId - Foreign keys: MatchId → GameMatches
18	Currencies	Currency catalogue (TOKENPAD, ELITE TOKEN). - Primary keys: CurrencyId - Foreign keys: none
19	PlayerCurrencies	Wallet balance of a player per currency. - Primary keys: PlayerId, CurrencyId - Foreign keys: PlayerId → PlayerProfiles; CurrencyId → Currencies
20	Cosmetics	Cosmetic items with code, type, default price and currency, and display configuration. - Primary keys: CosmeticId - Foreign keys: CurrencyId → Currencies
21	PlayerCosmetics	Items owned by a player and the equipped flag. - Primary keys: PlayerId, CosmeticId - Foreign keys: PlayerId → PlayerProfiles; CosmeticId → Cosmetics
22	Achievements	Achievement definitions with type, target and reward. - Primary keys: AchievementId - Foreign keys: none
23	PlayerAchievements	Achievement progress and claim time of a player. - Primary keys: PlayerId, AchievementId - Foreign keys: PlayerId → PlayerProfiles; AchievementId → Achievements
24	Missions	Daily and weekly mission definitions with target and reward. - Primary keys: MissionId - Foreign keys: none

No	Table	Description
25	PlayerMissions	Mission progress of a player in one period (PeriodKey) and the claim time. - Primary keys: PlayerId, MissionId, PeriodKey - Foreign keys: PlayerId → PlayerProfiles; MissionId → Missions
26	Techniques	Techniques taught by the curriculum. - Primary keys: TechniqueId - Foreign keys: none
27	TechniqueReferences	Versioned reference values used to judge a technique. - Primary keys: ReferenceId - Foreign keys: TechniqueId → Techniques
28	Lessons	Lessons with code, group, order, target, reward (JSONB), configuration (JSONB), prerequisite and status (DRAFT, PUBLISHED, UNPUBLISHED). - Primary keys: LessonId - Foreign keys: none
29	Drills	Drills that practise one technique. - Primary keys: DrillId - Foreign keys: TechniqueId → Techniques
30	LessonDrills	Drills of a lesson with their target (many-to-many). - Primary keys: LessonId, DrillId - Foreign keys: LessonId → Lessons; DrillId → Drills
31	PlayerTechniqueMastery	Mastery score of a player per technique. - Primary keys: PlayerId, TechniqueId - Foreign keys: PlayerId → PlayerProfiles; TechniqueId → Techniques
32	PlayerTechniquesErrors	Technique errors detected for a player in a lesson or match. - Primary keys: ErrorLogId - Foreign keys: PlayerId → PlayerProfiles; TechniqueId → Techniques; MatchId → GameMatches; LessonId → Lessons
33	PlayerLessonDrills	Best result of a player per lesson drill: stars, accuracy, timing, attempts and first-pass time. - Primary keys: PlayerId, LessonId, DrillId - Foreign keys: PlayerId → PlayerProfiles; LessonId, DrillId → LessonDrills
34	CosmeticPrices	Prices of an item in each accepted currency. - Primary keys: CosmeticId, CurrencyId - Foreign keys: CosmeticId → Cosmetics; CurrencyId → Currencies
35	GameCommands	Idempotency log of client commands (purchase, claim, sync) per player and command ID. - Primary keys: PlayerId, CommandId - Foreign keys: PlayerId → PlayerProfiles
36	TrainingSessions	Each training attempt with lesson, drill, shots, hits, timing, stars and receipt status. - Primary keys: AttemptId - Foreign keys: PlayerId → PlayerProfiles; LessonId → Lessons; DrillId → Drills; LessonId, DrillId → LessonDrills
37	MatchScore	Score of each game of a match (weak entity identified by match and game number). - Primary keys: MatchId, GameNumber - Foreign keys: MatchId → GameMatches
38	GameMatchPlayers	Participants of a match with team, slot, character, bot flag, Elo input and outcome. - Primary keys: MatchId, PlayerId - Foreign keys: MatchId → GameMatches; PlayerId → PlayerProfiles
39	CurrencyTransactions	Ledger of every currency change with reason and reference. - Primary keys: TransactionId - Foreign keys: PlayerId → PlayerProfiles; CurrencyId → Currencies
40	ratingledger	Elo before, delta and after for one player in one ranked match. - Primary keys: playerid, matchid - Foreign keys: playerid → PlayerProfiles; matchid → GameMatches
41	aitelemetrysessions	Telemetry batch of a drill or match with source, mode and status. - Primary keys: batchid - Foreign keys: playerid → PlayerProfiles
42	ExpertReviews	Expert labels (level, errors, data group) for a telemetry batch. - Primary keys: ReviewId - Foreign keys: BatchId → aitelemetrysessions; ReviewerId → Users
43	CoachingDemoSessions	Simulated demonstration sessions kept apart from real player data. - Primary keys: SessionId - Foreign keys: none
44	aitelemetryparts	Compressed parts of a telemetry batch. - Primary keys: batchid, partindex - Foreign keys: batchid → aitelemetrysessions

No	Table	Description
45	aianalysisjobs	Analysis job of a batch with status pending, processing, completed or failed, attempts and error. - Primary keys: batchid - Foreign keys: batchid → aitelemetrysessions
46	aicoachingsessions	Coaching result of a batch: findings, summary and recommendations. - Primary keys: batchid - Foreign keys: playerid → PlayerProfiles
47	aicoachingprofiles	Aggregated coaching profile of a player. - Primary keys: playerid - Foreign keys: playerid → PlayerProfiles
48	ContentCatalogs	Published versions of the content catalogue sent to clients. - Primary keys: Version - Foreign keys: none
49	ProgressRewardGrants	Log of one-time progression rewards (for example THOT, GEMSTONE) per player. - Primary keys: PlayerId, RewardKey - Foreign keys: PlayerId → PlayerProfiles
50	DedicatedOperations	Operations on dedicated matches (stop requests and their acknowledgement). - Primary keys: OperationId - Foreign keys: MatchId → GameMatches
51	AdminAudits	Audit trail of administrative actions with actor, action, target and data. - Primary keys: AuditId - Foreign keys: none

2.1 Data Dictionary of the Core Tables

The core tables of the account, match, training and moderation domains are listed column by column. Types are PostgreSQL types; * marks NOT NULL.

Table 9. Data dictionary – Users

Column	Type	Null	Key	Notes
UserId	UUID	*	PK	
Email	VARCHAR(254)	*		
PasswordHash	VARCHAR(254)	*		
AuthVersion	INT	*		
RoleId	INT	*	FK → Roles	
Status	VARCHAR(20)	*		ACTIVE, SUSPENDED, BANNED
CreatedAt	TIMESTAMPTZ	*		

Table 10. Data dictionary – PlayerProfiles

Column	Type	Null	Key	Notes
UserId	UUID	*	PK, FK → Users	
Avatar	VARCHAR(80)	*		
PlayerName	VARCHAR(18)	*		
Elo	INT	*		
IsMale	BOOLEAN	*		
TimePlayedSeconds	BIGINT	*		
Level	INT	*		
Exp	INT	*		
WinStreak	INT	*		
DayStreak	INT	*		
WinCount	INT	*		
LoseCount	INT	*		

Column	Type	Null	Key	Notes
Revision	BIGINT	*		
LearningEpoch	BIGINT	*		
LearningResetAt	TIMESTAMPTZ			
MissionEpoch	BIGINT	*		
EconomyEpoch	BIGINT	*		
ExperienceChoice	VARCHAR(20)			
TutorialCompleted	BOOLEAN	*		
TutorialSkipped	BOOLEAN	*		
TutorialCheckpoint	INT	*		
DailyStreakEnabledAt	TIMESTAMPTZ			
RankWinStreak	INT	*		
RankLossStreak	INT	*		
BestWinStreak	INT	*		
LifetimeMatches	BIGINT	*		
LifetimePoints	BIGINT	*		
LifetimeReturns	BIGINT	*		
LifetimeServes	BIGINT	*		
LifetimeBotMatches	BIGINT	*		
LifetimeHumanMatches	BIGINT	*		
LifetimeHumanWins	BIGINT	*		
LifetimeRankedWins	BIGINT	*		
LifetimeDoublesWins	BIGINT	*		
LifetimeHigherRatedWins	BIGINT	*		

Table 11. Data dictionary – GameMatches

Column	Type	Null	Key	Notes
AuthorityMode	VARCHAR(16)	*		shared or dedicated
ServerHeartbeatAt	TIMESTAMPTZ			
MatchId	UUID	*	PK	
Kind	VARCHAR(32)	*		
Status	VARCHAR(20)	*		PREPARING, PLAYING, FINISHED, CANCELLED
RatingPolicyVersion	VARCHAR(64)	*		
Mode	SMALLINT	*		
PhotonSession	VARCHAR(100)			
SearchId	UUID			
Region	VARCHAR(30)			
Night	BOOLEAN	*		
Venue	SMALLINT	*		
CreatedAt	TIMESTAMPTZ	*		
UpdatedAt	TIMESTAMPTZ	*		
StartedAt	TIMESTAMPTZ			
EndedAt	TIMESTAMPTZ			
Sequence	BIGINT			

Table 12. Data dictionary – GameMatchPlayers

Column	Type	Null	Key	Notes
MatchId	UUID	*	PK, FK → GameMatches	
PlayerId	UUID	*	PK, FK → PlayerProfiles	
Team	SMALLINT	*		
EloBefore	INT	*		
GroupId	TEXT			
RosterIndex	SMALLINT	*		
IsReplayWriter	BOOLEAN	*		
ValidReturns	INT	*		
ValidServes	INT	*		
EndReason	VARCHAR(500)	*		
ReportedAt	TIMESTAMPTZ			
LastSeenAt	TIMESTAMPTZ	*		
Outcome	VARCHAR(16)			
LocalForfeit	BOOLEAN	*		
FinalizedAt	TIMESTAMPTZ			

Table 13. Data dictionary – MatchScore

Column	Type	Null	Key	Notes
MatchId	UUID	*	PK, FK → GameMatches	
GameNumber	SMALLINT	*	PK	
TeamAScore	INT	*		
TeamBScore	INT	*		
Completed	BOOLEAN	*		

Table 14. Data dictionary – ratingledger

Column	Type	Null	Key	Notes
playerid	UUID	*	PK, FK → PlayerProfiles	
matchid	UUID	*	PK, FK → GameMatches	
beforeelo	INT	*		
afterelo	INT	*		
opponentelo	INT	*		
delta	INT	*		
policyversion	TEXT	*		
createdat	TIMESTAMPTZ	*		

Table 15. Data dictionary – Lessons

Column	Type	Null	Key	Notes
LessonId	SERIAL	*	PK	
LessonName	VARCHAR(254)	*		
Description	VARCHAR(254)	*		
Code	VARCHAR(50)			
SortOrder	INT	*		
Type	VARCHAR(25)	*		

Column	Type	Null	Key	Notes
Target	INT	*		
Config	JSONB	*		
PublishStatus	VARCHAR(20)	*		
DraftConfig	JSONB			
ContentVersion	BIGINT	*		
Reward	JSONB	*		

Table 16. Data dictionary – TrainingSessions

Column	Type	Null	Key	Notes
AttemptId	UUID	*	PK	
Sequence	BIGINT			
PlayerId	UUID	*	FK → PlayerProfiles	
LessonId	INT	*	FK → LessonDrills	
DrillId	INT	*	FK → LessonDrills	
Data	JSONB	*		
CatalogVersion	VARCHAR(64)	*		
ProgressEpoch	BIGINT	*		
Fingerprint	VARCHAR(64)	*		
Completed	BOOLEAN	*		
IsCurrent	BOOLEAN	*		
CreatedAt	TIMESTAMPTZ	*		
UpdatedAt	TIMESTAMPTZ	*		

Table 17. Data dictionary – UserSanctions

Column	Type	Null	Key	Notes
SanctionId	UUID	*	PK	
UserId	UUID	*	FK → Users	
ActorId	UUID	*	FK → Users	
Restriction	VARCHAR(20)	*		BANNED or SUSPENDED; SUSPENDED requires ExpiresAt
Reason	VARCHAR(2000)	*		
CreatedAt	TIMESTAMPTZ	*		
ExpiresAt	TIMESTAMPTZ			
RevokedAt	TIMESTAMPTZ			
RevokedBy	UUID		FK → Users	
RevocationReason	VARCHAR(2000)			
ReportMatchId	UUID		FK → GameMatches	
ReporterId	UUID		FK → Users	

Table 18. Data dictionary – SanctionAppeals

Column	Type	Null	Key	Notes
AppealId	UUID	*	PK	
SanctionId	UUID	*	FK → UserSanctions	UNIQUE: one appeal per sanction
UserId	UUID	*	FK → Users	
Statement	VARCHAR(4000)	*		

Column	Type	Null	Key	Notes
Status	VARCHAR(20)	*		PENDING, CHECKING, ACCEPTED, DECLINED
CreatedAt	TIMESTAMPTZ	*		
UpdatedAt	TIMESTAMPTZ	*		
ReviewerId	UUID		FK → Users	
DecisionReason	VARCHAR(2000)			

2.2 Design Notes

- Team game totals are not stored in GameMatches; they are derived from the completed MatchScore rows, which avoids inconsistent copies (AGENTS.md).
- GameMatchPlayers keeps only facts that belong to the participant in that match, including the fixed Elo input needed before a rating ledger row exists; names and cosmetics are joined from the profile tables.
- PlayerProfiles is wide (35 columns) because it is the hot read model of the menu; statistics that change together are kept in one row to avoid joins on every screen.
- TrainingSessions references Lessons, Drills and the composite LessonDrills key: the composite key guarantees that the drill belongs to the lesson, and the single keys keep historical attempts when a lesson is re-configured.
- Lessons.Reward and Lessons.Config are JSONB because each lesson type has different parameters; the API validates them before saving.
- AdminAudits stores the actor ID without a foreign key so that audit records survive the deletion of an administrator account.
- CoachingDemoSessions holds simulated data in its own table so that it can never be mixed with real sessions used for AI evaluation.
- Idempotency: GameCommands, TrainingSessions receipts and the UNIQUE constraints on rewards (ProgressRewardGrants, PlayerMissions period key) make retried requests safe.

3. Detailed Design

Each subsection gives the class diagram of one feature group, the specification of its classes and the sequence diagrams of its main functions. Client classes (Unity) and server classes (Web API) are drawn in the same diagram only when the diagram shows their interaction; the boundary is the HTTP API, marked as a dependency to the controller.

3.1 Account and Authentication

This feature group implements FR-01 to FR-05. The Unity AccountScreen calls AccountService, which uses ApiClient to reach AuthController. AuthService performs the rules of BR-01 to BR-05 through IUnitOfWork and sends e-mails through IEmailSender.

3.1.1 Class Diagram

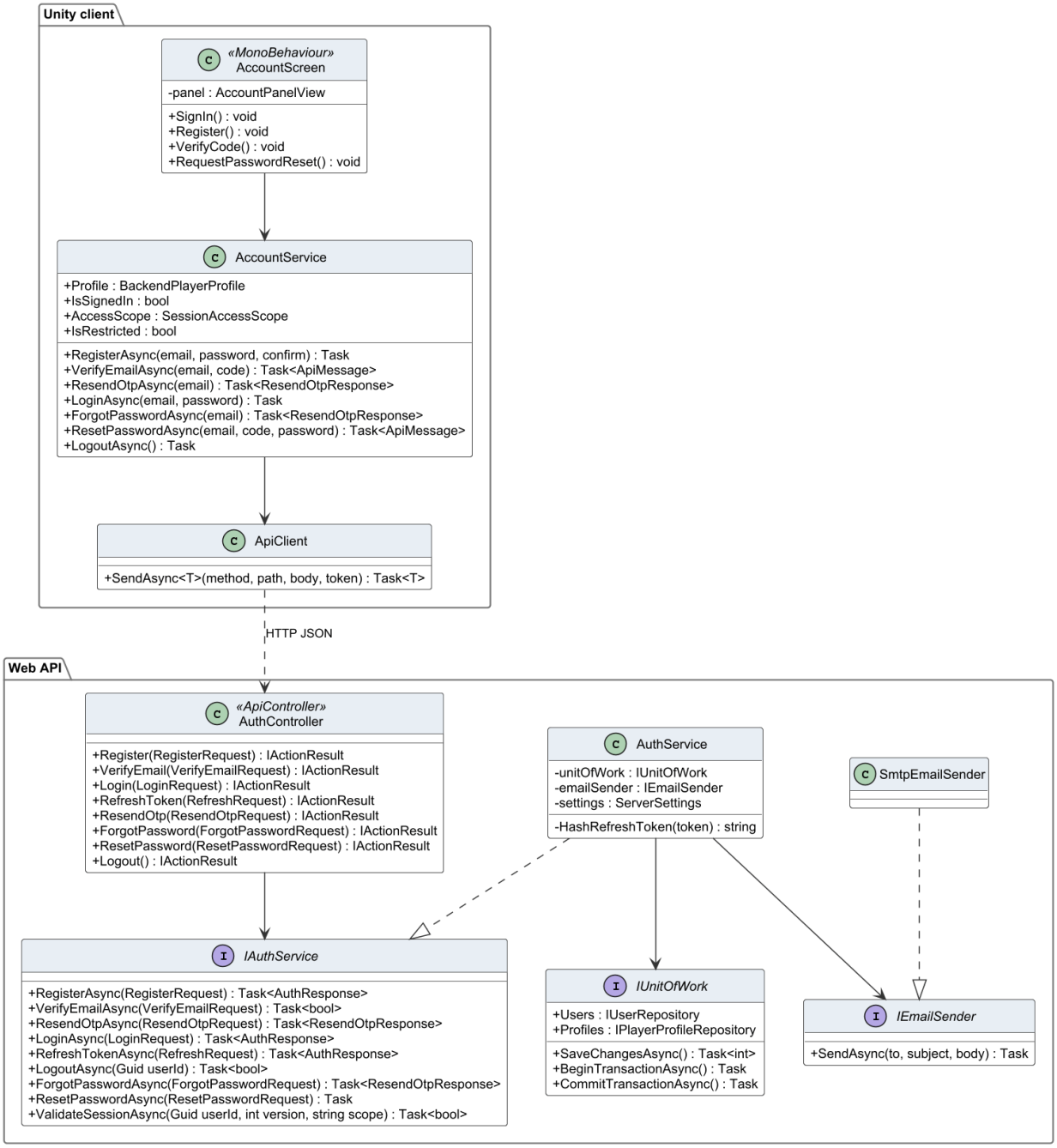


Figure 13. Class diagram – account and authentication

3.1.2 Class Specifications

Table 19. Class specifications – account and authentication

Class	Responsibility	Main attributes	Main operations
AccountScreen «MonoBehaviour»	Login, register, verification and password-reset pages of the Login scene; client-side validation	- panel : AccountPanelView	+ SignIn() : void + Register() : void + VerifyCode() : void + RequestPasswordReset() : void

Class	Responsibility	Main attributes	Main operations
AccountService	Client facade for authentication; keeps the session, scope and profile and refreshes tokens	+ Profile : BackendPlayerProfile + IsSignedIn : bool + AccessScope : SessionAccessScope + IsRestricted : bool	+ RegisterAsync(email, password, confirm) : Task + VerifyEmailAsync(email, code) : Task<ApiMessage> + ResendOtpAsync(email) : Task<ResendOtpResponse> + LoginAsync(email, password) : Task + ForgotPasswordAsync(email) : Task<ResendOtpResponse> + ResetPasswordAsync(email, code, password) : Task<ApiMessage> + LogoutAsync() : Task
ApiClient	Sends HTTP requests with the bearer token and unwraps the ApiResponse envelope	–	+ SendAsync<T>(method, path, body, token) : Task<T>
AuthController «ApiController»	REST endpoints /api/Auth/*; rate-limited	–	+ Register(RegisterRequest) : IActionResult + VerifyEmail(VerifyEmailRequest) : IActionResult + Login(LoginRequest) : IActionResult + RefreshToken(RefreshRequest) : IActionResult + ResendOtp(ResendOtpRequest) : IActionResult + ForgotPassword(ForgotPasswordRequest) : IActionResult + ResetPassword(ResetPasswordRequest) : IActionResult + Logout() : IActionResult
IAuthService «interface»	Contract of the authentication use cases	–	+ RegisterAsync(RegisterRequest) : Task<AuthResponse> + VerifyEmailAsync(VerifyEmailRequest) : Task<bool> + ResendOtpAsync(ResendOtpRequest) : Task<ResendOtpResponse> + LoginAsync(LoginRequest) : Task<AuthResponse> + RefreshTokenAsync(RefreshRequest) : Task<AuthResponse> + LogoutAsync(Guid userId) : Task<bool> + ForgotPasswordAsync(ForgotPasswordRequest) : Task<ResendOtpResponse> + ResetPasswordAsync(ResetPasswordRequest) : Task + ValidateSessionAsync(Guid userId, int version, string scope) : Task<bool>
AuthService	Registration, OTP, sign-in, token issue and rotation, password reset, session validation	- unitOfWork : IUnitOfWork - emailSender : IEmailSender - settings : ServerSettings	- HashRefreshToken(token) : string
IEmailSender «interface»	Abstraction for sending e-mail	–	+ SendAsync(to, subject, body) : Task
SmtpEmailSender	Sends e-mail through Gmail SMTP with STARTTLS	–	–
IUnitOfWork «interface»	Gives services access to repositories and one shared transaction	+ Users : IUserRepository + Profiles : IPlayerProfileRepository	+ SaveChangesAsync() : Task<int> + BeginTransactionAsync() : Task + CommitTransactionAsync() : Task

3.1.3 Register and verify e-mail

The visitor registers; the service hashes the password, stores the HMAC of a six-digit code and queues the e-mail. The alternative fragments show a confirmation mismatch, a registered e-mail, a valid code and an invalid or expired code.

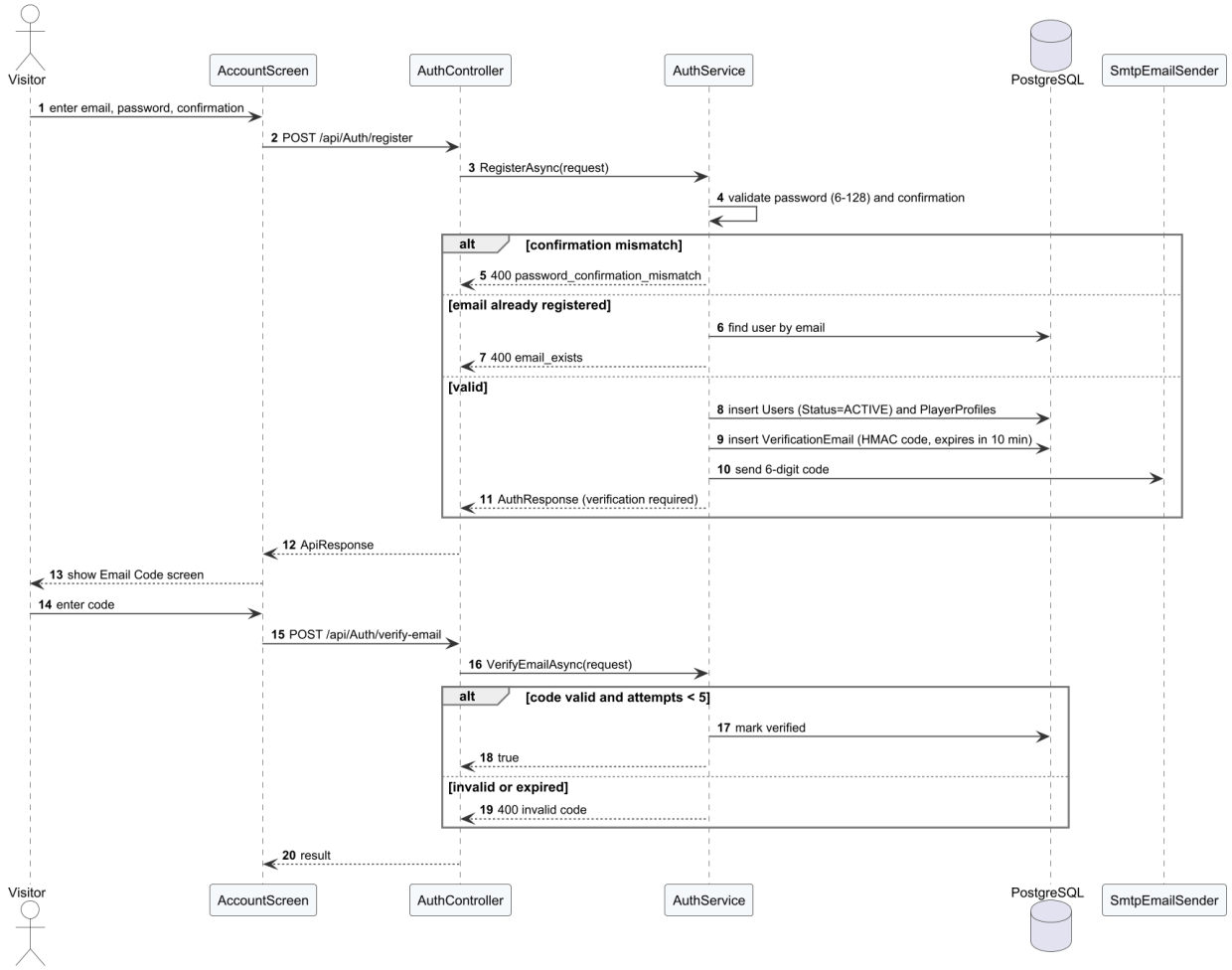


Figure 14. Sequence diagram – Register and verify e-mail

3.1.4 Sign in

The service checks the password hash and the account state. A wrong password, an unverified e-mail and a restricted account return different results; an active account receives the JWT and refresh token, after which the client loads the profile.

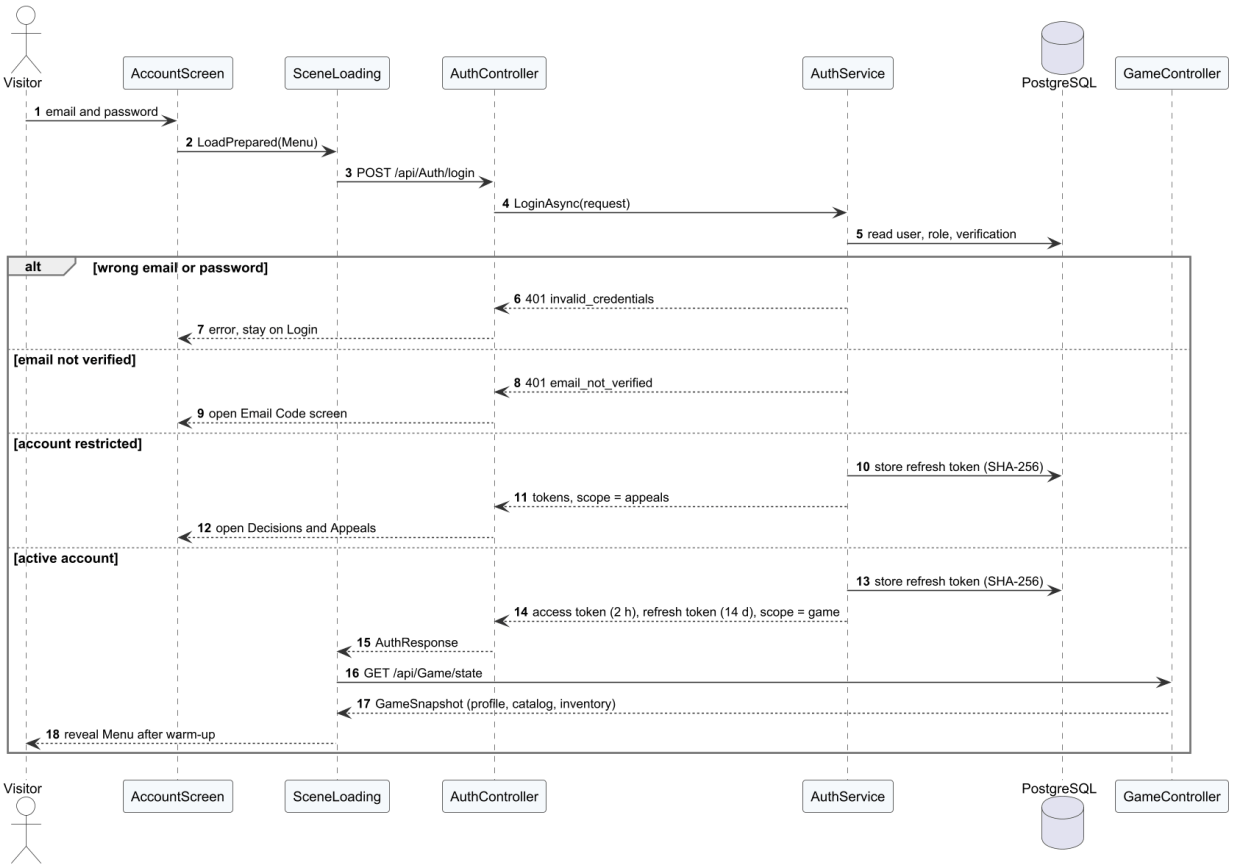


Figure 15. Sequence diagram – Sign in

3.1.5 Reset password

The reset request always returns the same message; the mail queue sends the code only for a verified account. A valid code changes the hash and revokes every refresh token.

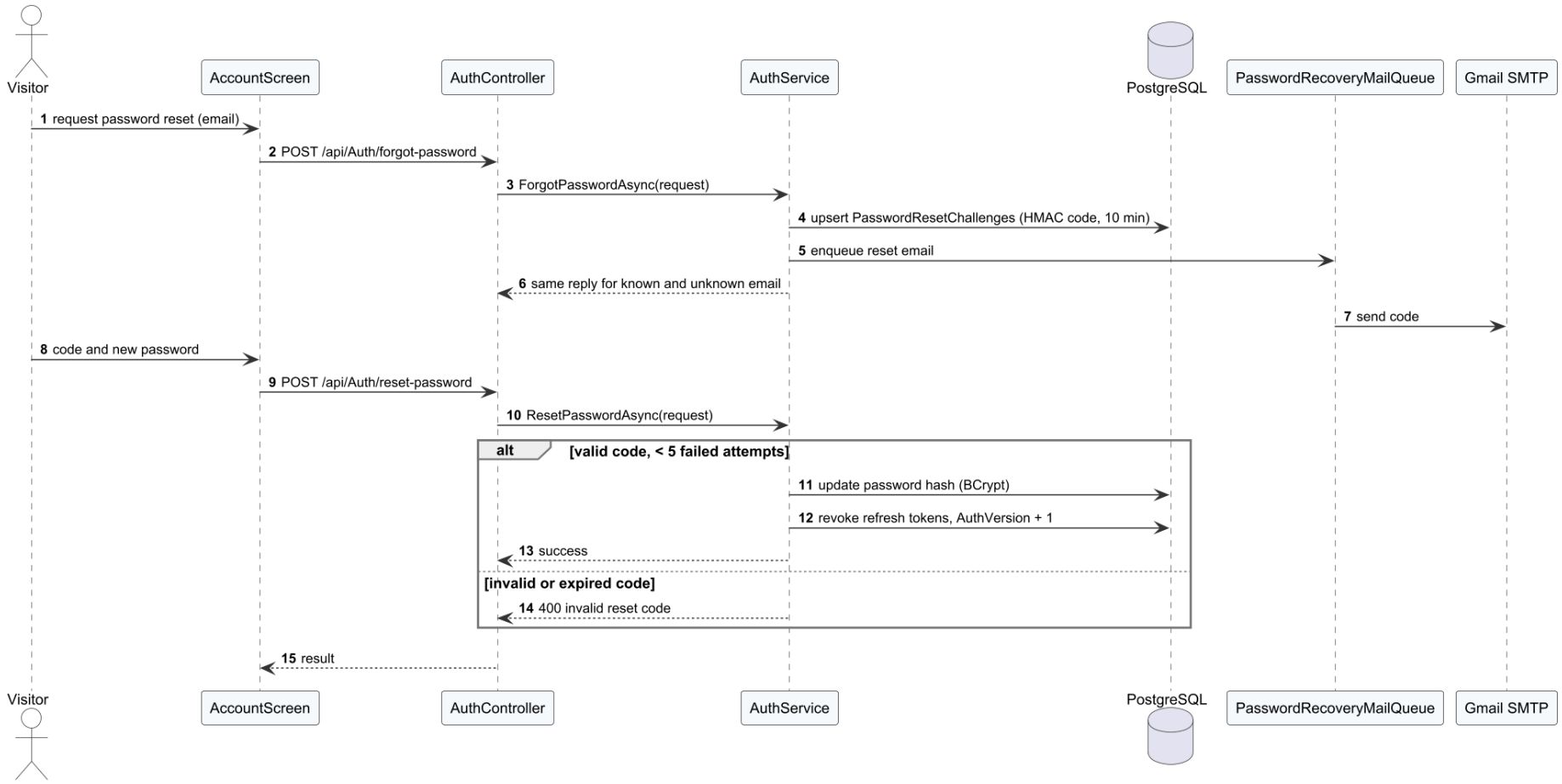


Figure 16. Sequence diagram – Reset password

3.2 Gameplay Core and Ball Physics

The gameplay core implements FR-11 to FR-13. BotMatch drives the match phases. Each MatchActor reads its intent from ShotInput (player) or a bot. PaddleContact accepts the hit, ShotTrajectory computes the launch velocity, BallFlight advances the ball and SinglesRules decides the outcome. The rule and physics classes have no Unity dependencies, so the same code runs in the client, in the dedicated server and in EditMode unit tests.

Table 20. Gameplay and physics parameters

Parameter	Value	Source / meaning
Simulation step	120 Hz (1/120 s)	BallFlight fixed step, independent of frame rate
Gravity	9.81 m/s²	Ball flight
Bounce restitution	0.73	Vertical speed kept after a bounce
Ball radius	0.037 m	Line calls use the ball edge
Court half width / half length / kitchen	3.048 / 6.7056 / 2.1336 m	SinglesRules.Court, official dimensions
Contact window	±65 ms	PaddleContact; outside the window the swing misses
Perfect contact	≤ 0.145 m and ±30 ms	Higher speed and accuracy
Serve timer	10 s	Fault ServeTimeout
Contact delay per shot	serve 0.80 s, dink 0.367 s, drop 0.533 s, smash 0.50 s, volley/reset 0.233 s, drive 0.433 s	ShotIntent.ContactDelay: time from release to the animation's contact frame

3.2.1 Class Diagram

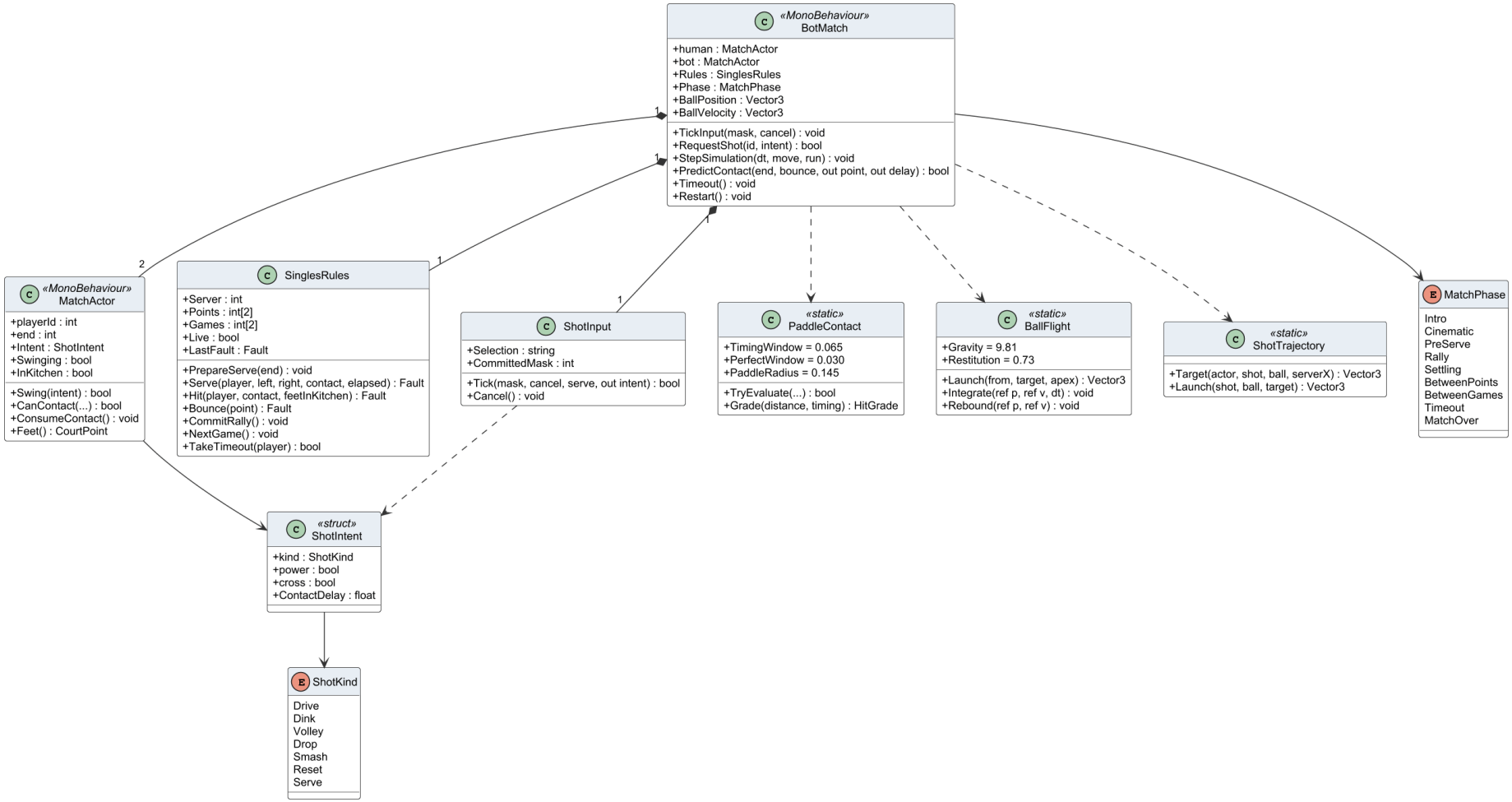


Figure 17. Class diagram – gameplay core (bot match)

3.2.2 Class Specifications

Table 21. Class specifications – gameplay core

Class	Responsibility	Main attributes	Main operations
BotMatch «MonoBehaviour»	Orchestrates a local singles match: phases, serve, rally, scoring and result	+ human : MatchActor + bot : MatchActor + Rules : SinglesRules + Phase : MatchPhase + BallPosition : Vector3 + BallVelocity : Vector3	+ TickInput(mask, cancel) : void + RequestShot(id, intent) : bool + StepSimulation(dt, move, run) : void + PredictContact(end, bounce, out point, out delay) : bool + Timeout() : void + Restart() : void
MatchActor «MonoBehaviour»	Moves and animates one player and performs hits	+ playerId : int + end : int + Intent : ShotIntent + Swinging : bool + InKitchen : bool	+ Swing(intent) : bool + CanContact(...) : bool + ConsumeContact() : void + Feet() : CourtPoint
SinglesRules	Decides faults, points, side-outs, games and match end (BR-10 to BR-15)	+ Server : int + Points : int[2] + Games : int[2] + Live : bool + LastFault : Fault	+ PrepareServe(end) : void + Serve(player, left, right, contact, elapsed) : Fault + Hit(player, contact, feetInKitchen) : Fault + Bounce(point) : Fault + CommitRally() : void + NextGame() : void + TakeTimeout(player) : bool
ShotInput	Decodes the J/K/L/U/I key state into one shot intent on the first release	+ Selection : string + CommittedMask : int	+ Tick(mask, cancel, serve, out intent) : bool + Cancel() : void
ShotIntent «struct»	Requested shot kind with power and cross modifiers	+ kind : ShotKind + power : bool + cross : bool + ContactDelay : float	–
ShotKind «enum»	Seven shot kinds of the local game	Drive Dink Volley Drop Smash Reset Serve	–
PaddleContact «static»	Checks the contact window (±65 ms) and the contact quality (Perfect, Good)	+ TimingWindow = 0.065 + PerfectWindow = 0.030 + PaddleRadius = 0.145	+ TryEvaluate(...) : bool + Grade(distance, timing) : HitGrade
BallFlight «static»	Fixed-step ball flight: gravity, drag-free arc and bounce with restitution 0.73	+ Gravity = 9.81 + Restitution = 0.73	+ Launch(from, target, apex) : Vector3 + Integrate(ref p, ref v, dt) : void + Rebound(ref p, ref v) : void
ShotTrajectory «static»	Computes launch velocity for a target point and flight time	–	+ Target(actor, shot, ball, serverX) : Vector3 + Launch(shot, ball, target) : Vector3
MatchPhase «enum»	Phases of a match (Intro, PreServe, Rally, Settling, BetweenPoints, BetweenGames, Timeout, MatchOver, Cinematic)	Intro Cinematic PreServe Rally Settling BetweenPoints BetweenGames Timeout MatchOver	–

3.3 Training and Offline Synchronisation

Training implements FR-06 to FR-10. TrainingDirector runs a drill locally with the gameplay core and evaluates it with TrainingEngine. Online, the attempt is sent at once; offline, OfflineTrainingStore keeps it until the device reconnects. On the server OnlineGameService re-evaluates the attempt with the same engine and returns a receipt.

3.3.1 Class Diagram

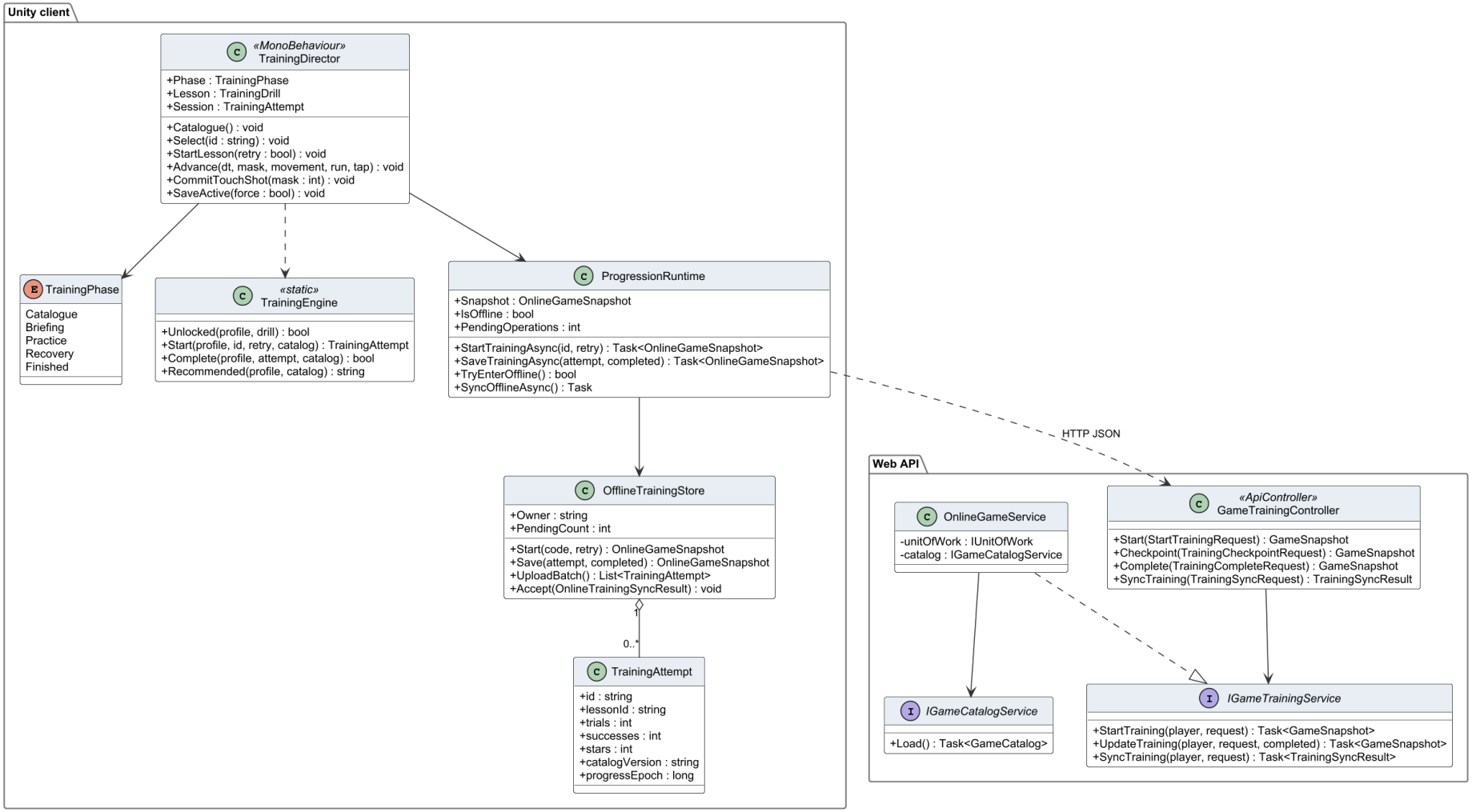


Figure 18. Class diagram – training and synchronisation

3.3.2 Class Specifications

Table 22. Class specifications – training

Class	Responsibility	Main attributes	Main operations
TrainingDirector «MonoBehaviour»	Runs a drill in the Training scene: briefing, shots, feedback and result popup	+ Phase : TrainingPhase + Lesson : TrainingDrill + Session : TrainingAttempt	+ Catalogue() : void + Select(id : string) : void + StartLesson(retry : bool) : void + Advance(dt, mask, movement, run, tap) : void + CommitTouchShot(mask : int) : void + SaveActive(force : bool) : void
TrainingPhase «enum»	Phases of the training screen	Catalogue Briefing Practice Recovery Finished	–
TrainingEngine «static»	Pure rules for drill evaluation: hits, timing and stars (BR-18)	–	+ Unlocked(profile, drill) : bool + Start(profile, id, retry, catalog) : TrainingAttempt + Complete(profile, attempt, catalog) : bool + Recommended(profile, catalog) : string
ProgressionRuntime	Client-side cache of profile, wallet, catalogue and the synchronisation queue	+ Snapshot : OnlineGameSnapshot + IsOffline : bool + PendingOperations : int	+ StartTrainingAsync(id, retry) : Task<OnlineGameSnapshot> + SaveTrainingAsync(attempt, completed) : Task<OnlineGameSnapshot> + TryEnterOffline() : bool + SyncOfflineAsync() : Task
OfflineTrainingStore	Durable local queue of offline attempts with request IDs	+ Owner : string + PendingCount : int	+ Start(code, retry) : OnlineGameSnapshot + Save(attempt, completed) : OnlineGameSnapshot + UploadBatch() : List<TrainingAttempt> + Accept(OnlineTrainingSyncResult) : void
TrainingAttempt	Data of one drill attempt sent to the server	+ id : string + lessonId : string + trials : int + successes : int + stars : int + catalogVersion : string + progressEpoch : long	–
GameTrainingController «ApiController»	REST endpoints for starting, completing and synchronising training	–	+ Start(StartTrainingRequest) : GameSnapshot + Checkpoint(TrainingCheckpointRequest) : GameSnapshot + Complete(TrainingCompleteRequest) : GameSnapshot + SyncTraining(TrainingSyncRequest) : TrainingSyncResult
IGameTrainingService «interface»	Contract of the training use cases	–	+ StartTraining(player, request) : Task<GameSnapshot> + UpdateTraining(player, request, completed) : Task<GameSnapshot> + SyncTraining(player, request) : Task<TrainingSyncResult>
OnlineGameService	Server implementation of training, match settlement, matchmaking and economy use cases	- unitOfWork : IUnitOfWork - catalog : IGameCatalogService	–
IGameCatalogService «interface»	Provides the published content catalogue and its version	–	+ Load() : Task<GameCatalog>

3.3.3 Practise a drill online

The server first checks that the lesson is published and unlocked, then records each trial; at the end it applies stars and the first-pass reward in one transaction.

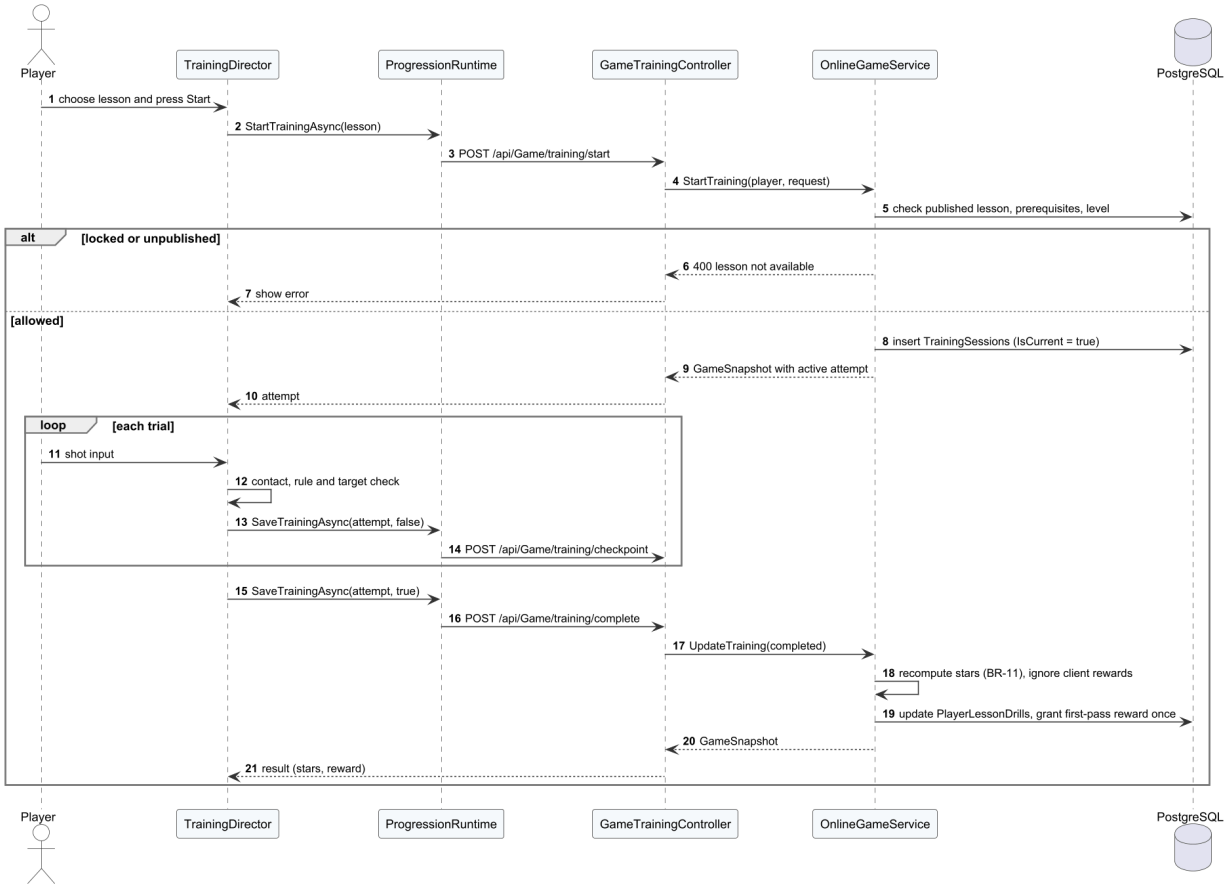


Figure 19. Sequence diagram – Practise a drill online

3.3.4 Synchronise offline attempts

Each queued attempt is sent with its request ID. A new and valid attempt is applied, an already applied one is answered DUPLICATE, and a stale or invalid one is REJECTED.

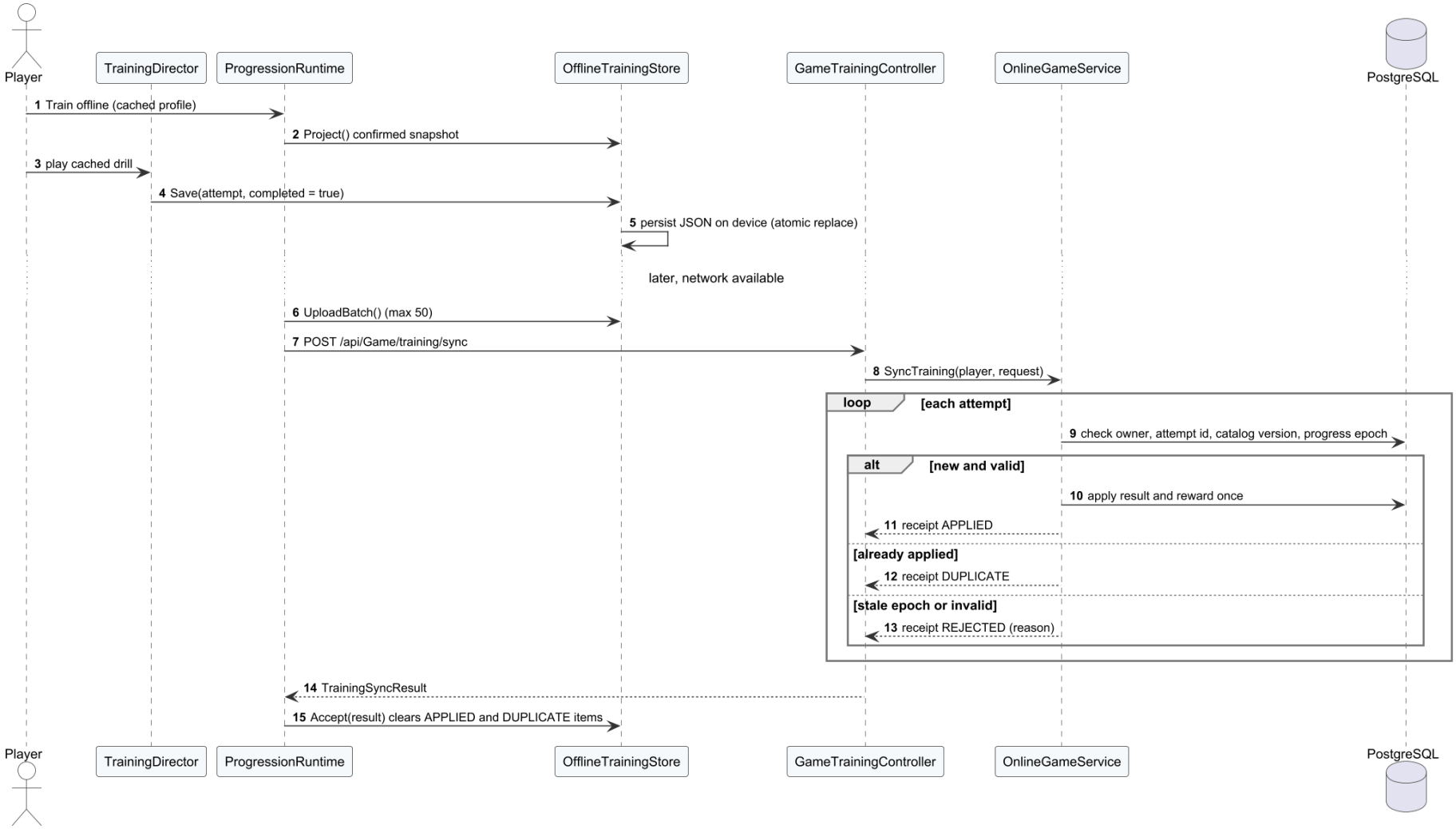


Figure 20. Sequence diagram – Synchronise offline attempts

3.4 Bot AI

The bot design implements FR-13 and BR-31. LocalBot provides three fixed levels. BhTreeBot executes a Unity Behavior graph whose decisions (position, shot choice, aggression, deliberate errors) are parameterised by BotPolicy and a BotTierProfile selected from the opponent's Elo. The IAdaptiveBotContext interface hides whether the bot plays locally or on the dedicated server, so the same bot is used for practice and for the ranked bot fallback.

Table 23. Bot parameters

Parameter	Value	Source / meaning
Fixed levels	Easy, Medium, Hard	LocalBot: fixed reaction and accuracy
Adaptive tiers	Elo < 500 beginner; < 2000 intermediate; ≥ 2000 advanced	BotPolicy tier selection
Ranked bot Elo	own + 120 (own − 100 after 5 losses)	EloPolicy.BotElo (BR-26)
Ranked bot accuracy	min(100, 25 + (Elo − 100) / 120) %	EloPolicy.BotAccuracy

3.4.1 Class Diagram

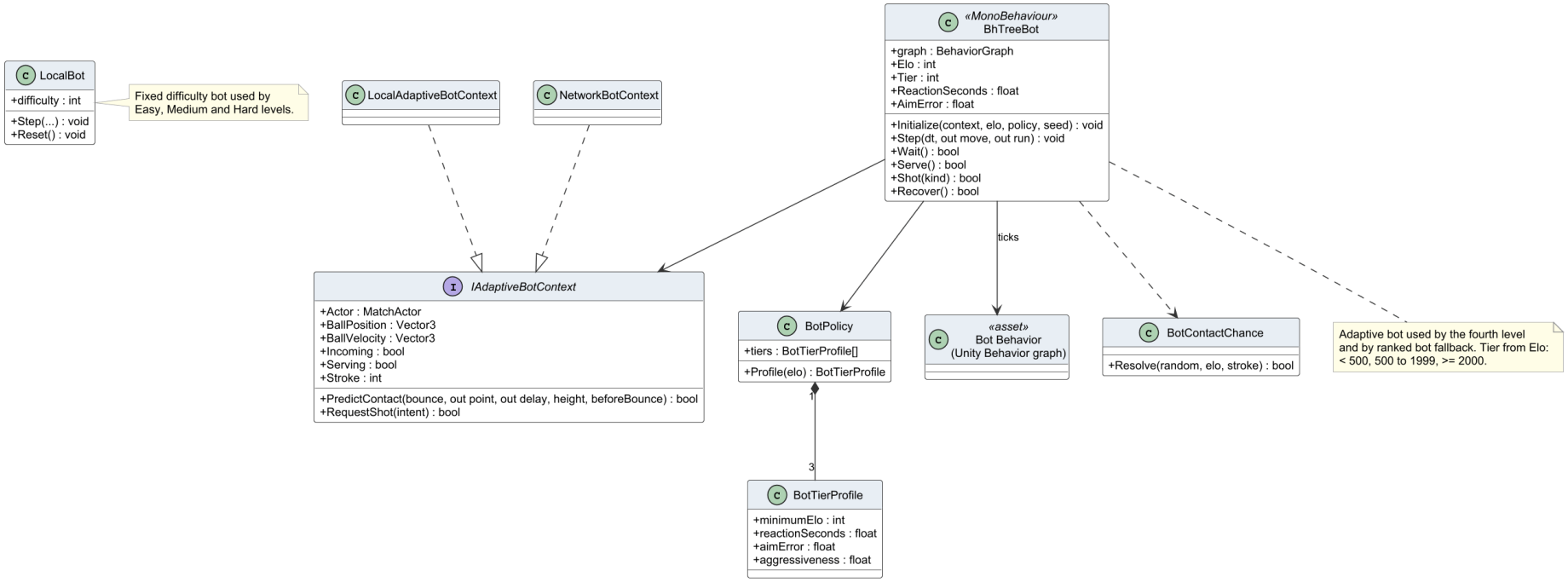


Figure 21. Class diagram – bots

3.4.2 Class Specifications

Table 24. Class specifications – bot AI

Class	Responsibility	Main attributes	Main operations
IAdaptiveBotContext «interface»	Information the bot needs about ball, court and opponent, independent of local or network play	+ Actor : MatchActor + BallPosition : Vector3 + BallVelocity : Vector3 + Incoming : bool + Serving : bool + Stroke : int	+ PredictContact(bounce, out point, out delay, height, beforeBounce) : bool + RequestShot(intent) : bool
LocalAdaptiveBotContext	Bot context for a local bot match	–	–
NetworkBotContext	Bot context for a ranked bot on the dedicated server	–	–
BhTreeBot «MonoBehaviour»	Adaptive bot driven by the Unity Behavior graph; tier chosen from Elo	+ graph : BehaviorGraph + Elo : int + Tier : int + ReactionSeconds : float + AimError : float	+ Initialize(context, elo, policy, seed) : void + Step(dt, out move, out run) : void + Wait() : bool + Serve() : bool + Shot(kind) : bool + Recover() : bool
BotPolicy	Maps a tier to reaction time, accuracy, aggression and error rate	+ tiers : BotTierProfile[]	+ Profile(elo) : BotTierProfile
BotTierProfile	Parameter set of one bot tier	+ minimumElo : int + reactionSeconds : float + aimError : float + aggressiveness : float	–
LocalBot	Fixed-level bot (Easy, Medium, Hard) with deterministic settings	+ difficulty : int	+ Step(...) : void + Reset() : void
Bot Behavior (Unity Behavior graph) «asset»	Behaviour tree asset: position, choose shot, hit, recover	–	–
BotContactChance	Probability that the bot reaches and returns a ball	–	+ Resolve(random, elo, stroke) : bool

3.4.3 Play a bot match

The loop fragment repeats rallies until one side wins two games; at the end the client reports the result to GameMatchesController, which settles rewards.

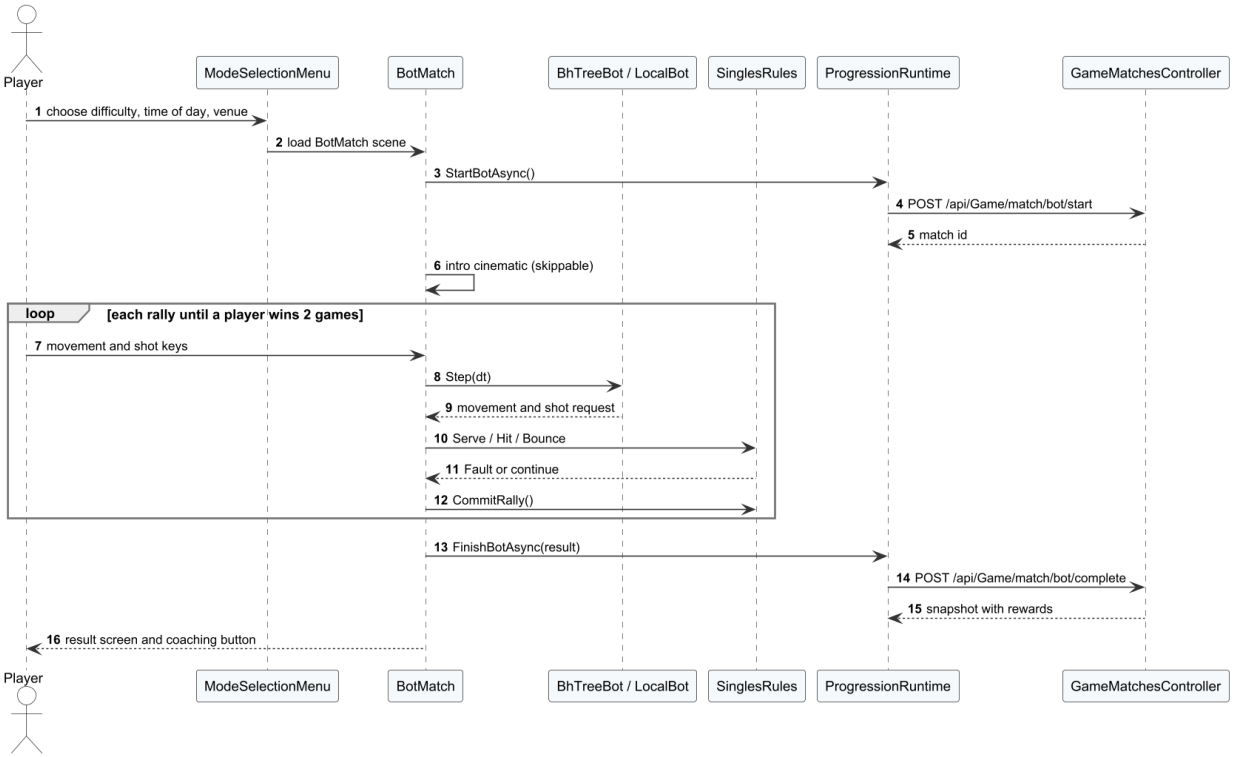


Figure 22. Sequence diagram – Play a bot match

3.5 Online Matchmaking and Netcode

Online play implements FR-14 to FR-18. MultiplayerApp is a state machine over FlowState. PhotonSession connects to Fusion; NetworkMatch and NetworkPlayer replicate state from the authority. Clients send only input (32-input history per tick) and render remote objects with snapshot interpolation; the protocol version (18) must match between client and server or the connection is refused.

3.5.1 Class Diagram

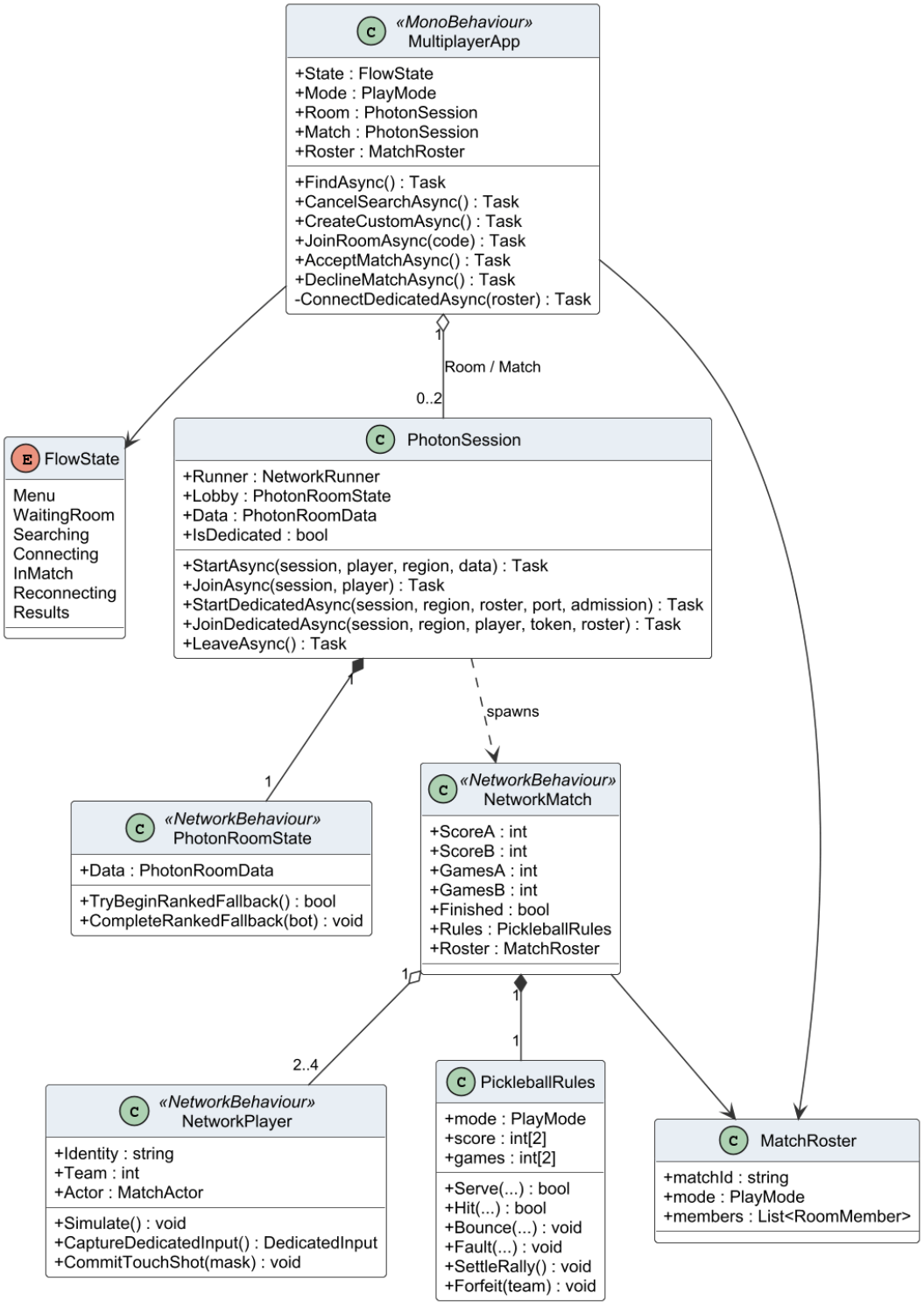


Figure 23. Class diagram – online client

3.5.2 Class Specifications

Table 25. Class specifications – online play

Class	Responsibility	Main attributes	Main operations
MultiplayerApp «MonoBehaviour»	Online flow controller: lobby, search, ready check, loading, match, reconnection and results	+ State : FlowState + Mode : PlayMode + Room : PhotonSession + Match : PhotonSession + Roster : MatchRoster	+ FindAsync() : Task + CancelSearchAsync() : Task + CreateCustomAsync() : Task + JoinRoomAsync(code) : Task + AcceptMatchAsync() : Task + DeclineMatchAsync() : Task - ConnectDedicatedAsync(roster) : Task
FlowState «enum»	States of the online flow (Menu, WaitingRoom, Delay, Searching, Connecting, InMatch, Reconnecting, Results)	Menu WaitingRoom Searching Connecting InMatch Reconnecting Results	–
PhotonSession	Starts and joins Fusion sessions in shared or client mode and handles disconnects	+ Runner : NetworkRunner + Lobby : PhotonRoomState + Data : PhotonRoomData + IsDedicated : bool	+ StartAsync(session, player, region, data) : Task + JoinAsync(session, player) : Task + StartDedicatedAsync(session, region, roster, port, admission) : Task + JoinDedicatedAsync(session, region, player, token, roster) : Task + LeaveAsync() : Task
PhotonRoomState «NetworkBehaviour»	Networked room state: roster, phase and host	+ Data : PhotonRoomData	+ TryBeginRankedFallback() : bool + CompleteRankedFallback(bot) : void
NetworkMatch «NetworkBehaviour»	Networked match state replicated from the authority: score, phase and ball	+ ScoreA : int + ScoreB : int + GamesA : int + GamesB : int + Finished : bool + Rules : PickleballRules + Roster : MatchRoster	–
NetworkPlayer «NetworkBehaviour»	Networked player: input, position and animation	+ Identity : string + Team : int + Actor : MatchActor	+ Simulate() : void + CaptureDedicatedInput() : DedicatedInput + CommitTouchShot(mask) : void
PickleballRules	Rules engine used by the network match (singles and doubles)	+ mode : PlayMode + score : int[2] + games : int[2]	+ Serve(...) : bool + Hit(...) : bool + Bounce(...) : void + Fault(...) : void + SettleRally() : void + Forfeit(team) : void
MatchRoster	Teams, slots, bot flags and mode of a match	+ matchId : string + mode : PlayMode + members : List<RoomMember>	–

3.5.3 Class Diagram – dedicated server coordination

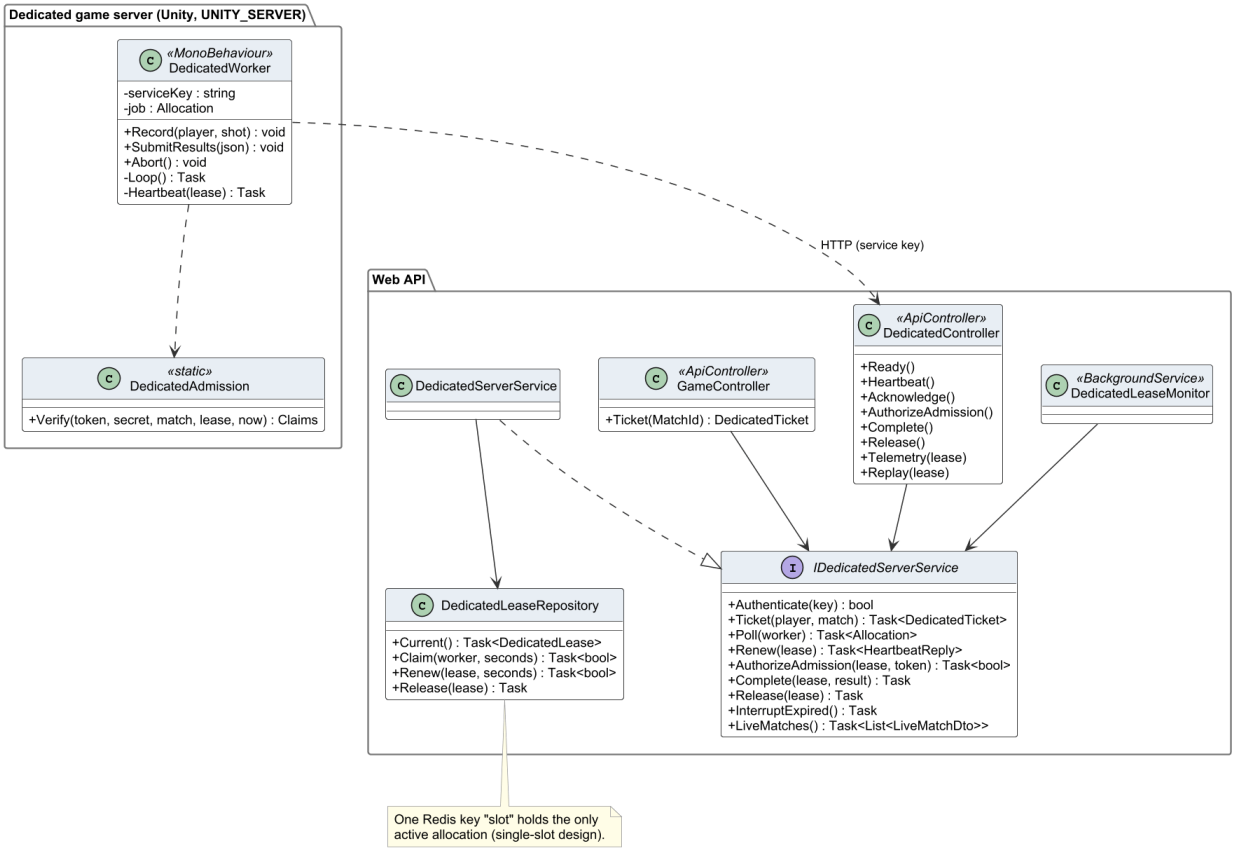


Figure 24. Class diagram – dedicated server coordination

Table 26. Class specifications – dedicated server

Class	Responsibility	Main attributes	Main operations
DedicatedWorker «MonoBehaviour»	Main loop of the dedicated server: lease, admission, heartbeats, telemetry, replay and result	- serviceKey : string - job : Allocation	+ Record(player, shot) : void + SubmitResults(json) : void + Abort() : void - Loop() : Task - Heartbeat(lease) : Task
DedicatedAdmission «static»	Validates HMAC tickets and admits players to the session	–	+ Verify(token, secret, match, lease, now) : Claims
DedicatedController «ApiController»	REST endpoints used by clients (ticket) and by the server (lease, heartbeat, result)	–	+ Ready() + Heartbeat() + Acknowledge() + AuthorizeAdmission() + Complete() + Release() + Telemetry(lease) + Replay(lease)
GameController «ApiController»	REST endpoints for profile, wallet, catalogue, purchase and claims	–	+ Ticket(MatchId) : DedicatedTicket

Class	Responsibility	Main attributes	Main operations
IDedicatedServerService «interface»	Contract of dedicated-server coordination	–	+ Authenticate(key) : bool + Ticket(player, match) : Task<DedicatedTicket> + Poll(worker) : Task<Allocation> + Renew(lease) : Task<HeartbeatReply> + AuthorizeAdmission(lease, token) : Task<bool> + Complete(lease, result) : Task + Release(lease) : Task + InterruptExpired() : Task + LiveMatches() : Task<List<LiveMatchDto>>
DedicatedServerService	Allocates the slot, signs tickets, records heartbeats and settles results	–	–
DedicatedLeaseRepository	Redis lease of the single server slot with expiry	–	+ Current() : Task<DedicatedLease> + Claim(worker, seconds) : Task<bool> + Renew(lease, seconds) : Task<bool> + Release(lease) : Task
DedicatedLeaseMonitor «BackgroundService»	Background worker that releases expired leases and interrupts orphan matches	–	–

3.5.4 Find a ranked match

The client polls the search. The alternatives show a human opponent found, the bot fallback after 180 seconds, all players accepting, and a decline or timeout that closes the offer.

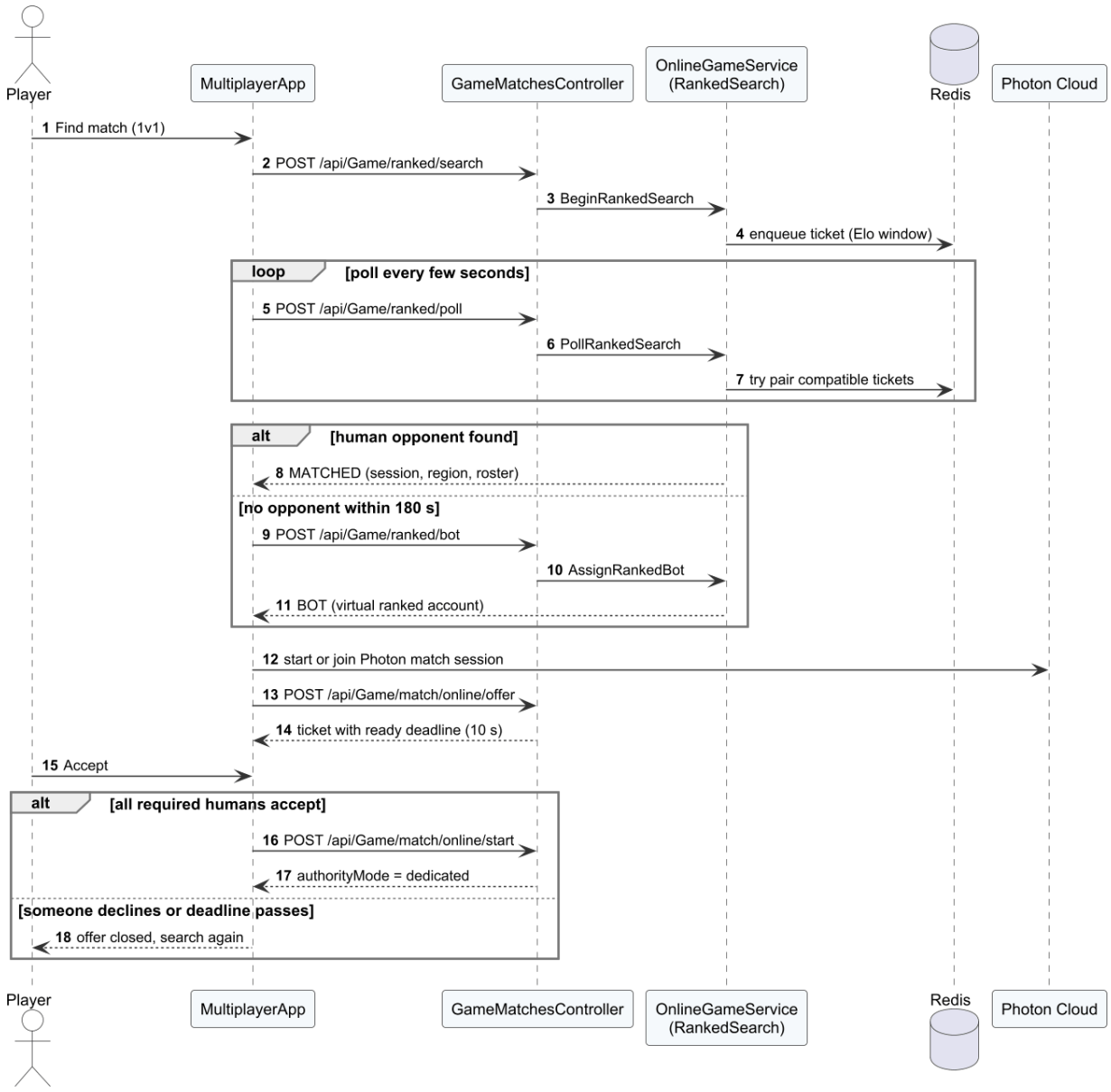


Figure 25. Sequence diagram – Find a ranked match

3.5.5 Invite a friend to a room

The invitation is rejected when the friend is not in the lobby, is blocked or the room is full; otherwise the friend polls and accepts it.

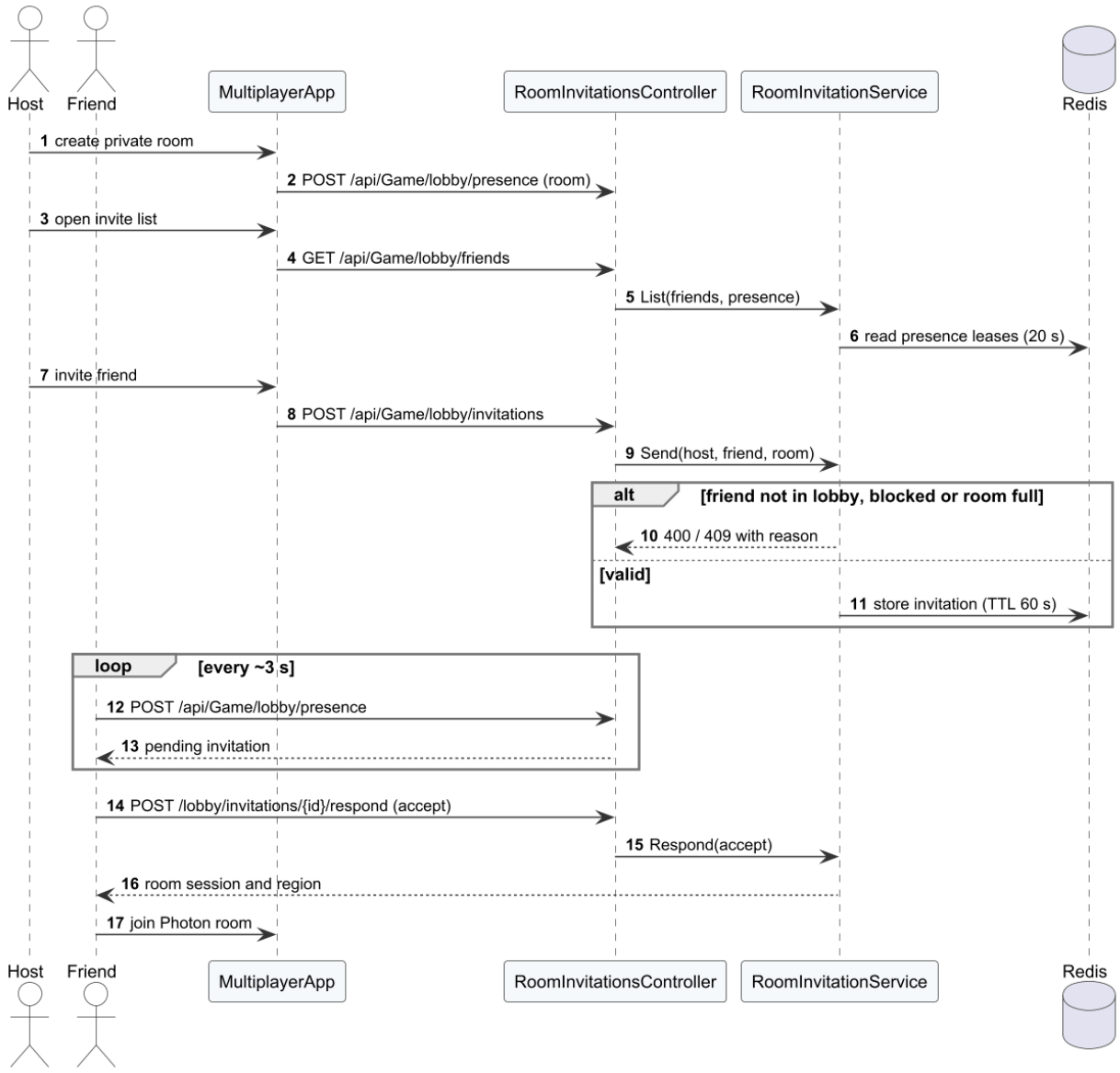


Figure 26. Sequence diagram – Invite a friend to a room

3.5.6 Allocate the dedicated server and play

When the slot is busy the client waits. When the slot is reserved the API signs tickets; the client polls every 500 ms until the server is ready; during play the server sends heartbeats and at the end the result.

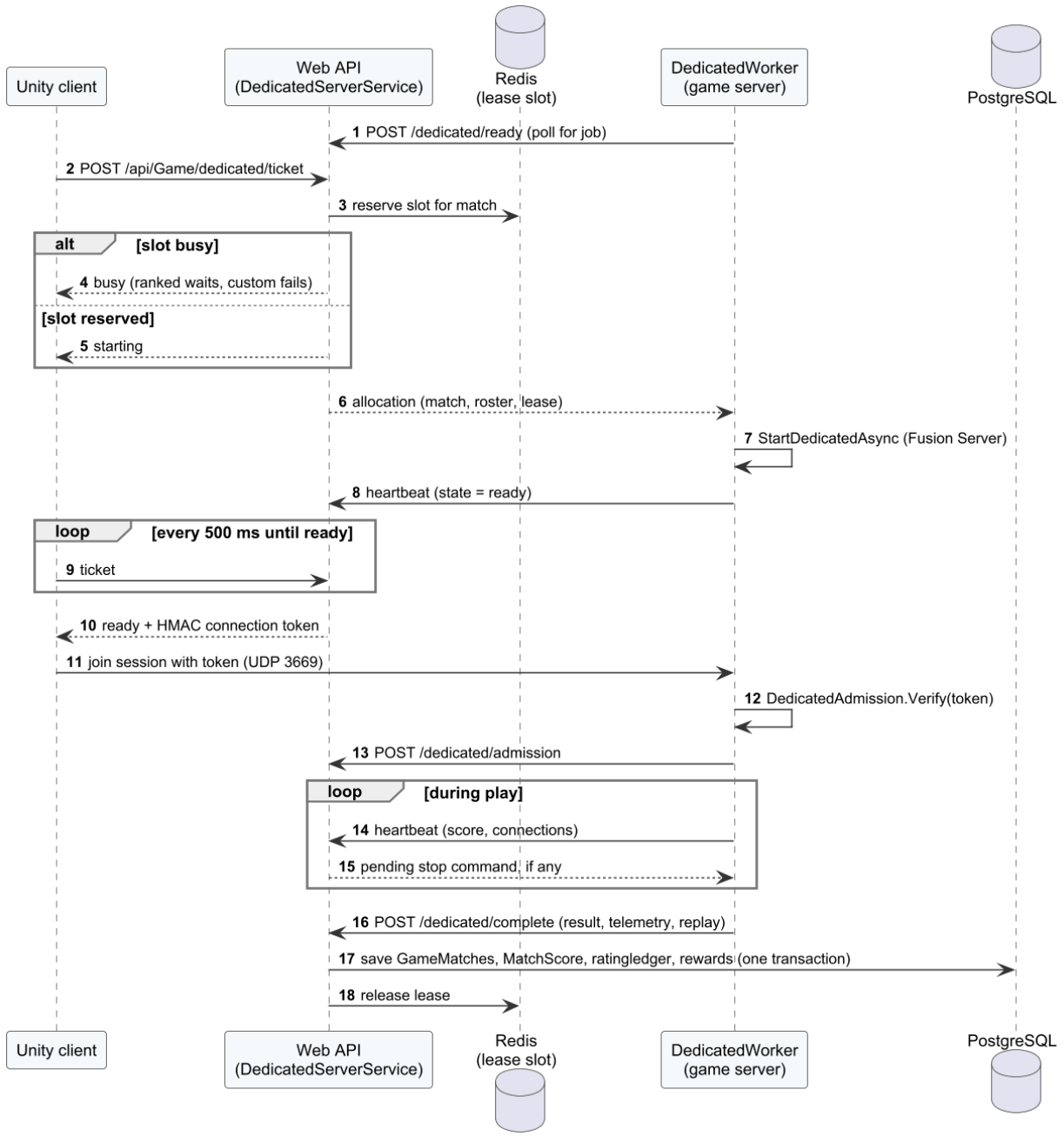


Figure 27. Sequence diagram – Allocate the dedicated server and play

3.6 AI Coaching

AI coaching implements FR-19 and NFR-05/NFR-06. Telemetry is uploaded by CoachingRuntime (drills and bot matches) or DedicatedWorker (online matches). HybridCoachingService stores it and creates an analysis job; HybridAiWorker calls the FastAPI service, which runs the rule engine and, when approved, the tier model.

Table 27. AI coaching parameters

Parameter	Value	Source / meaning
Model features	8	mean_abs_timing, late_rate, early_rate, kitchen_rate, momentum_rate, target_rate, recovery_rate, input_error_rate
Tiers	beginner, intermediate, advanced	tier.py
Minimum confidence / opportunities	0.65 / 20	Below these values no tier is returned
Finding acceptance	precision ≥ 0.80, recall ≥ 0.70	calibration.py, NFR-06
Model approval	real data, ≥ 10 holdout players per tier, macro-F1 ≥ 0.70 and ≥ baseline + 0.05	train.py
Recommendations	at most 3	engine.py

3.6.1 Class Diagram

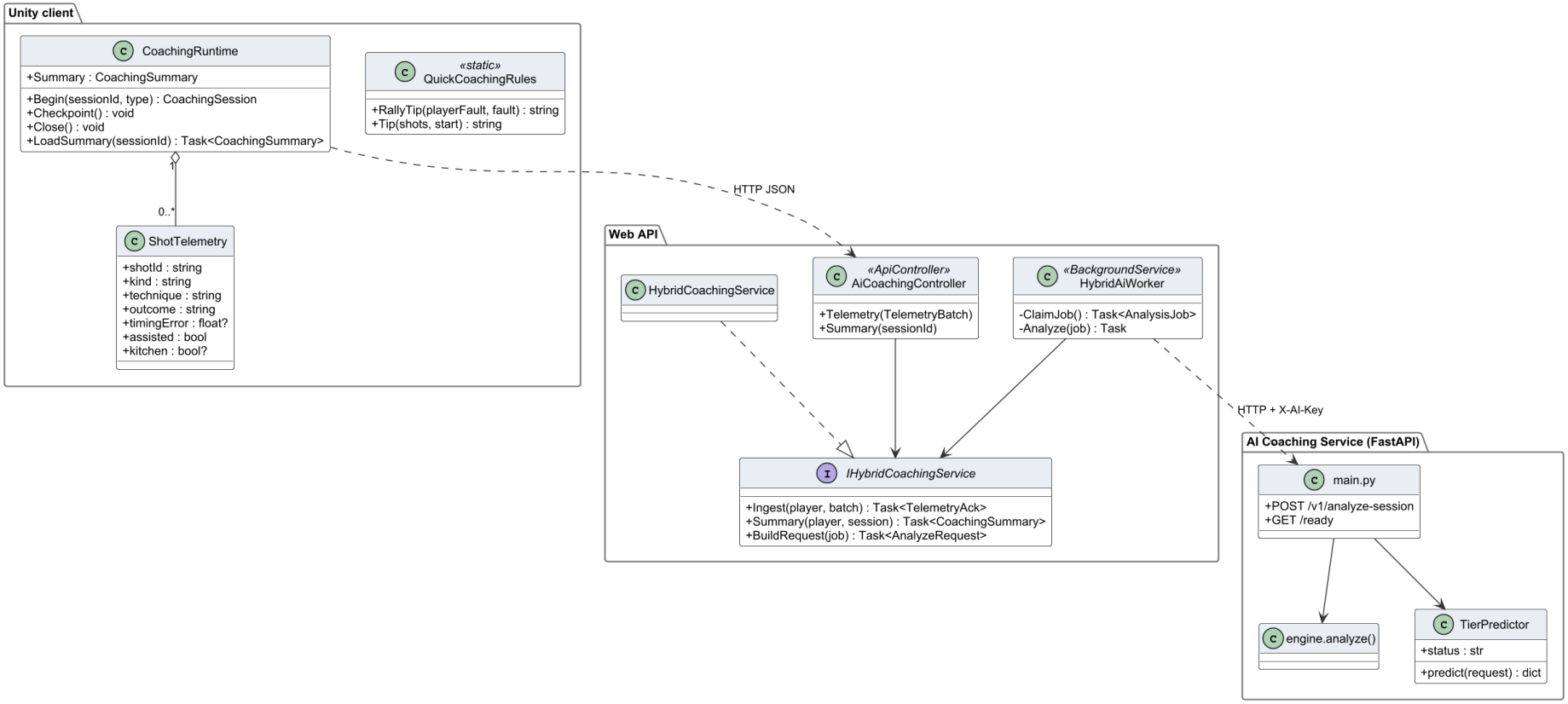


Figure 28. Class diagram – AI coaching

3.6.2 Class Specifications

Table 28. Class specifications – AI coaching

Class	Responsibility	Main attributes	Main operations
QuickCoachingRules «static»	Client-side quick tip shown when offline or before the AI result arrives	–	+ RallyTip(playerFault, fault) : string + Tip(shots, start) : string
CoachingRuntime	Uploads telemetry, polls the analysis job and shows the coaching popup	+ Summary : CoachingSummary	+ Begin(sessionId, type) : CoachingSession + Checkpoint() : void + Close() : void + LoadSummary(sessionId) : Task<CoachingSummary>
ShotTelemetry	Per-shot record: timing, contact, landing, position and errors	+ shotId : string + kind : string + technique : string + outcome : string + timingError : float? + assisted : bool + kitchen : bool?	–
AiCoachingController «ApiController»	REST endpoints for telemetry upload and coaching results	–	+ Telemetry(TelemetryBatch) + Summary(sessionId)
IHybridCoachingService «interface»	Contract of telemetry ingestion and coaching retrieval	–	+ Ingest(player, batch) : Task<TelemetryAck> + Summary(player, session) : Task<CoachingSummary> + BuildRequest(job) : Task<AnalyzeRequest>
HybridCoachingService	Stores telemetry, creates analysis jobs and returns results	–	–
HybridAiWorker «BackgroundService»	Sends pending jobs to the AI service with retries and stores the result	–	- ClaimJob() : Task<AnalysisJob> - Analyze(job) : Task
main.py	FastAPI entry point: /v1/analyze-session and health	+ POST /v1/analyze-session + GET /ready	–
engine.analyze()	Rule-based analysis producing findings, summary and recommendations	–	–
TierPredictor	Loads the approved model and predicts the tier with a confidence threshold	+ status : str	+ predict(request) : dict

3.6.3 Analyse a session

The worker marks the job processing and calls the service; on success the result is stored and the job completed, on failure the job is retried and finally marked failed.

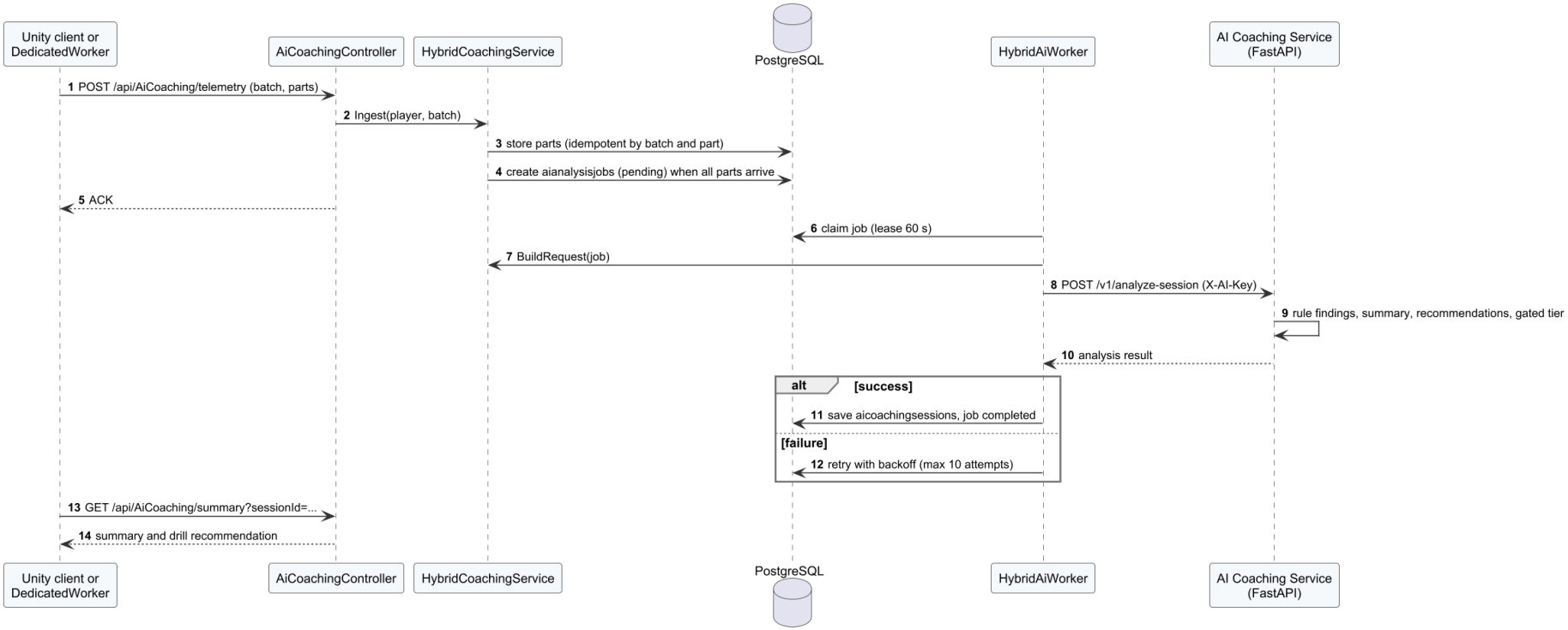


Figure 29. Sequence diagram – Analyse a session

3.7 Progress, Economy, Social and Replays

This group implements FR-20 to FR-26. The client screens share ProgressionRuntime. On the server, OnlineGameService settles rewards and purchases, OnlineSocialService manages friends and the leaderboard, and IMatchReplayService stores replays; GameRepository performs the EF Core queries and row locks inside the unit of work.

3.7.1 Class Diagram

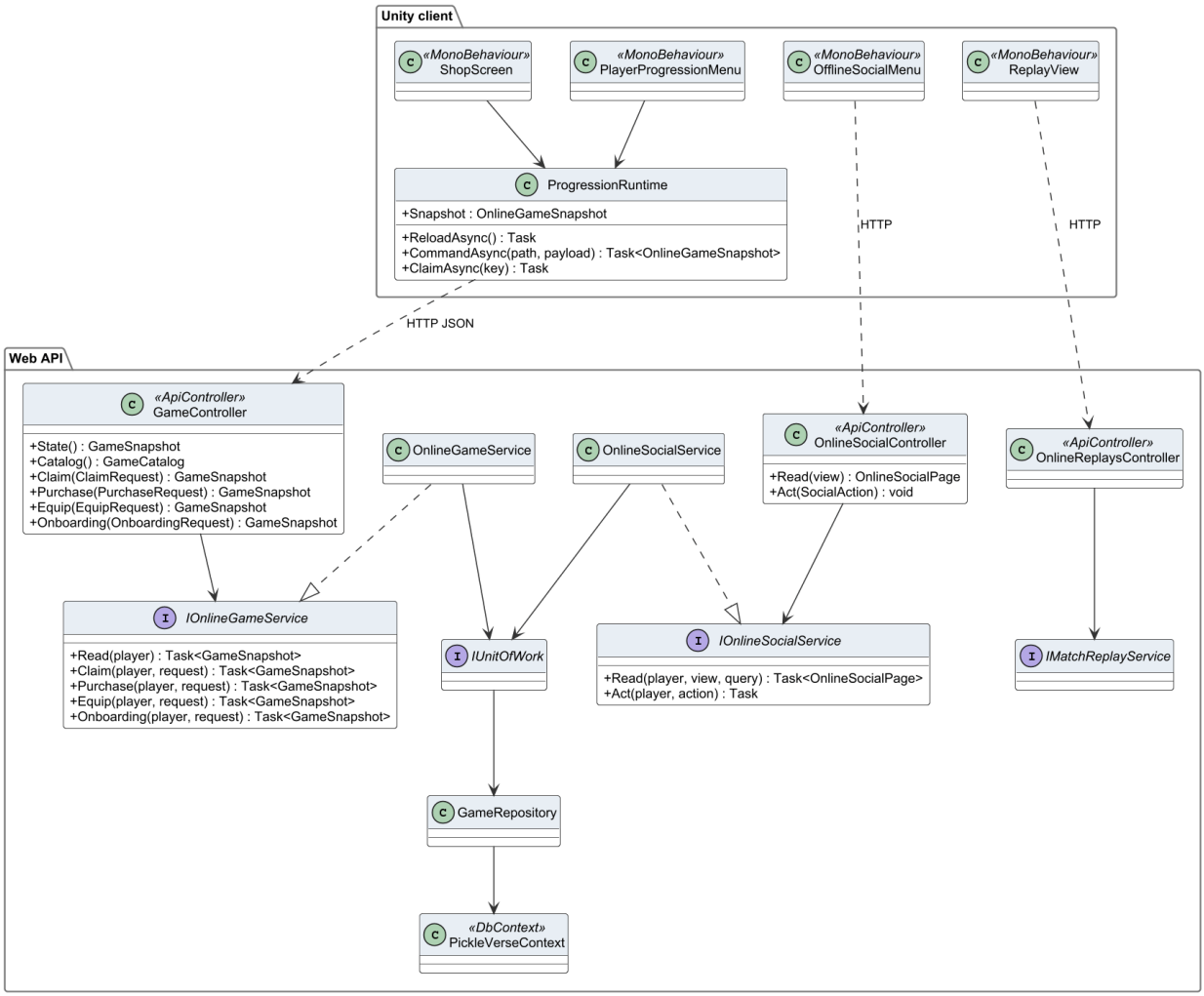


Figure 30. Class diagram – progress, economy, social and replays

3.7.2 Class Specifications

Table 29. Class specifications – progress, economy and social

Class	Responsibility	Main attributes	Main operations
ProgressionRuntime	Client-side cache of profile, wallet, catalogue and the synchronisation queue	+ Snapshot : OnlineGameSnapshot	+ ReloadAsync() : Task + CommandAsync(path, payload) : Task<OnlineGameSnapshot> + ClaimAsync(key) : Task
PlayerProgressionMenu «MonoBehaviour»	Progress, missions and history popups	–	–
OfflineSocialMenu «MonoBehaviour»	Friends and leaderboard drawer	–	–

Class	Responsibility	Main attributes	Main operations
ShopScreen «MonoBehaviour»	Shop scene: list, preview and purchase	–	–
ReplayView «MonoBehaviour»	Replay viewer: download and playback controls	–	–
GameController «ApiController»	REST endpoints for profile, wallet, catalogue, purchase and claims	–	+ State() : GameSnapshot + Catalog() : GameCatalog + Claim(ClaimRequest) : GameSnapshot + Purchase(PurchaseRequest) : GameSnapshot + Equip(EquipRequest) : GameSnapshot + Onboarding(OnboardingRequest) : GameSnapshot
OnlineSocialController «ApiController»	REST endpoints for friends, blocking and leaderboard	–	+ Read(view) : OnlineSocialPage + Act(SocialAction) : void
OnlineReplaysController «ApiController»	REST endpoints for replay upload and download	–	–
IOnlineGameService «interface»	Contract of progression, economy and match use cases	–	+ Read(player) : Task<GameSnapshot> + Claim(player, request) : Task<GameSnapshot> + Purchase(player, request) : Task<GameSnapshot> + Equip(player, request) : Task<GameSnapshot> + Onboarding(player, request) : Task<GameSnapshot>
IOnlineSocialService «interface»	Contract of social use cases	–	+ Read(player, view, query) : Task<OnlineSocialPage> + Act(player, action) : Task
IMatchReplayService «interface»	Contract of replay storage	–	–
OnlineGameService	Server implementation of training, match settlement, matchmaking and economy use cases	–	–
OnlineSocialService	Friend requests, friendships, blocks and leaderboard cache	–	–
IUnitOfWork «interface»	Gives services access to repositories and one shared transaction	–	–
GameRepository	EF Core queries and locks for matches, progression and economy	–	–
PickleVerseContext «DbContext»	EF Core DbContext mapping the 51 tables	–	–

3.7.3 Purchase a cosmetic

The command ID makes the purchase idempotent: an already processed request returns the stored receipt; otherwise ownership and balance are checked and the debit, ledger entry and ownership are written in one transaction.

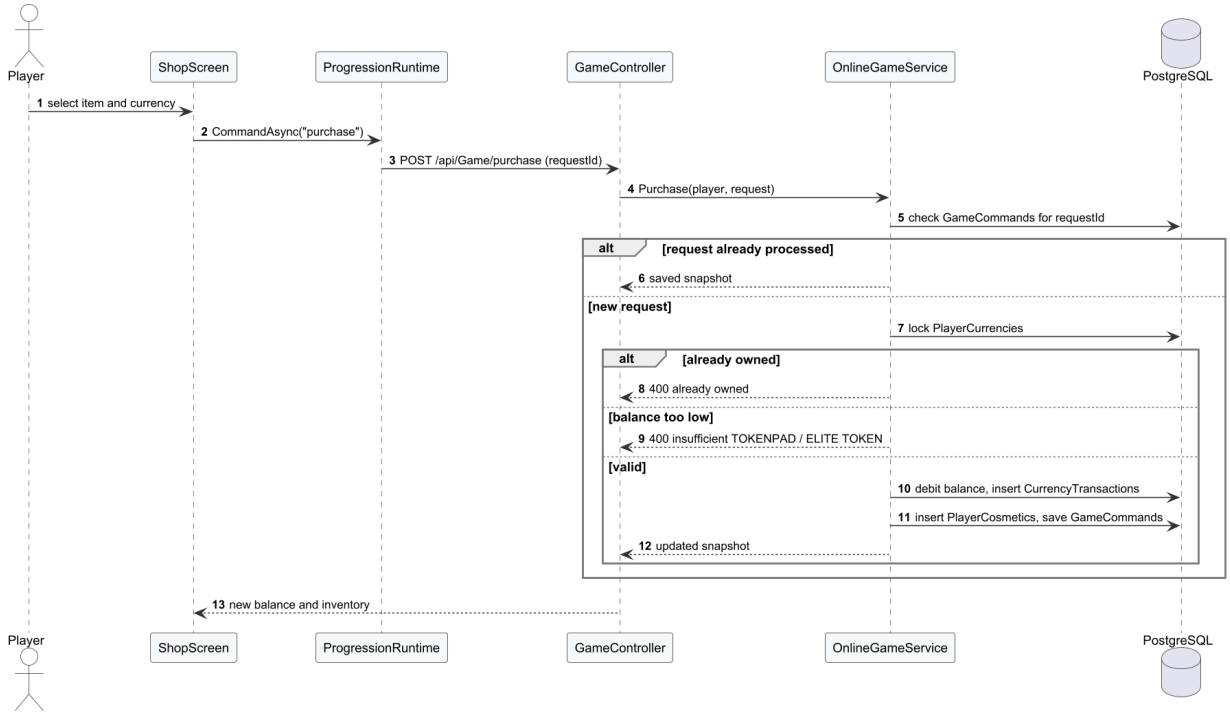


Figure 31. Sequence diagram – Purchase a cosmetic

3.8 Administration and Moderation

Administration implements FR-27 to FR-33. Every admin controller requires the Admin role; services write an AdminAudits record in the same transaction as the change.

3.8.1 Class Diagram

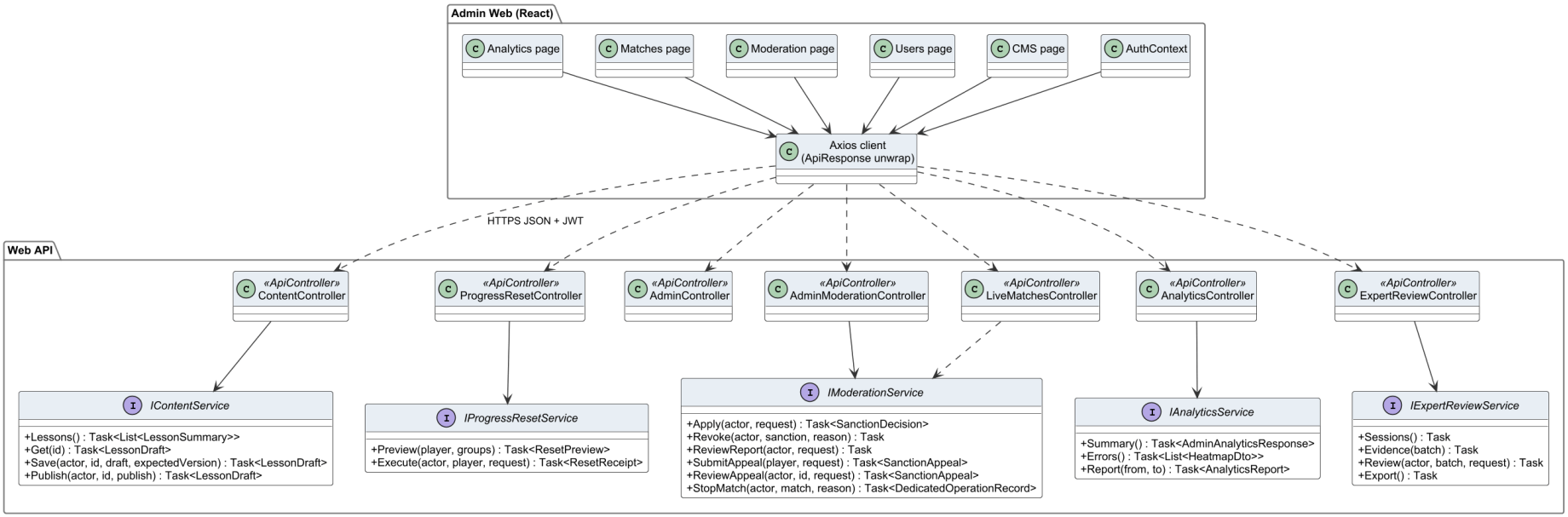


Figure 32. Class diagram – administration web and admin API

3.8.2 Class Specifications

Table 30. Class specifications – administration

Class	Responsibility	Main attributes	Main operations
AuthContext	Admin web session state and role check	–	–
CMS page	Lesson, drill, item and mission forms	–	–
Users page	Account search and actions	–	–
Moderation page	Reports, sanctions and appeals	–	–
Matches page	Live matches and stop dialog	–	–
Analytics page	Charts for the selected date range	–	–
Axios client (ApiResponse unwrap)	HTTP client that adds the JWT and unwraps the ApiResponse envelope	–	–
ContentController «ApiController»	Admin endpoints for lessons and catalogue	–	–
ProgressResetController «ApiController»	Admin endpoints for reset preview and execution	–	–
AdminController «ApiController»	Admin endpoints for users, roles, sanctions and e-mail activation	–	–
AdminModerationController «ApiController»	Admin endpoints for reports, appeals and match stop	–	–
LiveMatchesController «ApiController»	Admin endpoints for live and finished matches	–	–
AnalyticsController «ApiController»	Admin endpoints for analytics summary and error heat map	–	–
ExpertReviewController «ApiController»	Admin endpoints for expert labelling and dataset export	–	–
IContentService «interface»	Contract of content management	–	+ Lessons() : Task<List<LessonSummary>> + Get(id) : Task<LessonDraft> + Save(actor, id, draft, expectedVersion) : Task<LessonDraft> + Publish(actor, id, publish) : Task<LessonDraft>
IProgressResetService «interface»	Contract of progress reset	–	+ Preview(player, groups) : Task<ResetPreview> + Execute(actor, player, request) : Task<ResetReceipt>
IModerationService «interface»	Contract of moderation use cases	–	+ Apply(actor, request) : Task<SanctionDecision> + Revoke(actor, sanction, reason) : Task + ReviewReport(actor, request) : Task + SubmitAppeal(player, request) : Task<SanctionAppeal> + ReviewAppeal(actor, id, request) : Task<SanctionAppeal> + StopMatch(actor, match, reason) : Task<DedicatedOperationRecord>
IAnalyticsService «interface»	Contract of analytics queries	–	+ Summary() : Task<AdminAnalyticsResponse> + Errors() : Task<List<HeatmapDto>> + Report(from, to) : Task<AnalyticsReport>
IExpertReviewService «interface»	Contract of expert review and export	–	+ Sessions() : Task + Evidence(batch) : Task + Review(actor, batch, request) : Task + Export() : Task

3.8.3 Report, sanction and appeal

A player reports; the administrator accepts the report and applies a sanction; the restricted player appeals; the administrator accepts or declines the appeal.

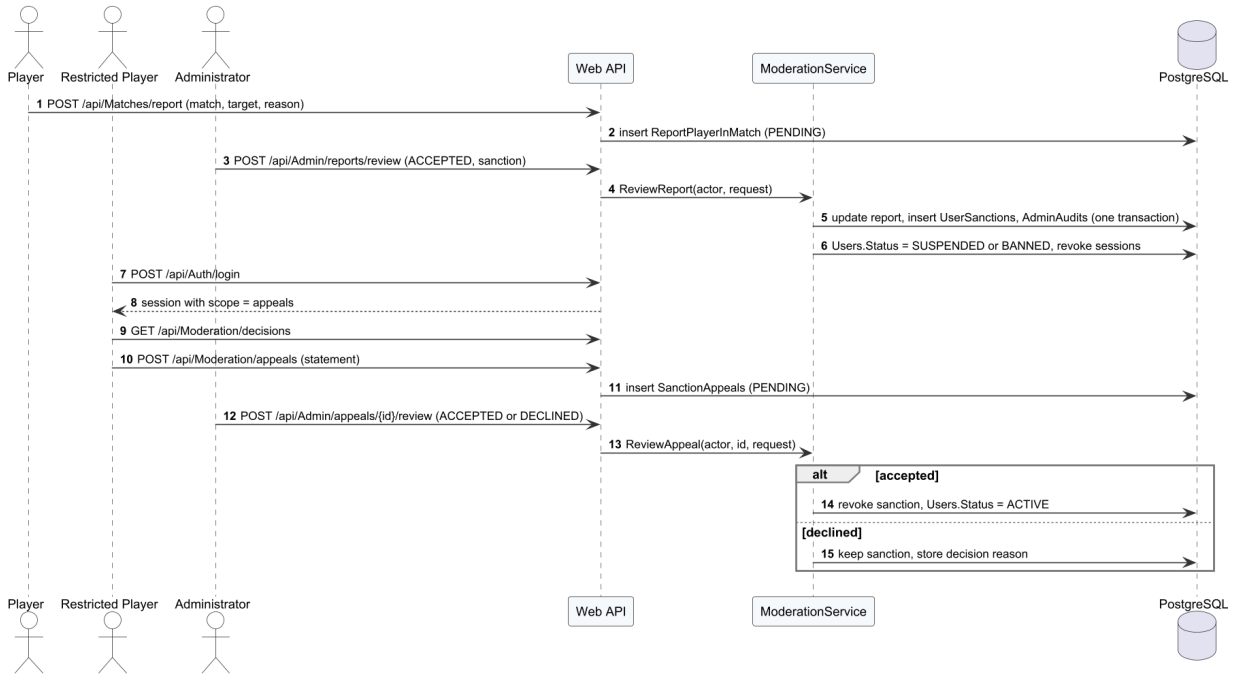


Figure 33. Sequence diagram – Report, sanction and appeal

3.8.4 Reset player progress

The dialog previews the impact; execution is refused when the revision changed after the preview.

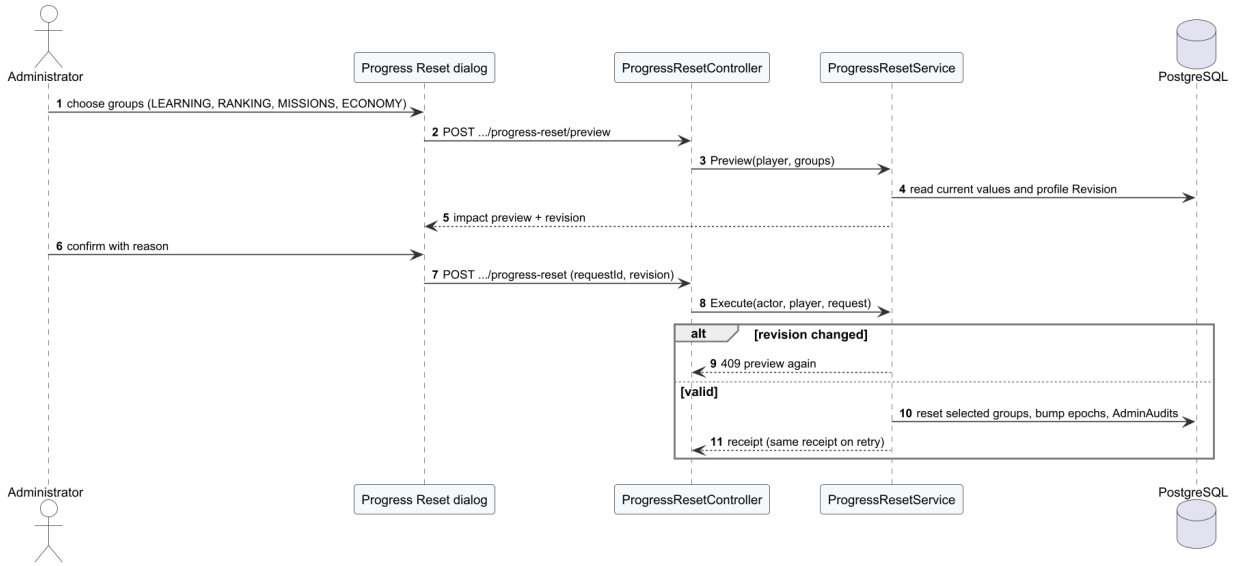


Figure 34. Sequence diagram – Reset player progress

3.8.5 Publish a lesson

Publishing is refused when the expected version changed; otherwise the lesson becomes PUBLISHED and a new catalogue version is produced.

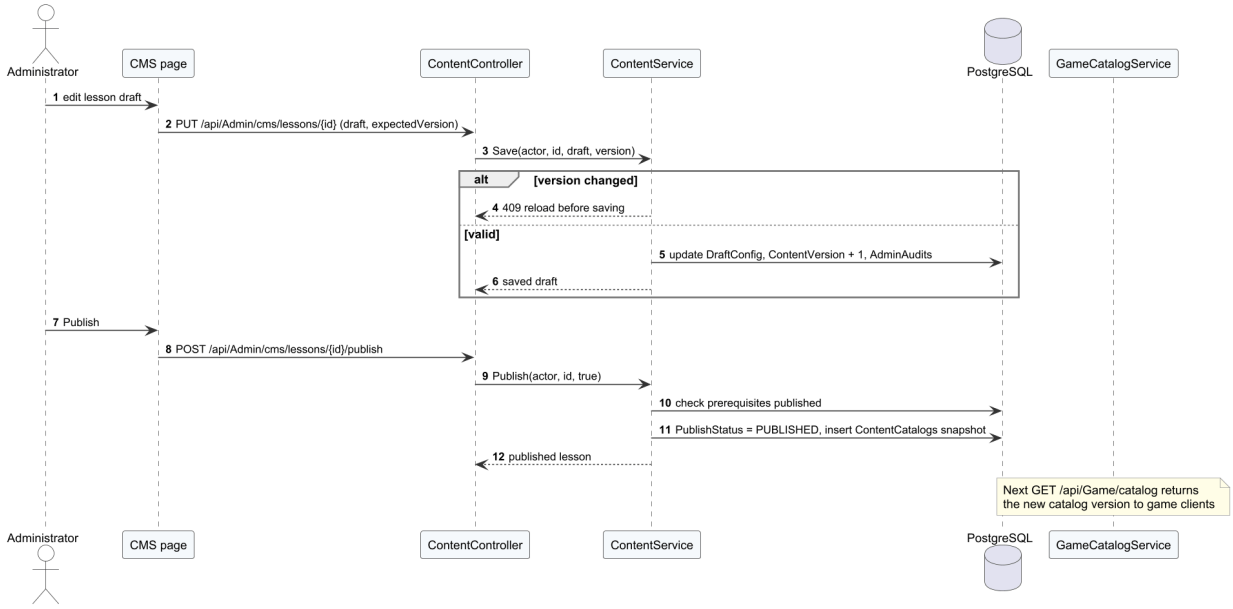


Figure 35. Sequence diagram – Publish a lesson

3.8.6 Stop a live match

The stop request is stored as a dedicated operation; the server reads it with the next heartbeat, ends the match as interrupted and acknowledges.

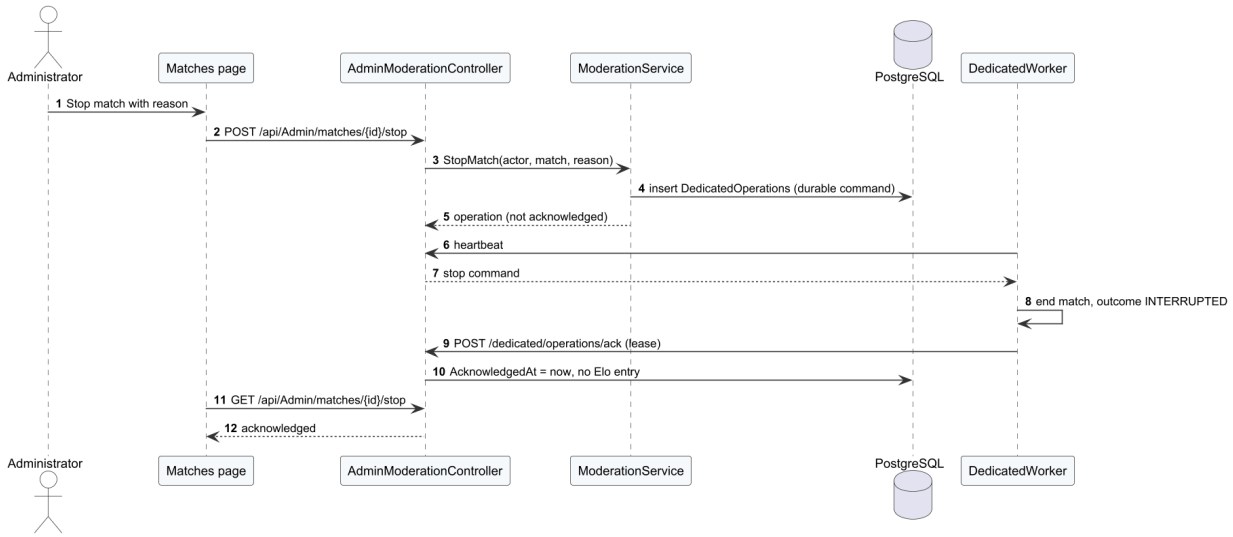


Figure 36. Sequence diagram – Stop a live match

3.9 API Design

The Web API exposes 122 REST endpoints in 24 controllers. Table below lists the conventions, and the full list of endpoints with their access level follows.

Table 31. API conventions

Item	Design
Base path and format	/api/<Controller>/...; JSON (UTF-8); enum values serialised as strings

Item	Design
Envelope	Success: ApiResponse<T> {success: true, message, data}. Error: ApiResponse {success: false, message, code, errors}
Status codes	400 validation, 401 missing or expired token, 403 role or scope not allowed, 404 not found, 409 conflict or stale version, 429 rate limit, 503 dependency unavailable, 500 internal (no details)
Authentication	Bearer JWT (HS256) with claims sub/NameIdentifier, email, role (Player, Admin or Restricted), access_scope (game or appeals) and auth_version; refresh tokens rotate
Rate limiting	Policy auth: 20 requests per minute per IP on authentication endpoints; rejection returns 429 with Retry-After: 60
Idempotency	Commands carry a request/command ID stored in GameCommands or receipts
Service calls	Dedicated server: service key header and HMAC-signed tickets; AI service: X-AI-Key header
Documentation	OpenAPI document generated at runtime with the ApiResponse schema

Table 32. Web API endpoints (122)

Controller	Method	Route	Action	Access
Achievements	GET	/api/Achievements	GetMyAchievements	Player
Achievements	POST	/api/Achievements/{achievementId}/claim	ClaimReward	Player
Admin	POST	/api/Admin/cms/cosmetics	CreateCosmetic	Admin
Admin	POST	/api/Admin/cms/drills	CreateDrill	Admin
Admin	POST	/api/Admin/cms/lessons	CreateLesson	Admin
Admin	POST	/api/Admin/cms/missions	CreateMission	Admin
Admin	GET	/api/Admin/matches/recent	GetRecentMatches	Admin
Admin	GET	/api/Admin/reports	GetReports	Admin
Admin	POST	/api/Admin/reports/review	ReviewReport	Admin
Admin	GET	/api/Admin/users	GetUsers	Admin
Admin	POST	/api/Admin/users/assign-role	AssignRole	Admin
Admin	POST	/api/Admin/users/ban	BanUser	Admin
Admin	POST	/api/Admin/users/{playerId}/reset-elo	ResetPlayerElo	Admin
Admin	POST	/api/Admin/users/{userId}/unban	UnbanUser	Admin
Admin	POST	/api/Admin/users/{userId}/verify-email	ManualVerifyUserEmail	Admin
AiCoaching	POST	/api/AiCoaching/log-error	LogTechniqueError	Player
AiCoaching	GET	/api/AiCoaching/mastery	GetMastery	Player
AiCoaching	GET	/api/AiCoaching/recommendation	GetRecommendation	Player
AiCoaching	GET	/api/AiCoaching/summary	Summary	Player
AiCoaching	POST	/api/AiCoaching/telemetry	Telemetry	Player
Analytics	GET	/?/error-heatmap	Errors	Player
Analytics	GET	/?/report	Report	Player
Analytics	GET	/?/summary	Summary	Player
AppActivity	POST	/api/Game/activity/checkpoint	Activity	Player
AppActivity	POST	/api/Game/activity/open	OpenActivity	Player
Auth	POST	/api/Auth/forgot-password	ForgotPassword	Anonymous
Auth	POST	/api/Auth/login	Login	Anonymous
Auth	POST	/api/Auth/logout	Logout	Player
Auth	POST	/api/Auth/refresh-token	RefreshToken	Anonymous
Auth	POST	/api/Auth/register	Register	Anonymous
Auth	POST	/api/Auth/resend-otp	ResendOtp	Anonymous
Auth	POST	/api/Auth/reset-password	ResetPassword	Anonymous
Auth	POST	/api/Auth/verify-email	VerifyEmail	Anonymous
Content	GET	/?	List	Player
Content	GET	/?/references	References	Player
Content	GET	/?/{id:int}	Get	Player
Content	PUT	/?/{id:int}	Save	Player
Content	POST	/?/{id:int}/publish	Publish	Player
Content	POST	/?/{id:int}/references	Create	Player
Content	POST	/?/{id:int}/unpublish	Unpublish	Player
Dedicated	POST	/api/Game/dedicated/admission	AuthorizeAdmission	Player
Dedicated	POST	/api/Game/dedicated/complete	Complete	Player
Dedicated	POST	/api/Game/dedicated/heartbeat	Heartbeat	Player
Dedicated	POST	/api/Game/dedicated/operations/ack	Acknowledge	Player
Dedicated	POST	/api/Game/dedicated/ready	Ready	Player
Dedicated	POST	/api/Game/dedicated/release	Release	Player
Dedicated	POST	/api/Game/dedicated/{lease}/replay	Replay	Player
Dedicated	POST	/api/Game/dedicated/{lease}/replay/chunks/{index:int}	Chunk	Player
Dedicated	POST	/api/Game/dedicated/{lease}/telemetry	Telemetry	Player
Economy	POST	/api/Economy/buy	BuyCosmetic	Player
Economy	GET	/api/Economy/currencies	GetCurrencies	Player
Economy	POST	/api/Economy/equip	EquipCosmetic	Player

Controller	Method	Route	Action	Access
Economy	GET	/api/Economy/my-inventory	GetMyCosmetics	Player
Economy	GET	/api/Economy/shop	GetShopItems	Player
ExpertReview	GET	/?/demo-sessions	DemoSessions	Player
ExpertReview	GET	/?/demo-sessions/{sessionId}	DemoEvidence	Player
ExpertReview	GET	/?/sessions	Sessions	Player
ExpertReview	GET	/?/sessions/{batch:guid}	Evidence	Player
ExpertReview	POST	/?/sessions/{batch:guid}/review	Review	Player
Friends	GET	/api/Friends	GetFriends	Player
Friends	POST	/api/Friends/block/{targetUserId}	BlockUser	Player
Friends	POST	/api/Friends/request	SendFriendRequest	Player
Friends	POST	/api/Friends/respond	RespondFriendRequest	Player
Friends	DELETE	/api/Friends/{friendId}	RemoveFriend	Player
Game	GET	/api/Game/catalog	Catalog	Player
Game	POST	/api/Game/claim	Claim	Player
Game	POST	/api/Game/equip	Equip	Player
Game	POST	/api/Game/onboarding	Onboarding	Player
Game	POST	/api/Game/purchase	Purchase	Player
Game	GET	/api/Game/state	State	Player
Game	POST	/api/Game/tutorial	Tutorial	Player
GameMatches	POST	/api/Game/match/bot/complete	FinishBot	Player
GameMatches	POST	/api/Game/match/bot/start	StartBot	Player
GameMatches	POST	/api/Game/match/online/heartbeat	HeartbeatOnline	Player
GameMatches	POST	/api/Game/match/online/leave	LeaveOnline	Player
GameMatches	POST	/api/Game/match/online/offer	OfferOnline	Player
GameMatches	POST	/api/Game/match/online/prepare	PrepareOnline	Player
GameMatches	POST	/api/Game/match/online/report	ReportOnline	Player
GameMatches	POST	/api/Game/match/online/start	StartOnline	Player
GameMatches	POST	/api/Game/ranked/bot	Bot	Player
GameMatches	POST	/api/Game/ranked/cancel	CancelSearch	Player
GameMatches	POST	/api/Game/ranked/poll	Poll	Player
GameMatches	POST	/api/Game/ranked/room-ready	PublishRoom	Player
GameMatches	POST	/api/Game/ranked/search	Search	Player
GameTraining	POST	/api/Game/training/checkpoint	Checkpoint	Player
GameTraining	POST	/api/Game/training/complete	Complete	Player
GameTraining	POST	/api/Game/training/start	Start	Player
GameTraining	POST	/api/Game/training/sync	SyncTraining	Player
Matches	POST	/api/Matches/finish	FinishMatch	Player
Matches	GET	/api/Matches/my-history	GetMyMatchHistory	Player
Matches	POST	/api/Matches/report	ReportPlayer	Player
Missions	GET	/api/Missions	GetMyMissions	Player
Missions	POST	/api/Missions/{missionId}/claim	ClaimReward	Player
Moderation	GET	/?/appeals	Appeals	Player
Moderation	GET	/?/appeals	Appeals	Player
Moderation	POST	/?/appeals	Appeal	Player
Moderation	POST	/?/appeals/{id:guid}/review	Review	Player
Moderation	GET	/?/decisions	Decisions	Player
Moderation	POST	/?/matches/{match:guid}/stop	Stop	Player
Moderation	GET	/?/sanctions	Sanctions	Player
OnlineReplays	GET	/api/Game/replays	List	Player
OnlineReplays	GET	/api/Game/replays/{match:guid}	Read	Player
OnlineReplays	PUT	/api/Game/replays/{match:guid}	Save	Player
OnlineReplays	GET	/api/Game/replays/{match:guid}/chunks/{index:int}	Chunk	Player
OnlineReplays	PUT	/api/Game/replays/{match:guid}/chunks/{index:int}	Upload	Player
OnlineSocial	GET	/api/Game/social	Read	Player
OnlineSocial	POST	/api/Game/social	Act	Player
Profiles	GET	/api/Profiles/leaderboard	GetLeaderboard	Player
Profiles	GET	/api/Profiles/me	GetMyProfile	Player
Profiles	PUT	/api/Profiles/me	UpdateProfile	Player
Profiles	POST	/api/Profiles/sync-time	SyncTime	Player
Profiles	GET	/api/Profiles/{playerId}	GetProfileById	Player
ProgressReset	POST	/?	Reset	Player
ProgressReset	POST	/?/preview	Preview	Player
Replays	POST	/api/Replays/save	SaveReplay	Player
Replays	GET	/api/Replays/{matchId}	GetReplay	Player
RoomInvitations	GET	/api/Game/lobby/friends	Friends	Player
RoomInvitations	POST	/api/Game/lobby/invitations	Send	Player
RoomInvitations	POST	/api/Game/lobby/invitations/{id}/respond	Respond	Player
RoomInvitations	POST	/api/Game/lobby/presence	Presence	Player
Training	GET	/api/Training/lessons	GetLessons	Player
Training	POST	/api/Training/submit-drill	SubmitDrill	Player

3.10 Security Design

Table 33. Security design

Concern	Design
Passwords	BCrypt hash with salt; 6–128 characters; never logged
Tokens	JWT HS256, 2-hour access token; refresh token 14 days stored as SHA-256 hash, rotated on use; auth_version in the token is incremented to revoke all sessions (password reset, ban)
Codes	OTP and reset codes stored as HMAC, 10-minute expiry, 5 attempts, 60-second resend cooldown
Authorisation	Role-based ([Authorize(Roles = "Admin")]) plus access scope: restricted accounts receive scope appeals and can call only moderation endpoints; resource ownership is checked in services (own matches, replays, telemetry)
Transport	HTTPS with TLS 1.2+ in production behind nginx; the staging environment currently uses HTTP and must be switched before release (Report 5 defect list)
Secrets	Read from environment variables and the server .env file. The committed appsettings.json still contains development credentials and Program.cs contains a fallback JWT key; both are tracked as security defects and must be removed and rotated
Abuse protection	Rate limiting on authentication; server-authoritative online results; idempotent commands; validation of telemetry ownership; HMAC tickets bind a player to a match and slot
CORS	AdminWebPolicy allows only the admin web origins; production serves the admin web and API under the same origin through nginx
Data protection	Other players never receive e-mail addresses; audit log for administrative actions; daily database backup
Error handling	GlobalExceptionHandler maps AppException types to status codes and hides internal details of unexpected exceptions

4. Workflows and State Changes

4.1 Activity Diagrams

Each review workflow is owned by one team member and is drawn as a UML activity diagram with one swimlane per participant, decisions and exception branches.

W01 – Learn and sync a drill (owner: Nguyễn Gia Huy)

The player signs in or opens offline training; a beginner is sent to the tutorial; the player practises a drill; the attempt is applied online or queued offline and synchronised when the network returns.

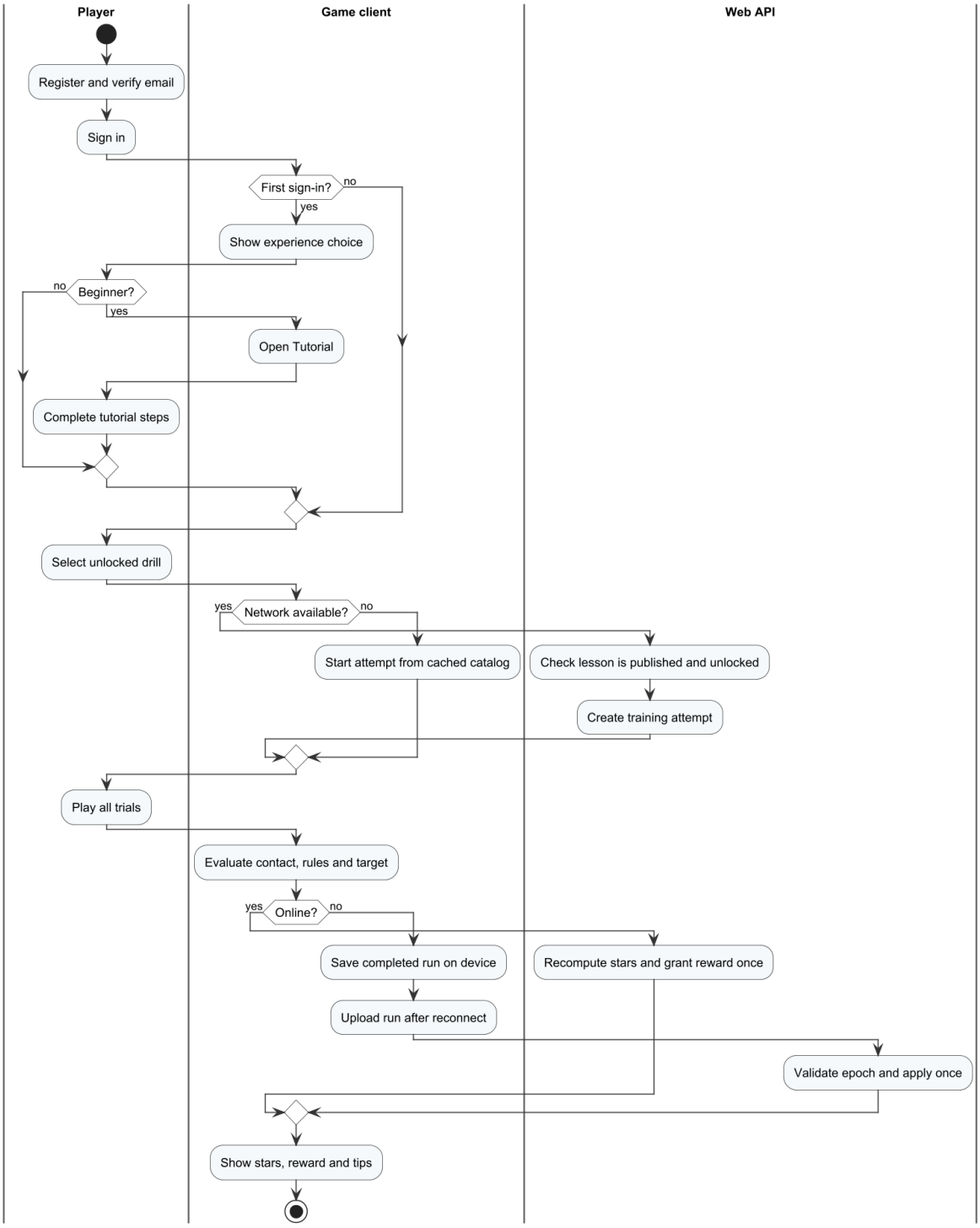


Figure 37. W01 – Learn and sync a drill (owner: Nguyễn Gia Huy)

W02 – Play a bot match and get a tip (owner: Trần Nguyễn Hiếu Thuận)

The player sets up a bot match and plays rallies until a side wins; the API settles the result, the AI service analyses the telemetry and the player reads the tip and opens the recommended drill.

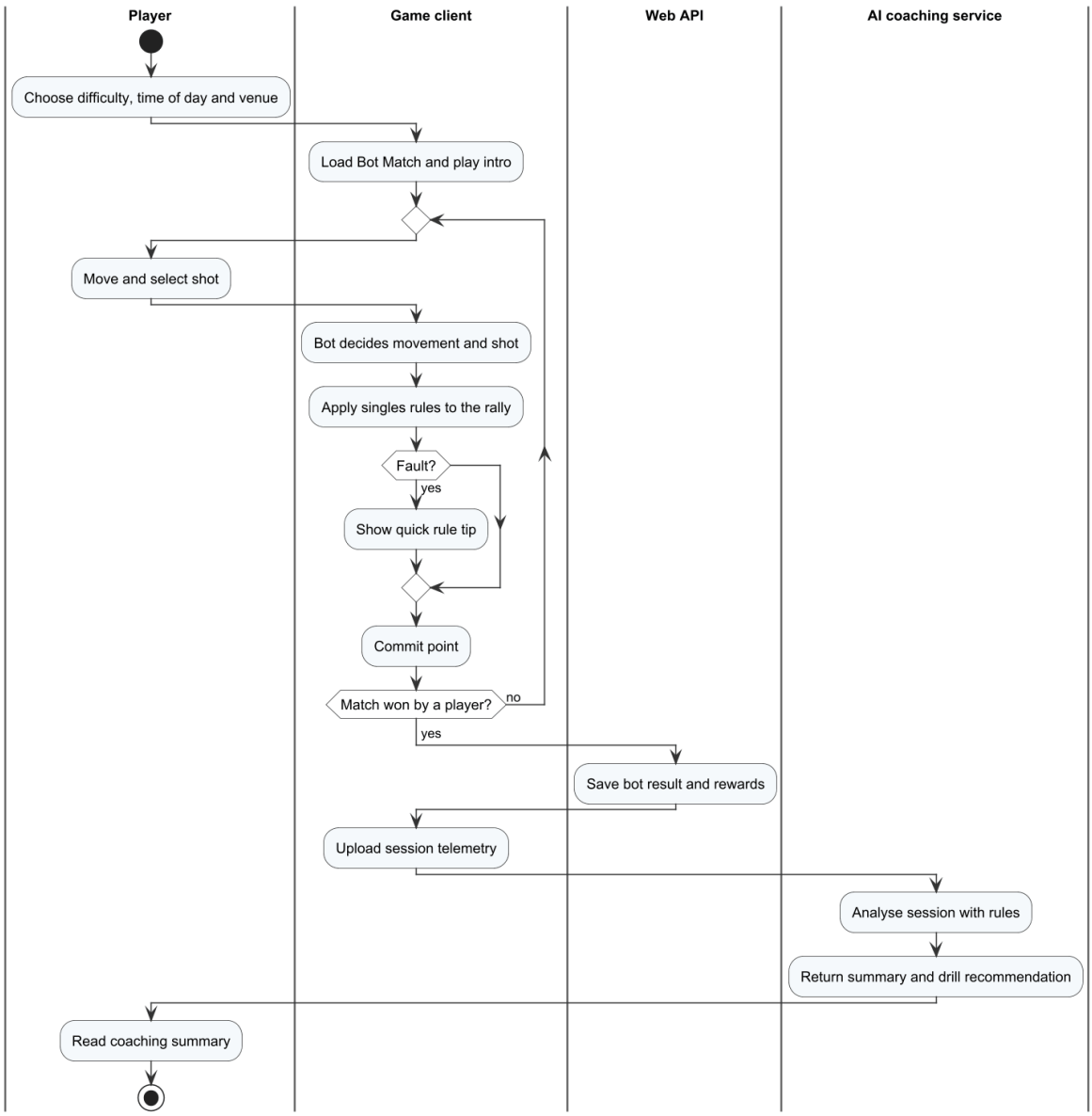


Figure 38. W02 – Play a bot match and get a tip (owner: Trần Nguyễn Hiếu Thuận)

W03 – Find and play an online match (owner: Nguyễn Văn Thành Đạt)

The player either creates a private room or searches a ranked match; the bot fallback applies after 180 seconds; all players must accept; the dedicated server runs the match unless it is stopped or the lease is lost.

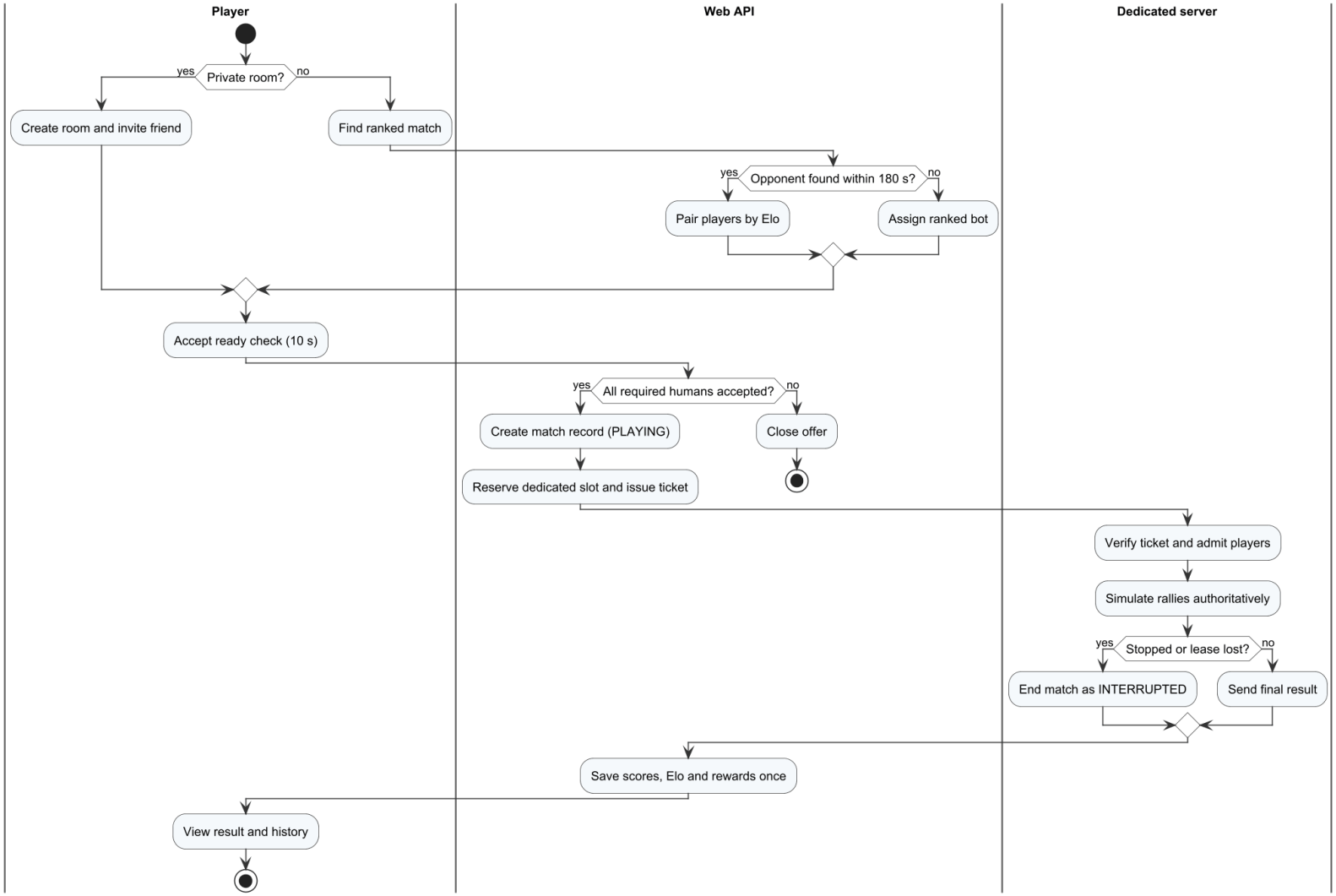


Figure 39. W03 – Find and play an online match (owner: Nguyễn Văn Thành Đạt)

W04 – Report, sanction and appeal (owner: Huỳnh Trung Kiên)

A player reports an opponent; the administrator reviews and may sanction; the restricted player may appeal; the administrator decides the appeal.

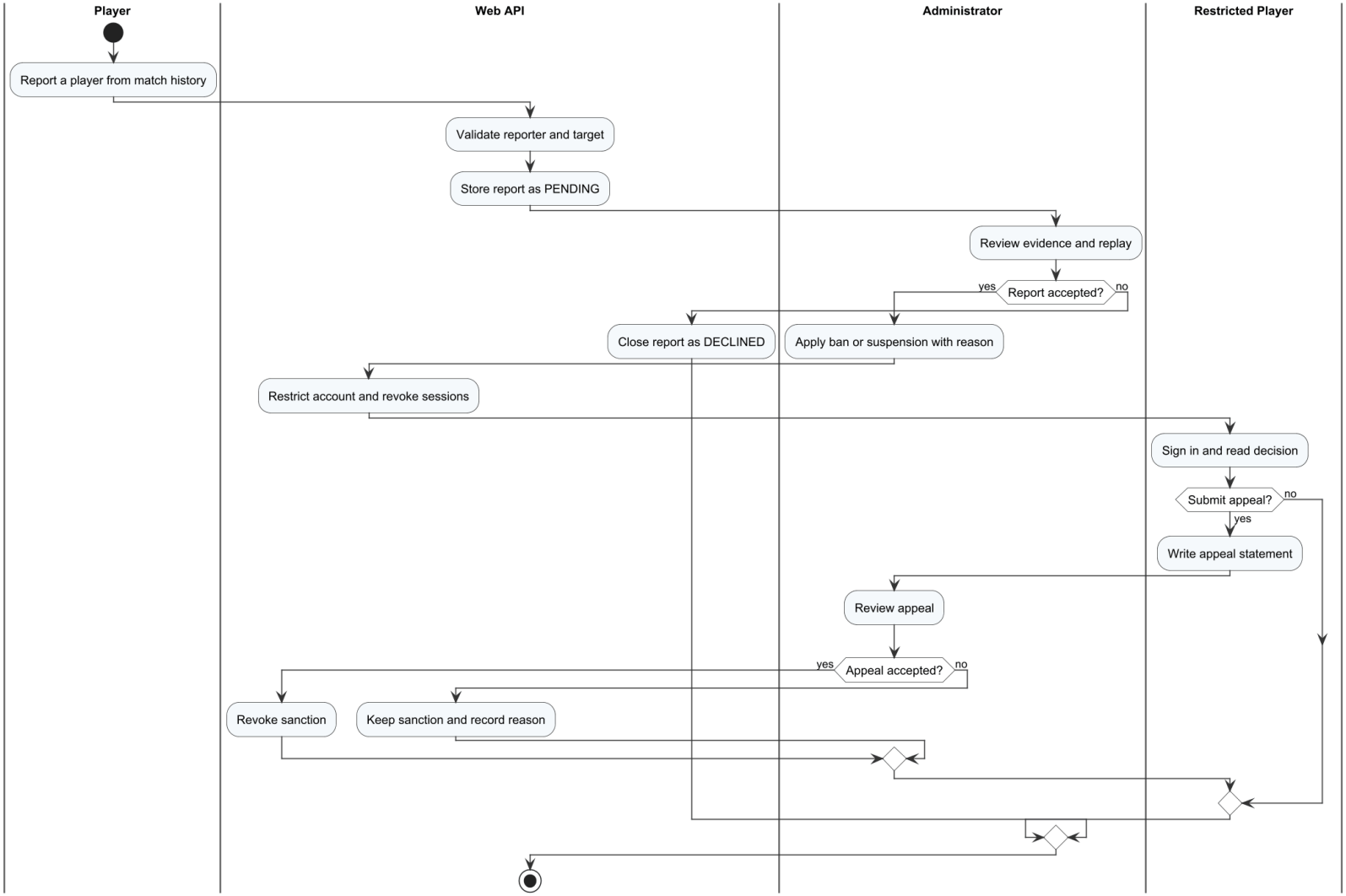


Figure 40. W04 – Report, sanction and appeal (owner: Huỳnh Trung Kiên)

Supporting workflow – publish training content

The administrator edits and publishes a lesson; the API checks the expected version and that prerequisites are published; clients receive the new catalogue version.

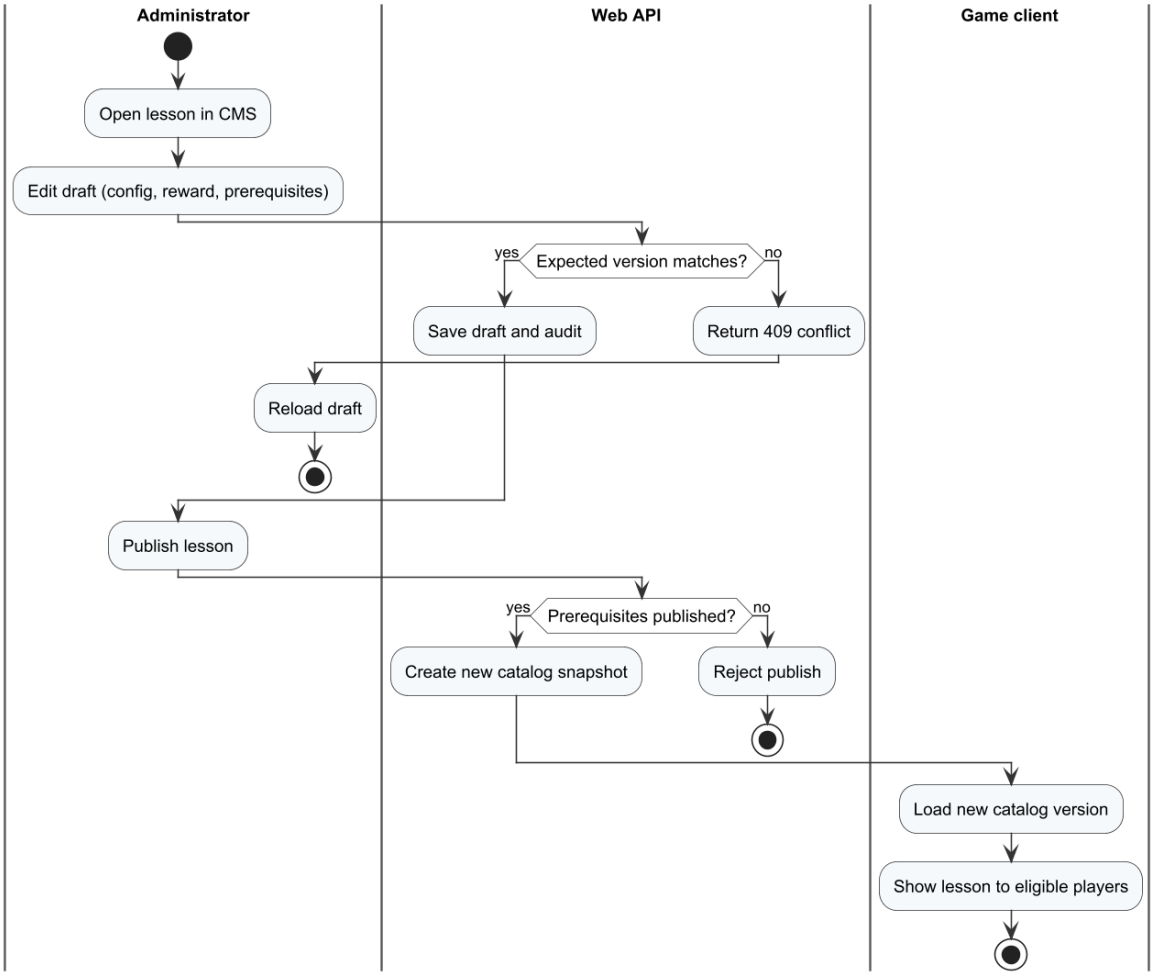


Figure 41. Supporting workflow – publish training content

4.2 State Machine Diagrams

The following state machines specify every entity or runtime object with a life cycle. Each transition is labelled event [guard] / action.

SM-01 Account status

ACTIVE, SUSPENDED and BANNED; suspensions end by expiry or revocation, and an accepted appeal revokes the sanction.

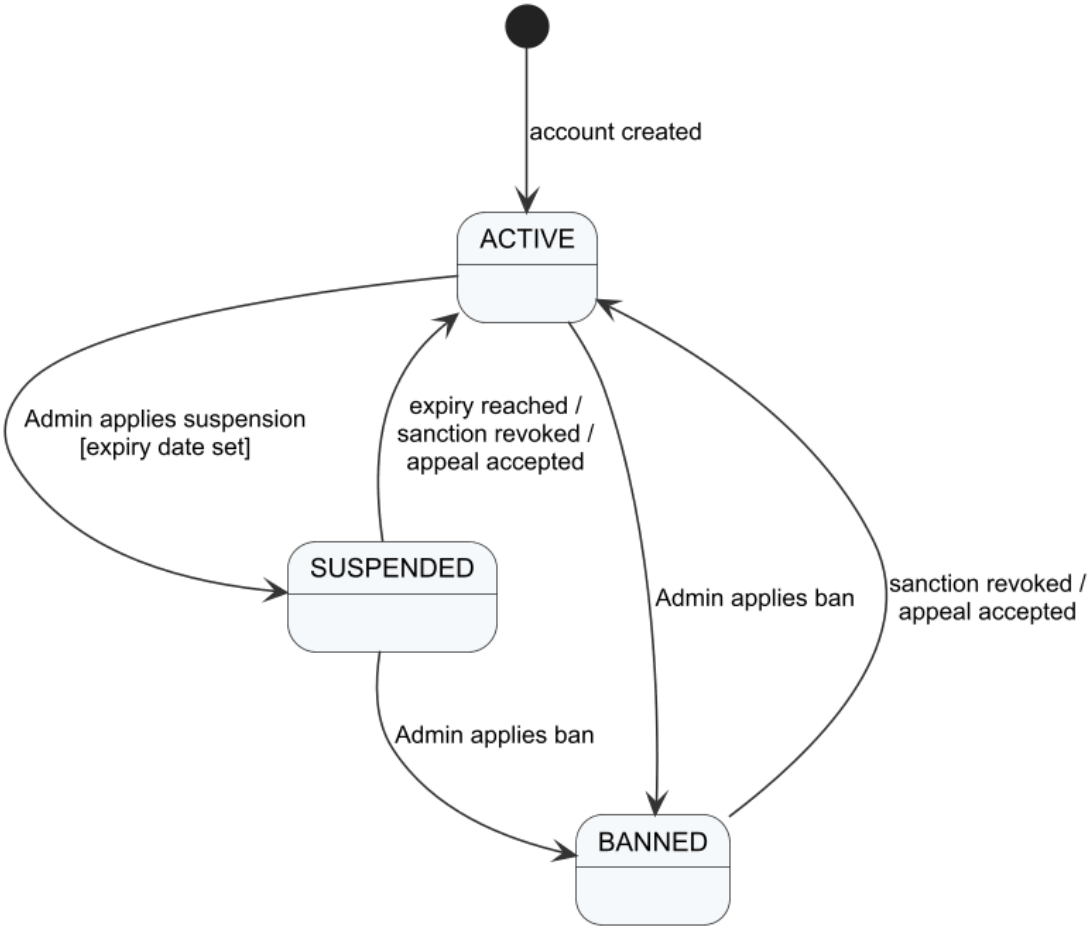


Figure 42. SM-01 Account status

SM-02 E-mail verification code

Pending, verified, expired and locked after 5 wrong attempts; a resend creates a new pending code.

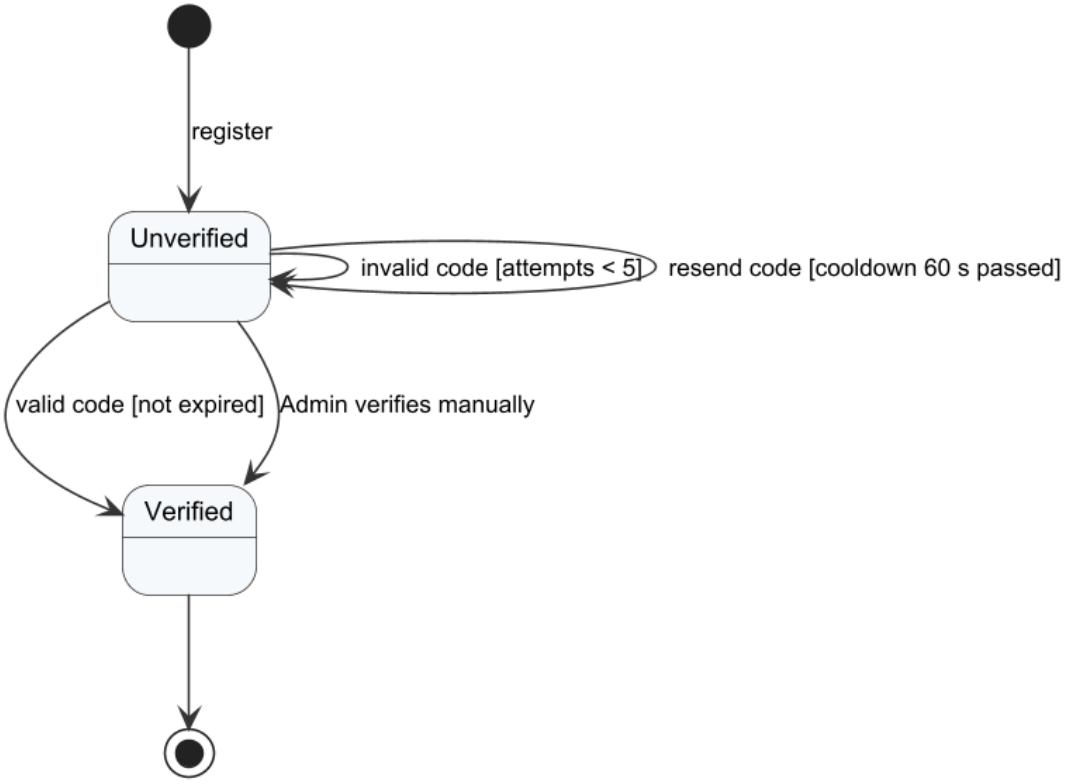


Figure 43. SM-02 E-mail verification code

SM-03 Training attempt and receipt

Briefing, practice, result; the receipt is APPLIED, DUPLICATE or REJECTED.

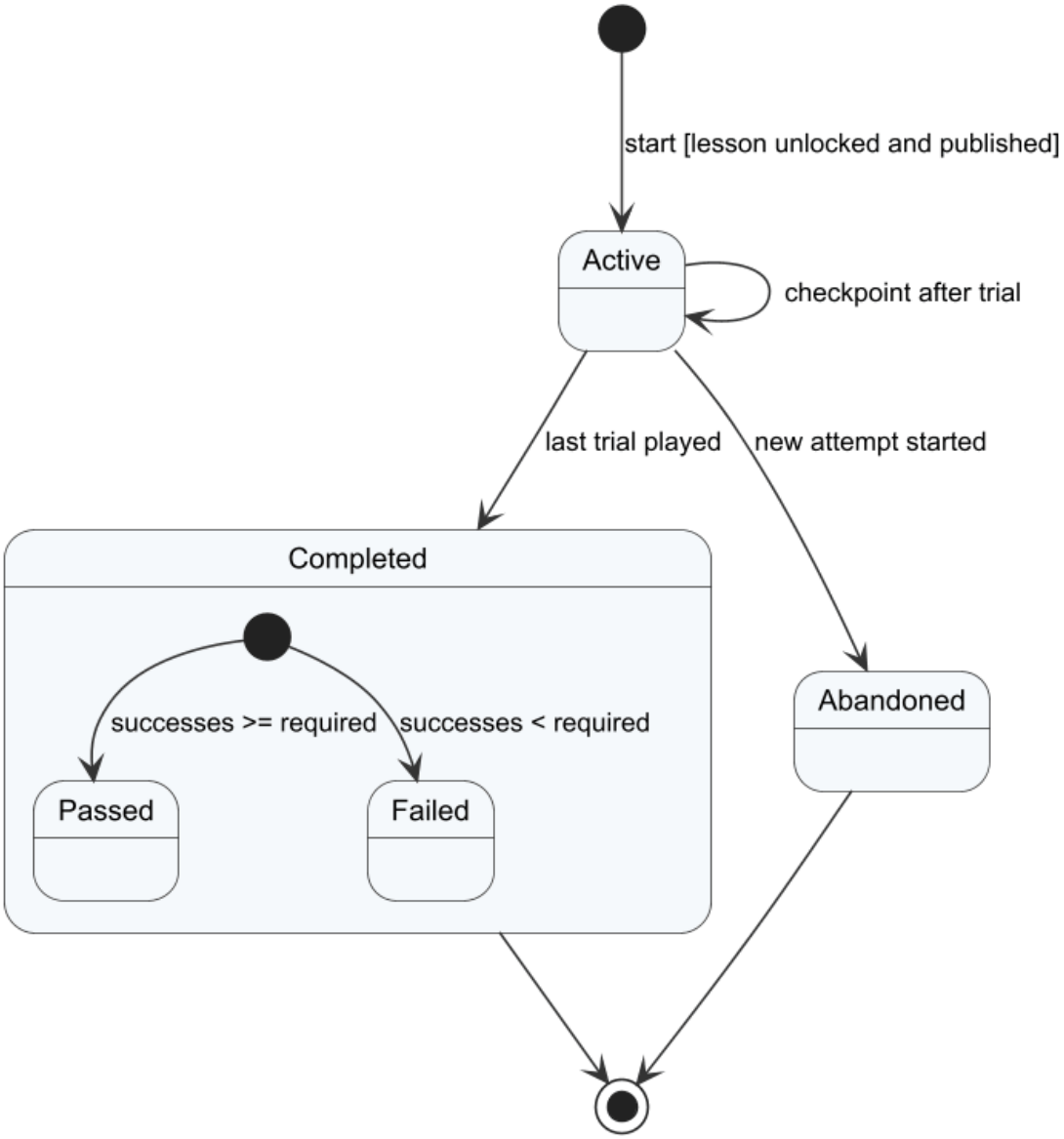


Figure 44. SM-03 Training attempt and receipt

SM-04 Offline queue item

Queued, sending, applied, duplicate or rejected; network failure returns the item to the queue.

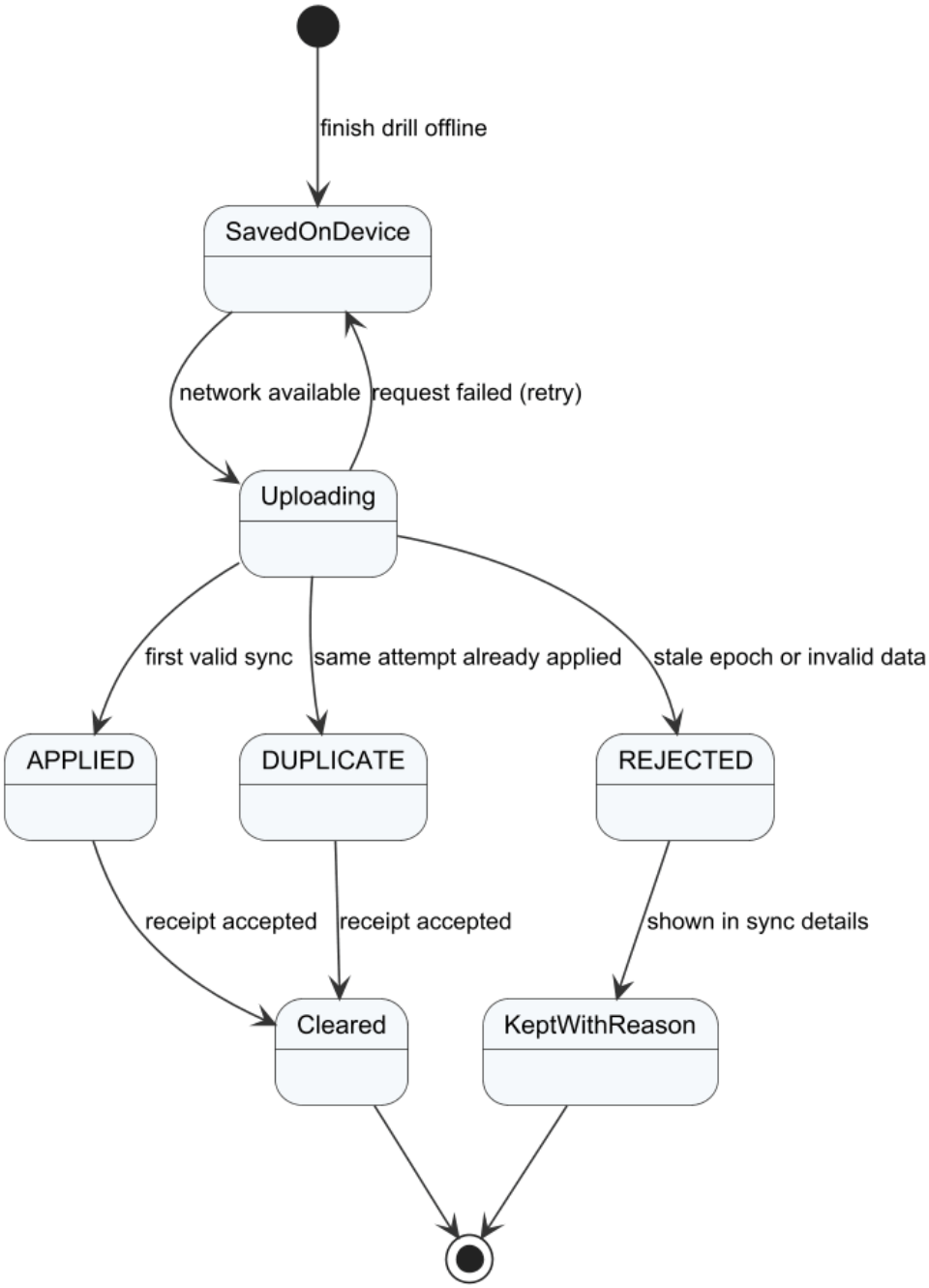


Figure 45. SM-04 Offline queue item

SM-05 Match phase (local match)

Intro, PreServe, Rally, Settling, BetweenPoints, BetweenGames, Timeout, MatchOver and Cinematic.



Figure 46. SM-05 Match phase (local match)

SM-06 Matchmaking search

SEARCHING ends in MATCHED, BOT (fallback), CANCELLED or TIMEOUT; a declined offer returns the other player to SEARCHING.

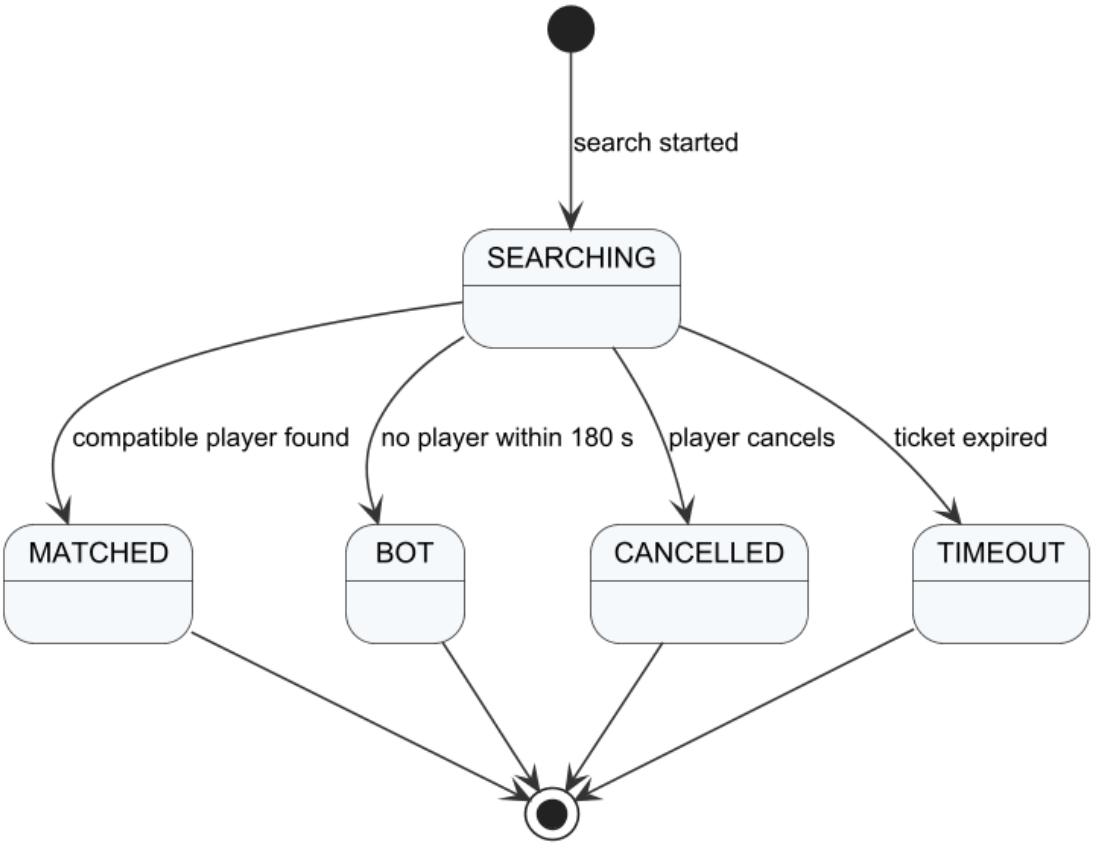


Figure 47. SM-06 Matchmaking search

SM-07 Online room phase

idle, searching, assigning-bot, ready, loading, playing, interrupted, closed and rejected.

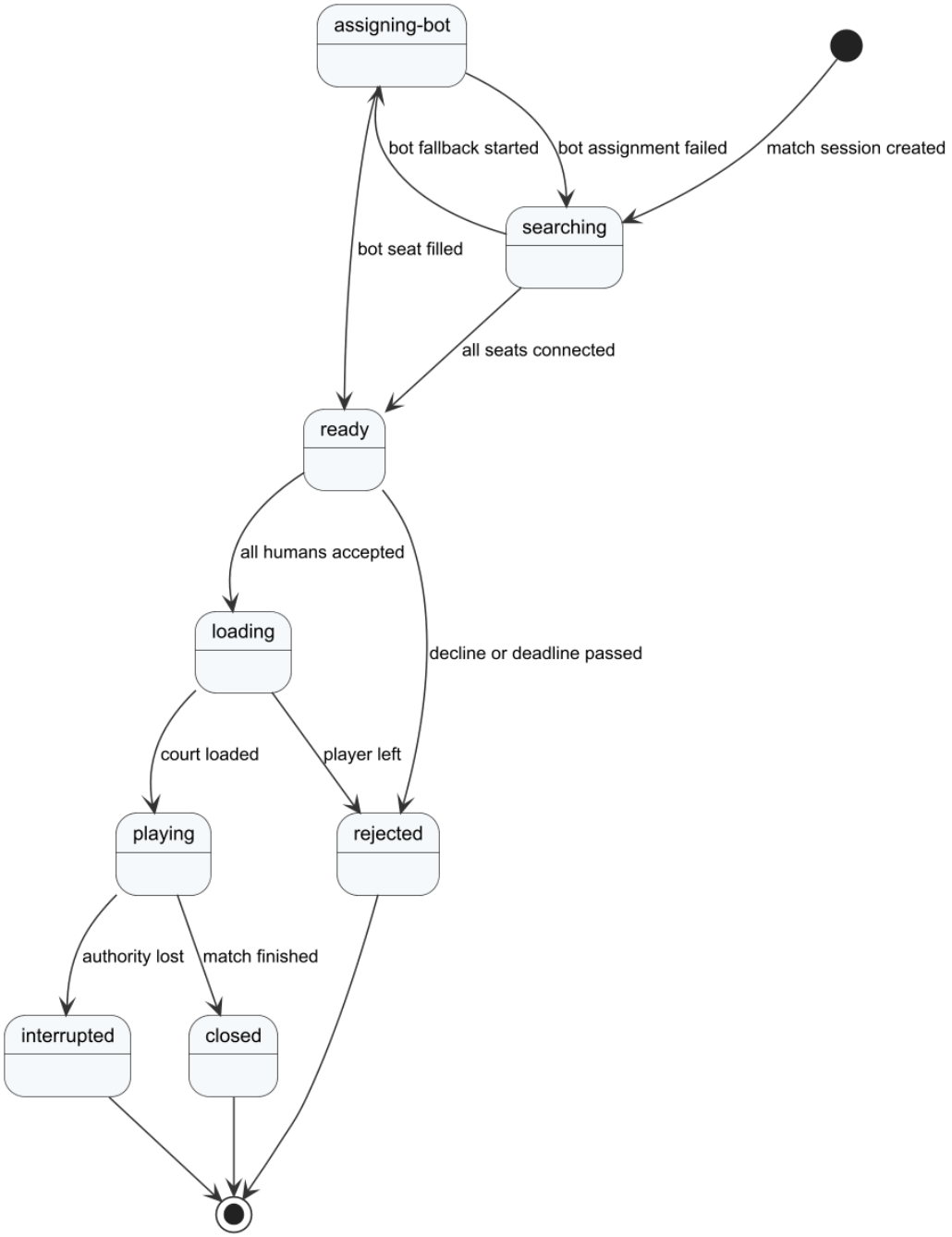


Figure 48. SM-07 Online room phase

SM-08 Game match

PREPARING, PLAYING, FINISHED and CANCELLED; interruption and administrator stop end the match without Elo.

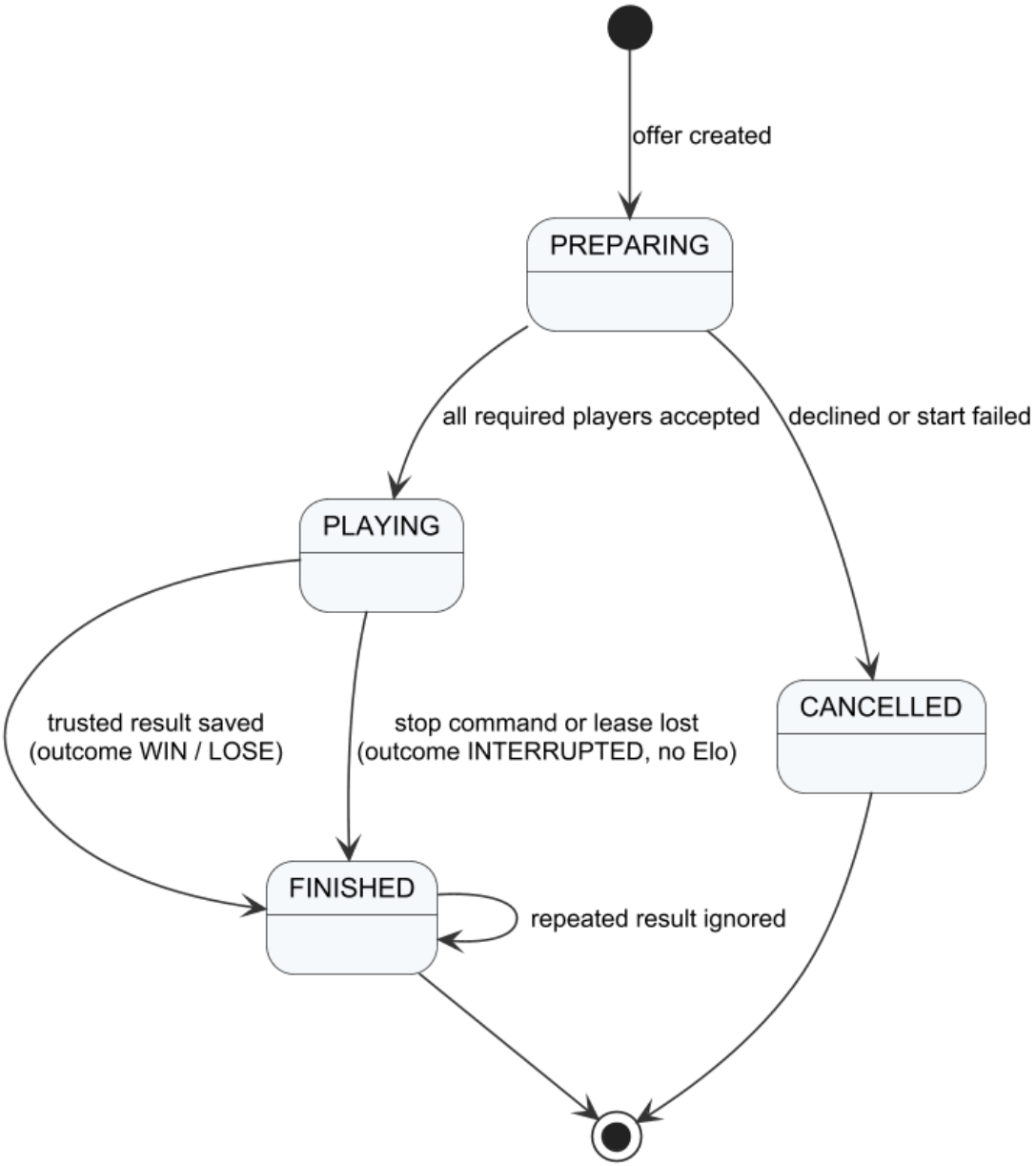


Figure 49. SM-08 Game match

SM-09 Dedicated server lease

queued, starting, ready, finishing, pending and busy; an expired lease releases the slot.

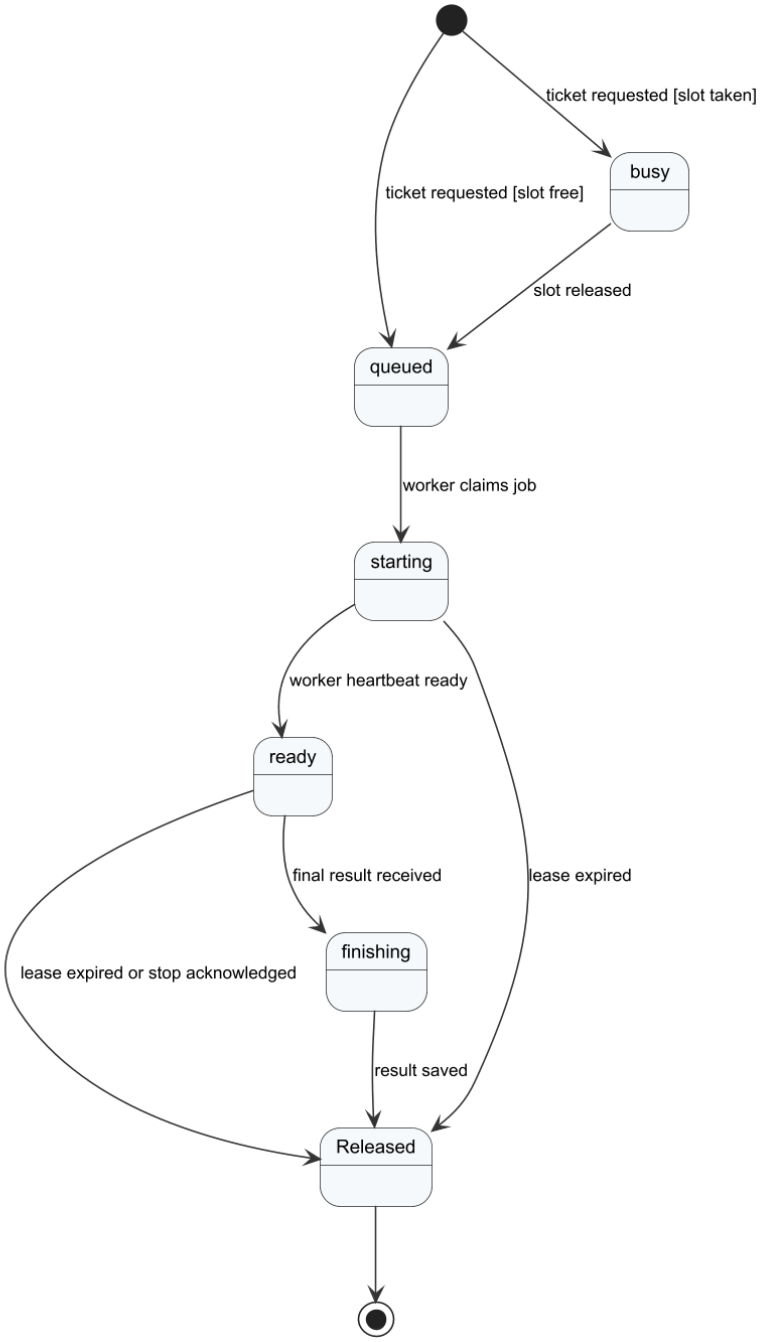


Figure 50. SM-09 Dedicated server lease

SM-10 Lesson publication

DRAFT, PUBLISHED and UNPUBLISHED.

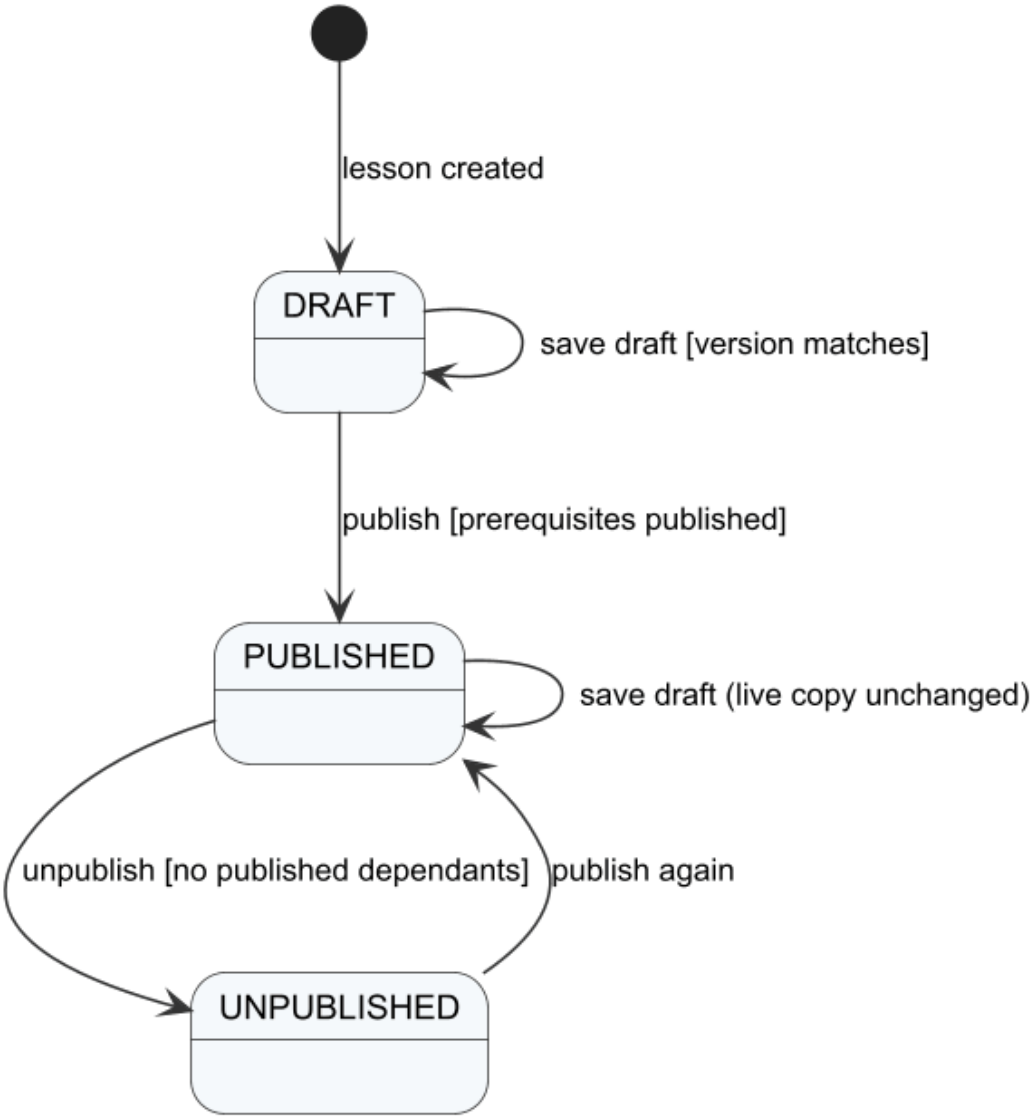


Figure 51. SM-10 Lesson publication

SM-11 Player report

PENDING, CHECKING, ACCEPTED and DECLINED.

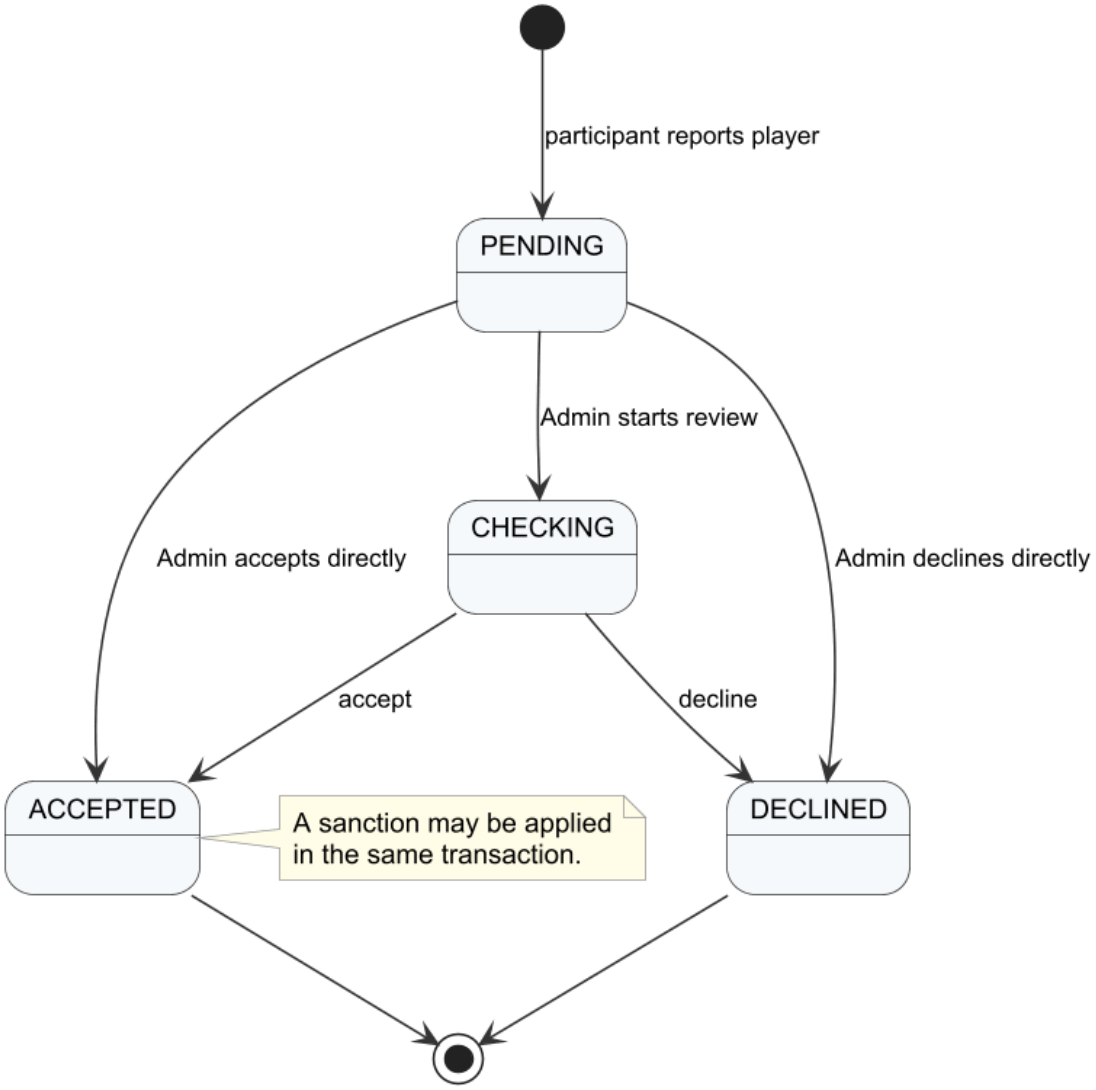


Figure 52. SM-11 Player report

SM-12 Sanction appeal

PENDING, CHECKING, ACCEPTED (sanction revoked) and DECLINED.

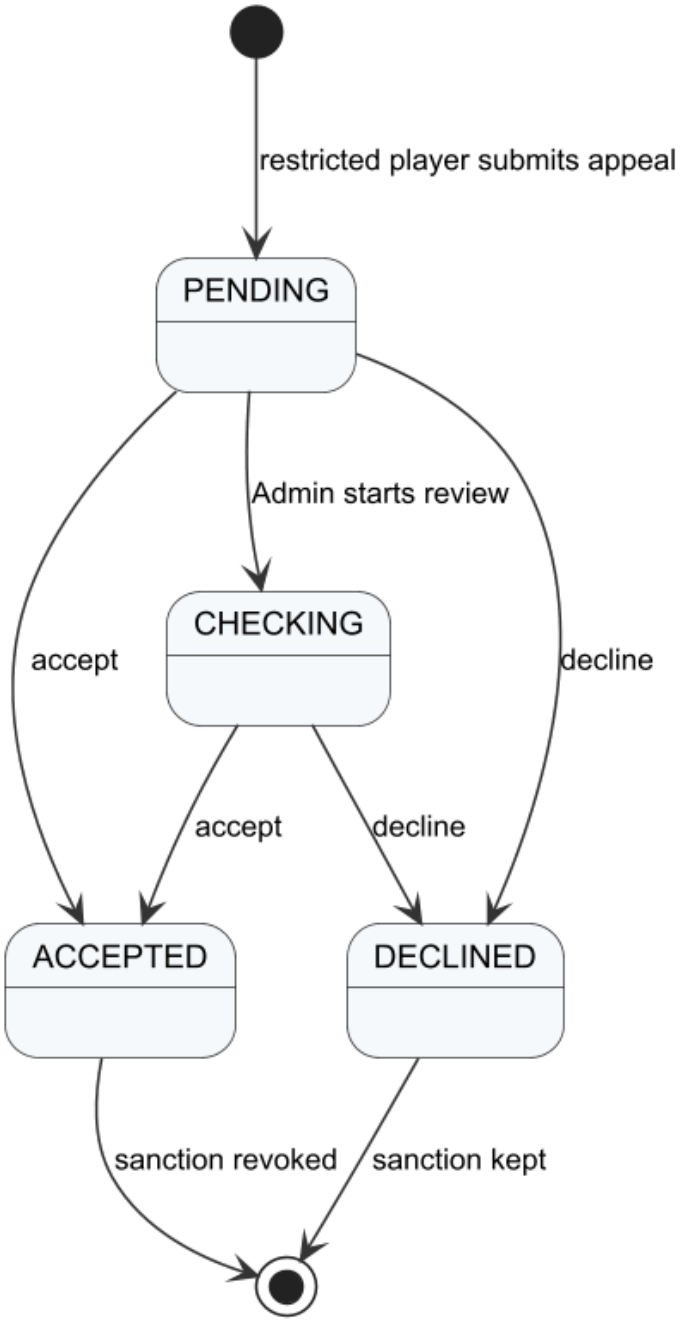


Figure 53. SM-12 Sanction appeal

SM-13 AI analysis job

pending, processing, completed and failed with retries.

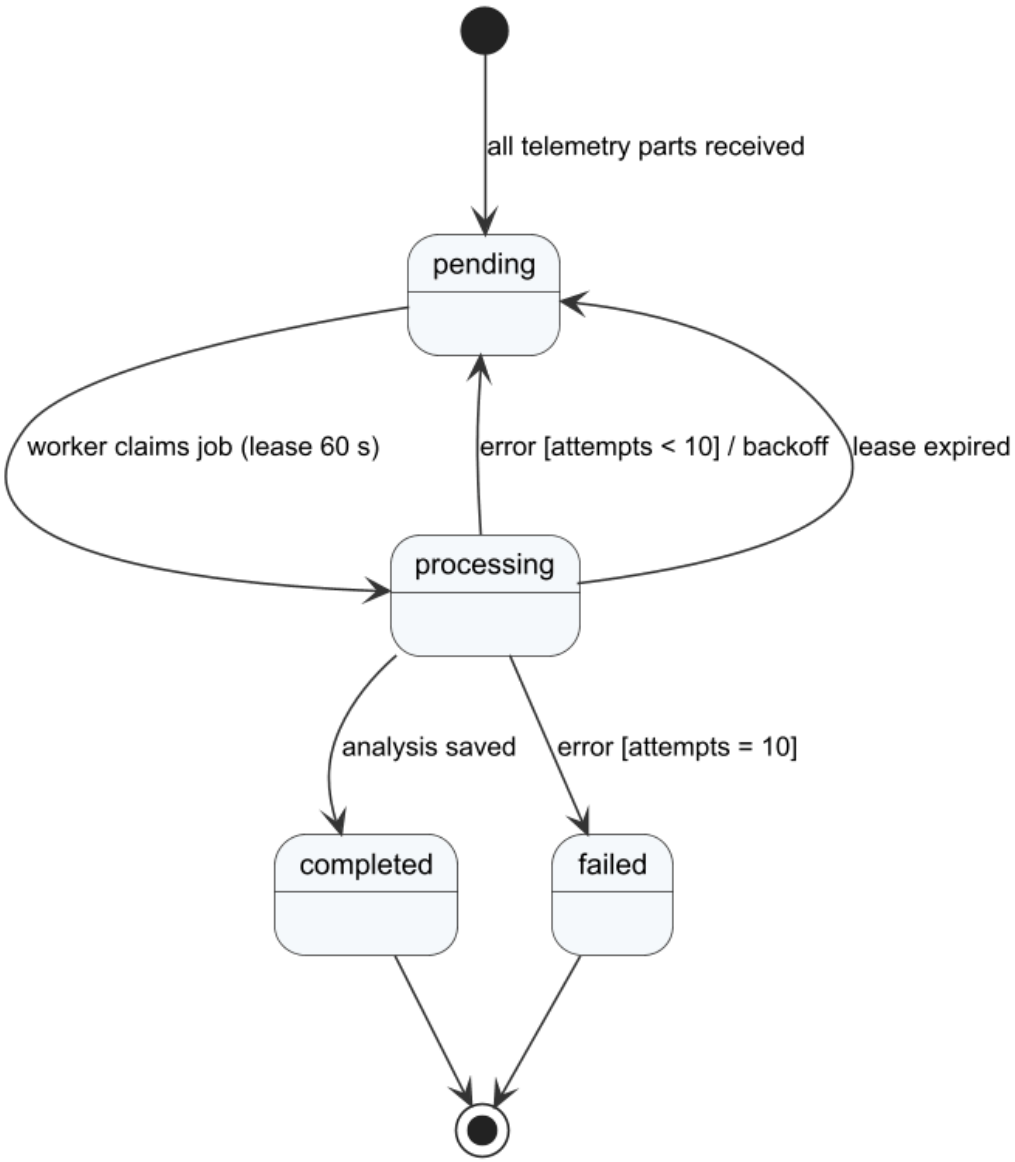


Figure 54. SM-13 AI analysis job

SM-14 Friend request

PENDING, ACCEPTED and DECLINED; blocking removes the friendship.

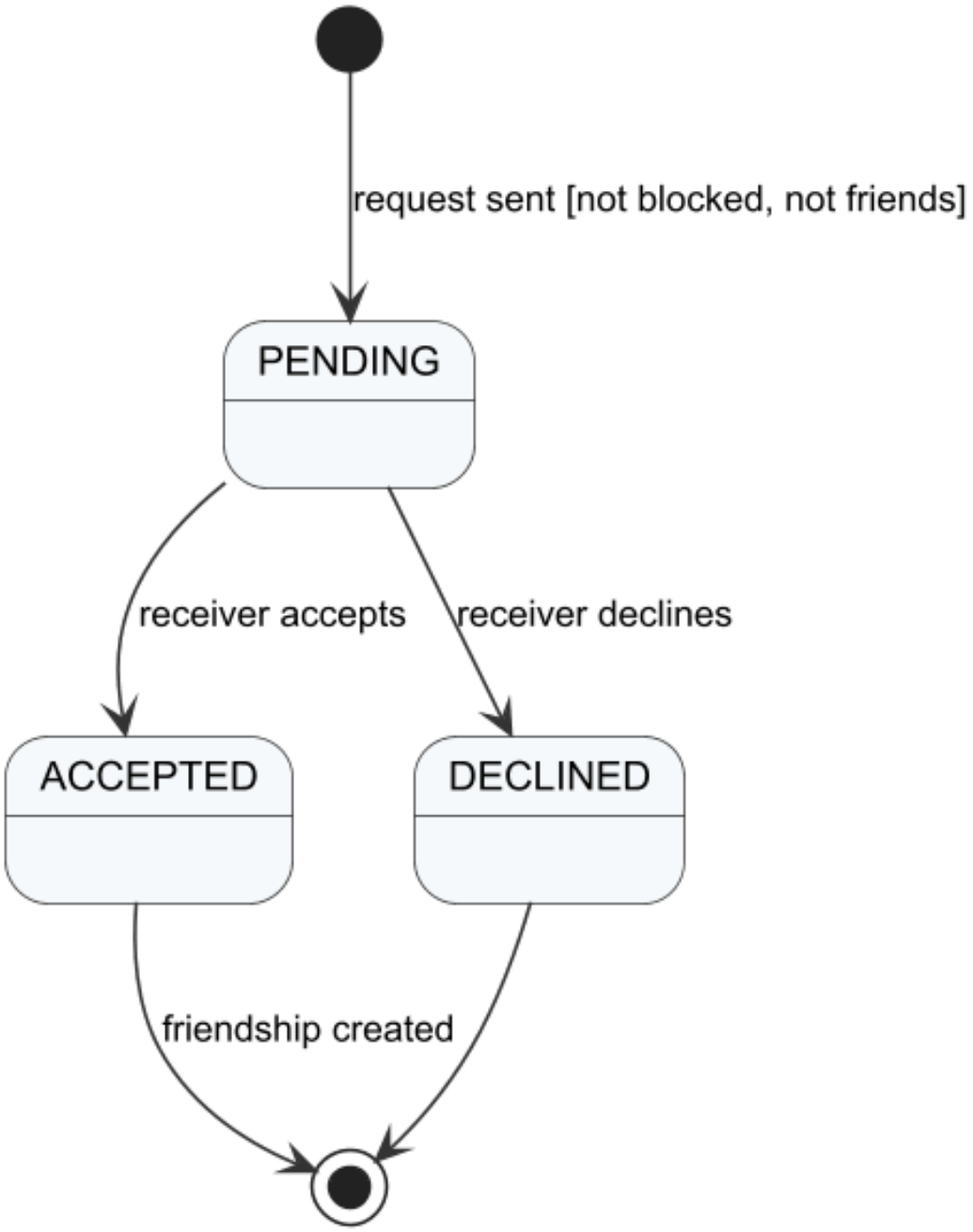


Figure 55. SM-14 Friend request